

Vue.js3 によるフロントエンド開発入門

WSC0134G-T1-02

～ 作って理解する SPA(TypeScript 対応)～

商標

- アマゾン ウェブ サービス、AWS および Amazon Web Services ロゴは、米国その他の諸国における、Amazon.com, Inc.またはその関連会社の商標です。
- Cisco、Cisco Systems、Cisco Systems ロゴは、Cisco Systems, Inc.および関連会社の米国ならびに他の国における商標もしくは登録商標です。
- Intel、インテルは、米国およびその他の国における Intel Corporation の商標です。
- ITIL[®]は AXELOS Limited の登録商標であり、AXELOS Limited の許可のもとに使用されています。すべての権利は留保されています。
- Microsoft、Windows は、米国 Microsoft Corporation の、米国およびその他の国における登録商標または商標です。
- Oracle と Java は、Oracle Corporation およびその子会社、関連会社の米国およびその他の国における登録商標です。
- PMI、PMP、PMBOK は、プロジェクトマネジメント協会(Project Management Institute, Inc.)の登録商標です。
- VMware、VMware ロゴは、VMware, Inc.の米国および各国での商標または登録商標です。
- BOOT CAMP、NEW TRAIN、MIX TRAIN、TRAINOCAMP は、トレノケート株式会社の登録商標です。

その他の会社名、製品名およびサービス名は、それぞれ各社の商標または登録商標です。

目次

はじめに	5
第 1 章 Vue3 環境構築 (Vite)	13
1.1. 開発環境の準備	16
1.2. Vite でプロジェクト作成.....	22
1.3. 依存ライブラリ導入.....	26
1.4. 開発サーバーの起動と動作確認	29
1.5. フォルダと主要ファイルの役割	33
章のまとめ	36
第 2 章 Vue の基礎知識.....	37
2.1. Vue とは	40
2.2. Vue コンポーネントの基本.....	42
2.3. リアクティブなデータ管理	46
2.3.1. ref()関数.....	47
2.3.2. reactive() 関数	52
2.3.3. computed()関数	56
2.4. ディレクティブ構文	60
2.4.1. v-bind ディレクティブ	61
2.4.2. v-if/v-else/v-show ディレクティブ	67
2.4.3. v-for ディレクティブ	73
2.5. イベント処理(v-on)	77
2.6. 双方向バインディング(v-model).....	83
章のまとめ	88
第 3 章 コンポーネント分割とデータの受け渡し.....	89
3.1. コンポーネントの分割	92
3.2. 親から子へ値を渡す (props)	98
3.3. 【参考】子から親へイベントを送る(emit).....	106
3.4. 複数の子コンポーネントを配置する.....	111

章のまとめ	117
第 4 章 ルーティングと SPA	119
4.1. Vue Router の導入	122
4.2. ページ(ルート)とコンポーネントの違い	128
4.3. ルート設定によるページ切り替え	130
4.4. ナビゲーションメニューの作成	135
4.5. provide と inject によるデータ共有	141
4.6. パラメータ付きルート	147
章のまとめ	153
第 5 章 【オプション】REST API 連携による CRUD 実装.....	155
5.1. REST API の役割と開発環境の全体像	158
5.2. REST API 環境構築	162
5.3. Express で検索機能の API 実装	170
5.3.1. 一覧画面の実装(REST API+Vue)	171
5.3.2. キーワード検索機能の実装(REST API+Vue).....	183
5.3.3. 詳細検索機能の実装	192
5.4. 登録処理の実装	199
5.5. 更新機能の実装	208
5.6. 削除機能の実装	218
章のまとめ	224

はじめに

コースの目標

コースの目標



- Viteで開発環境を構築・実行できる
- Vue 3の基本構文(v-bind, v-for, v-model, v-on, 条件分岐など)を理解して記述できる
- コンポーネントを分割し、データ連携ができる
- Vue Routerで画面遷移を実装できる

本コースを修了することで、以下のことができるようになります。

- Vite で開発環境を構築・実行できる
- Vue.js 3 の基本構文(v-bind, v-for, v-model, v-on, 条件分岐など)を理解して記述できる
- コンポーネントを分割し、データ連携ができる
- Vue Router で画面遷移を実装できる

コースの概要

コースの概要



第1章 Vue3環境構築 (Vite)

第2章 Vue基礎構文

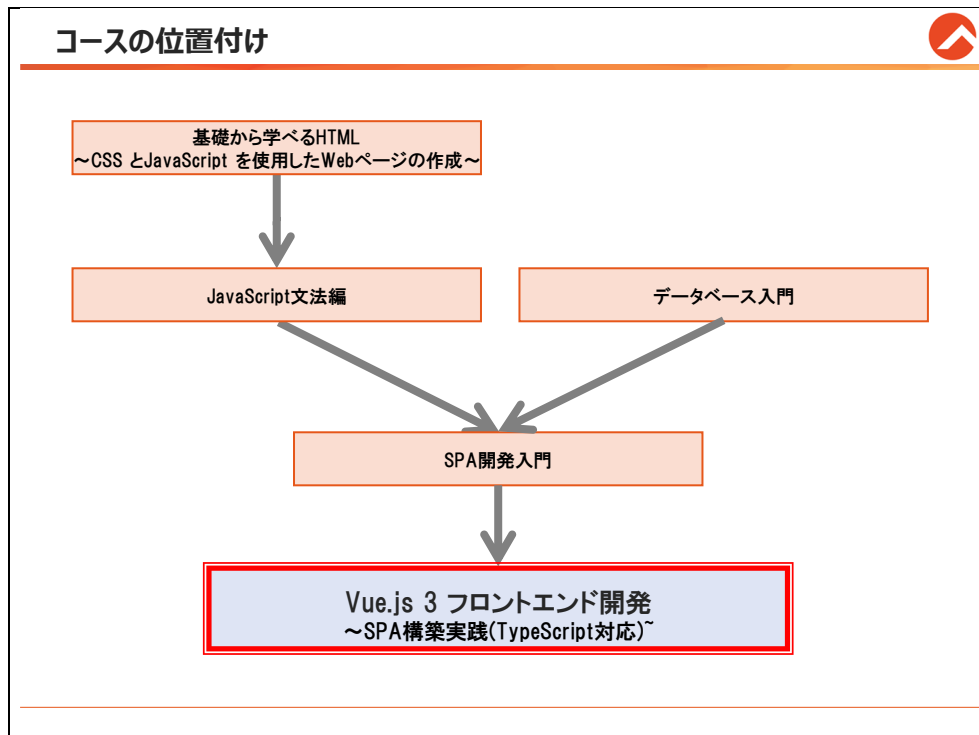
第3章 コンポーネント分割とデータの受け渡し

第4章 ルーティングとSPA

【オプション】第5章 REST API連携によるCRUD実装

-
1. Vue3 環境構築 (Vite)
 2. Vue 基礎構文
 3. コンポーネント分割とデータの受け渡し
 4. ルーティングと SPA
 5. 【オプション】REST API 連携による CRUD 実装

コースの位置付け



本コースでは、SPA 開発入門からさらにステップアップして、JavaScript のフレームワークである、「Vue 3」の習得を目指します。

このコースでは、以下の条件を満たしていることを前提としています。

- 「データベース入門」を受講しているか、
SQL(SELECT/INSERT/UPDATE/DELETE)の基本構文を知っている
- 「基礎から学べる HTML」を受講しているか、
HTML を使って簡単な Web ページを作成できる
- JavaScript の基礎文法を知っている
- Web アプリケーションの開発ができる
- Bootstrap の基礎を知っている

テキストの表記

■ 画面に表示される文字とキーボードキー

メニュー・コマンド・ダイアログボックス等画面に表示される文字とキーボードキーは、
[]で囲んで表記しています。

表記例

画面に表示される文字	[プロパティ]を選択します
キーボードキー	[Ctrl]・[Tab]・[Caps Lock]・[↑]・[→]

■ コンソール上の英字やコード・コマンド・パラメータ

コンソール上の英字やコード・コマンド・パラメータは、Courier New のフォントで表記
しています。

表記例

	Vlan Name	Status	Ports
1	default	Enabled	3-12,AUI,A
10	test1	Enabled	1

■ ユーザが任意に指定する値(コード・コマンド・パラメータ)

ユーザが任意に指定する値は、*Courier New Bold italic* のフォントで表記していま
す。

表記例

C:\¥AppSource> csc /debug+ /out: <i>MyApp.exe</i> <i>MyApp.cs</i>

If <i>command1</i> returns success, <i>command2</i> runs
\$ ls -l <i>sample.txt</i>

■ UNIX と Linux 関連のテキストでの表記

ユーザが任意に指定する値

ユーザが任意に指定する値は *Times New Roman Italic* のフォントで表示しています。

表記例

```
more [file1 [file2...]]  
ls -l file
```

■ 重要な語句や強調

本文中の重要な語句や強調は**太字**で表示しています。

表記例

```
この認証プロセスのことをログオンといいます。
```


第1章 Vue3 環境構築 (Vite)

章の目標

- Vite で Vue3 の開発環境を作成できる
- Vue3 プロジェクトのファイル構成を理解できる

章の概要

章の概要



- 開発環境の準備
- Viteでプロジェクト作成
- 依存ライブラリ導入
- 開発サーバーの起動と動作確認
- フォルダと主要ファイルの役割

この章では、Vue.js 3 の開発環境を最短手順で整え、実際に動かすところまでを進めます。

まず、Visual Studio Code (以降 VSCode) と Node.js を用意し、必要な拡張機能を入れます。

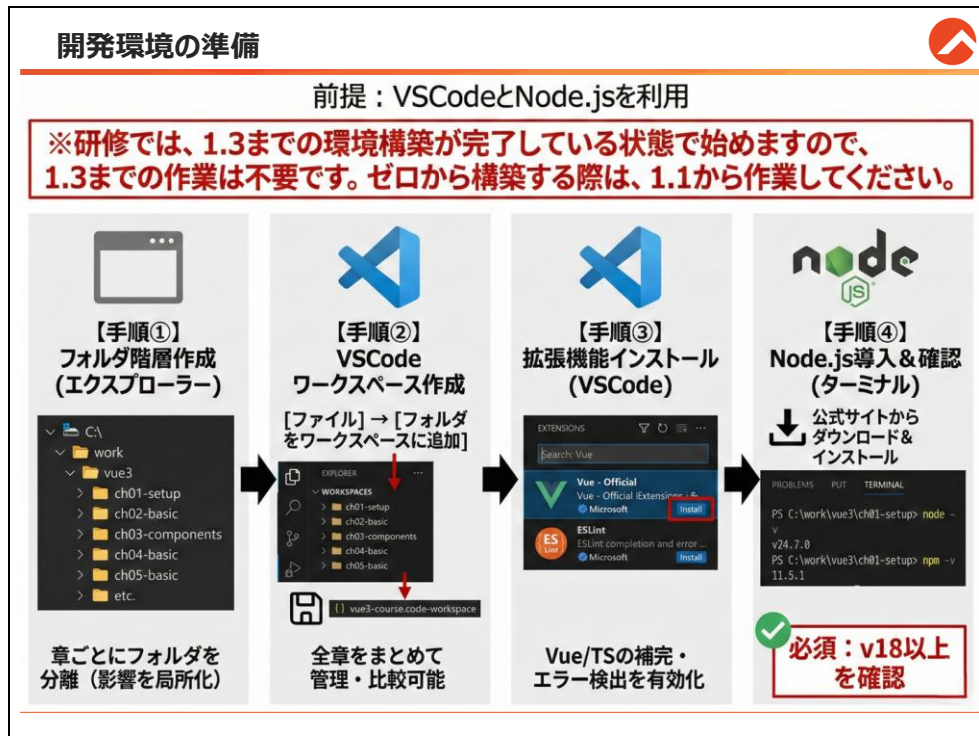
次に、Vite で新規プロジェクトを作成し、npm で依存ライブラリ (Bootstrap の CSS など) を導入します。

続いて、生成されたフォルダと主要ファイルを確認し、index.html → main.ts → App.vue の流れをつかみます。

最後に、開発サーバーを起動して初期画面を表示し、TypeScript の型チェックと Bootstrap のスタイル適用を確かめます。

章の終わりには、Vite の基本操作とプロジェクト構成を自分の言葉で説明できる状態になります。

1.1. 開発環境の準備



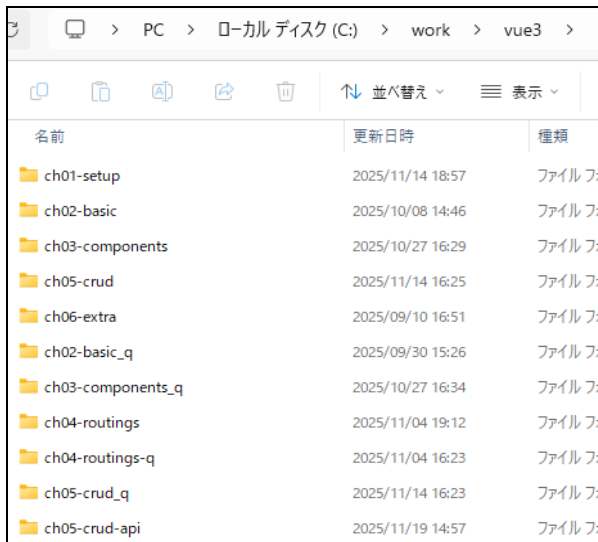
Vue.js 3 の環境構築のための準備

本書では、Vue.js 3 (以降、Vue3 と称する) の環境構築を行う前に、以下の事前準備を行います。

1. 章ごとに空のフォルダを作成しておく
2. 開発ツールの VSCode から、ワークスペースを作成し、各章のフォルダを読み込む
3. VSCode に「Vue」プラグインを導入
4. PC に Node.js を導入

【作業手順①】 親フォルダと章フォルダを作る(エクスプローラー)

1. エクスプローラーを開く
2. C:¥ に **work** フォルダがなければ作成 → その中に **vue3** フォルダを作成
 - 最終パス:C:¥work¥vue3



3. C:¥work¥vue3 の中に、章ごとの空フォルダを作る
 - 例:ch01-setup, ch02-basic, ch03-components, ch04-routings, ch05-crud

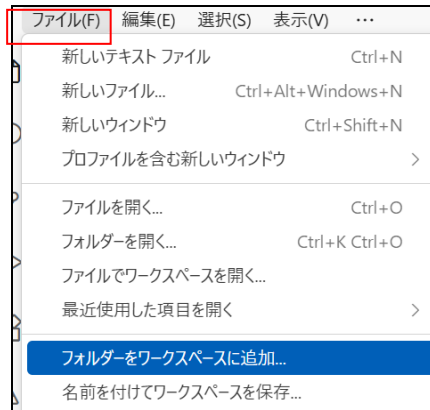
本研修では、最初からこのように章ごとのフォルダを用意してあります。

フォルダを分けることで、エラーや不具合があっても他の章には影響が及ばないようにするためです。

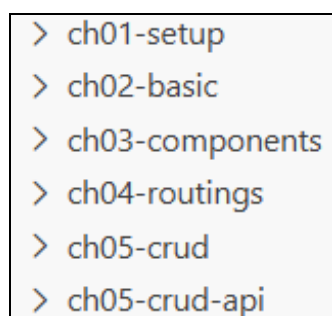
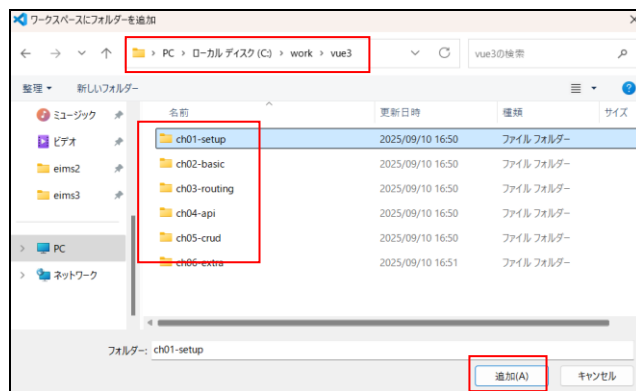
(Java プログラミング時に Eclipse でプロジェクトを分けて管理するのと同じ考え方です。)

【作業手順②】VSCode でマルチルート・ワークスペースを作る

1. VSCode を起動する

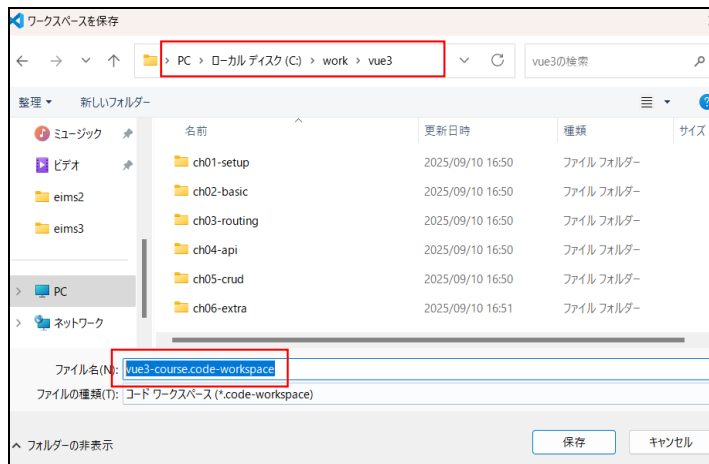


2. メニュー [ファイル] → [フォルダをワークスペースに追加] を選び、
C:\work\vue3\ch01-setup を追加する(同様に ch02-... ~ ch05-... も追加)



3. メニュー [ファイル] → [名前を付けてワークスペースを保存] を選び、
C:\work\vue3\vue3-course.code-workspace として保存する

第1章 Vue3 環境構築 (Vite)



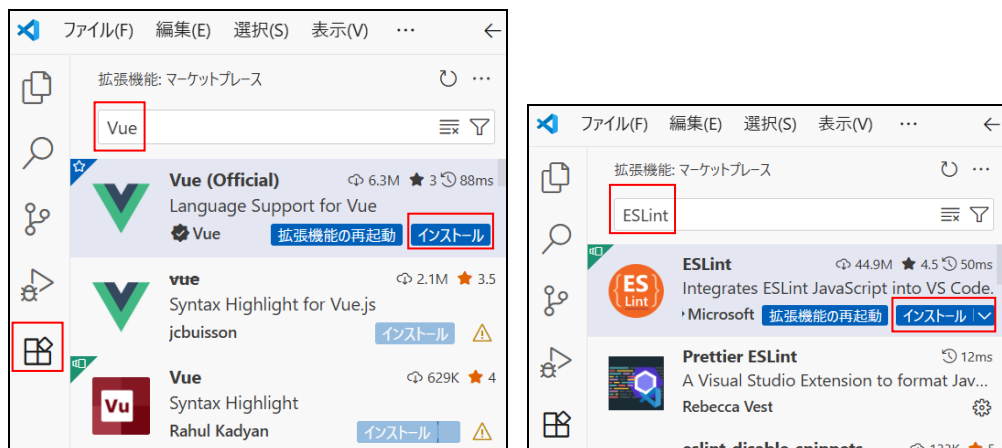
4. 以後は、このワークスペースを選ぶと、**全章のコードが並んで表示**される

この作業は、VSCode で複数の章フォルダを一度に開けるようにするためです。1 つの画面で全章のコードを見比べながら学習できるようになりますので、章ごとにプロジェクトを切り替える必要がなくなります。

【作業手順③】VSCode 拡張機能を入れる

VSCode は、基本的にメモ帳のようなテキスト編集ツールですので、Vue3 や TypeScript のソースコードを理解できません。追加のプラグインのインストールが必要です。

1. 左の拡張機能アイコンを開き、「Vue」と「ESLint」と検索する



2. **Vue(Official)**と、**ESLint** をインストールする

インストールが終わると、VSCode 上で Vue 3 + TypeScript の補完やエラー検出ができるようになります。

【作業手順④】 Node.js のインストール&バージョン確認

Vite や Vue の開発ツールは、Node.js を土台に動きますので、インストールが必要です。

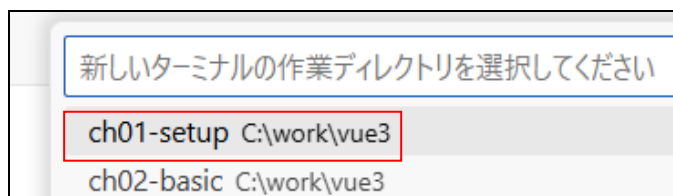
1. Node.js をダウンロード&インストールする

<https://nodejs.org/ja/download>



2. VSCode のメニュー [ターミナル] → [新しいターミナル] → [ch01-setup]を開く

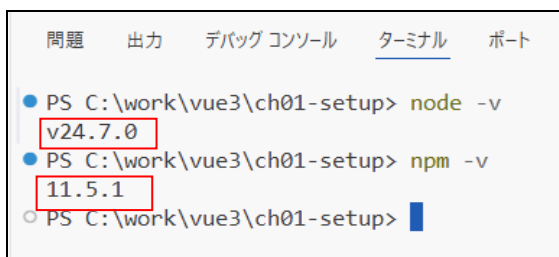
※Node.jsを操作するためには、Windowsのコマンドプロンプトと同じようなコマンド欄を利用します。Windowsのコマンドプロンプトや、PowerShellを利用しても良いですが、以下のように、**VSCode 上の「ターミナル」**で行うと便利です。



3. 画面下段に、ターミナルが表示されるので、次の 2 つを入力してバージョンが表示されれば OK です。バージョンを確認するのは「正しい環境で実行できるか」を確かめるためです。

```
node -v  
npm -v
```

node -v のバージョンが、v18 以上なら、問題ありません。(2025 年 9 月時点)

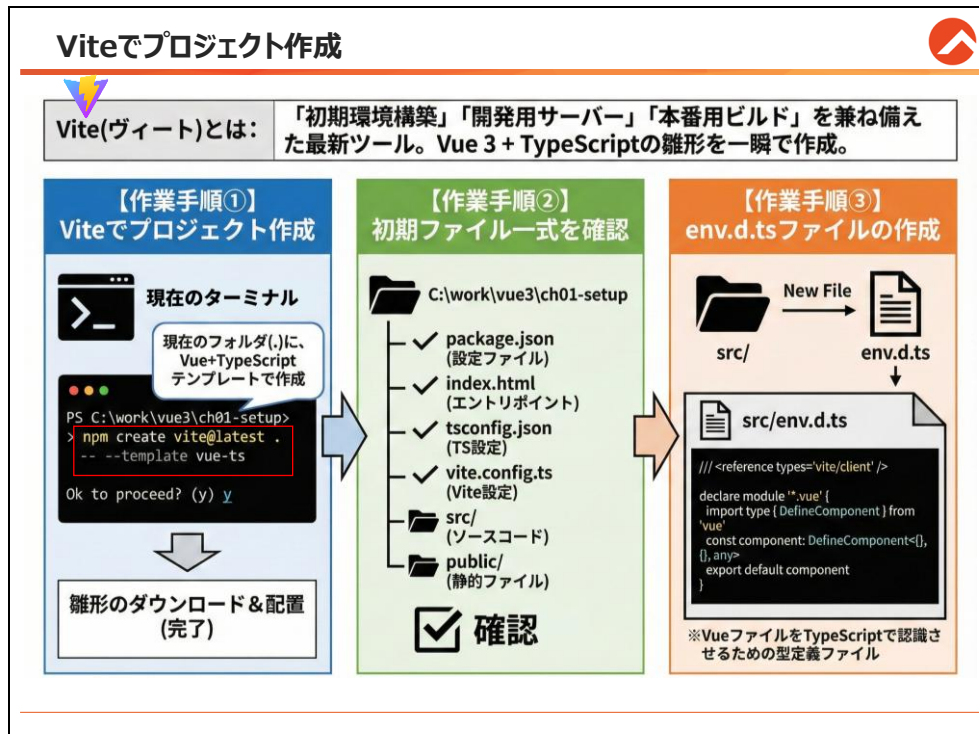


The screenshot shows a terminal window with the following content:

```
問題 出力 デバッグ コンソール ターミナル ポート  
● PS C:\work\vue3\ch01-setup> node -v  
v24.7.0  
● PS C:\work\vue3\ch01-setup> npm -v  
11.5.1  
○ PS C:\work\vue3\ch01-setup> █
```

The versions v24.7.0 and 11.5.1 are highlighted with red boxes.

1.2. Vite でプロジェクト作成



Vite とは

Vite(ヴィート)は「初期環境構築」と「開発用サーバー」と「本番用ビルド」を兼ね備えた最新のツールです。これで **Vue 3 + TypeScript** の雛形を一瞬で作成できます。

【作業手順①】 Vite でプロジェクト作成

まず、コマンドを利用し、Vue3 プロジェクトの基本テンプレートを指定フォルダにダウンロードしましょう。

Vite を操作する際は、Node.js を経由しますので、Node.js と同様に VSCode 上のターミナルを使い操作します。

ターミナルが「PS C:\work\vue3\ch01-setup>」の状態になっていることを確認し、以下の作業手順に従ってコマンドを入力します。

1. Vite テンプレートを現在のフォルダに作成

(※ドット . は「このフォルダに作る」の意味)

```
npm create vite@latest . -- --template vue-ts
```

--template vue-ts を指定することで、最初から TypeScript 対応の状態になります。

2. 何か聞かれたら「y」を入力する

既存ファイルは空なので上書きしても問題ありません。

```

問題 出力 デバッグ コンソール ターミナル ポート

● PS C:\work\vue3\ch01-setup> node -v
v24.7.0
● PS C:\work\vue3\ch01-setup> npm -v
11.5.1
○ PS C:\work\vue3\ch01-setup> npm create vite@latest . -- --template vue-ts
Need to install the following packages:
create-vite@7.1.1
Ok to proceed? (y) y

```

3. 以下のようなメッセージが出たら、そのまま「Enter」

```

> create-vite . --template vue-ts

|
|
| Use rolldown-vite (Experimental)?
| ○ Yes
| ● No
|

```

4. 以下のようなメッセージが出たら、初回は方向キーを動かして、「No」にして「Enter」
2 回目以降のプロジェクト作成なら、Yes のまま「Enter」

```

|
|
| Install with npm and start now?
| ● Yes / ○ No
|

```

```

問題 出力 デバッグ コンソール ターミナル ポート

> npx
> create-vite . --template vue-ts

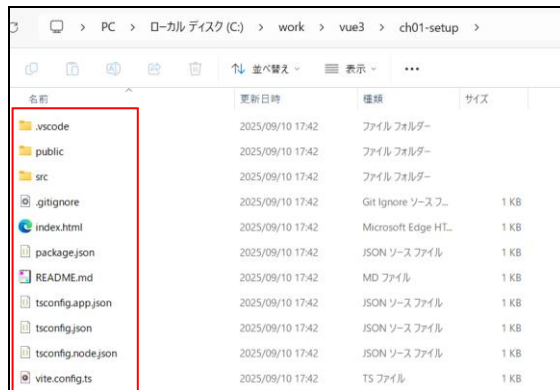
|
|
| Scaffolding project in C:\work\vue3\ch01-setup...
|
| Done. Now run:
|
| npm install
| npm run dev
○ PS C:\work\vue3\ch01-setup>

```

しばらく待つと、Vue3 プロジェクトの雛形がダウンロード&配置されます。

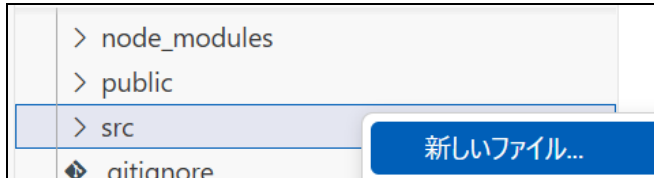
【作業手順②】 生成されたプロジェクトの初期ファイル式を確認

C:\work\vue3\ch01-setup のエクスプローラーに package.json, index.html, tsconfig.json, vite.config.ts, src が確認できれば OK です。



【作業手順③】 env.d.ts ファイルの作成

このファイルは、環境変数ファイルですが、Vite のバージョンによっては、自動で生成されないため手動で作成しておく必要があります。本研修では、vue ファイルを複数使うため、このファイルが必須です。



src フォルダを右クリックし、「新しいファイル」をクリックします。

その後、「**env.d.ts**」ファイルを作成してください。

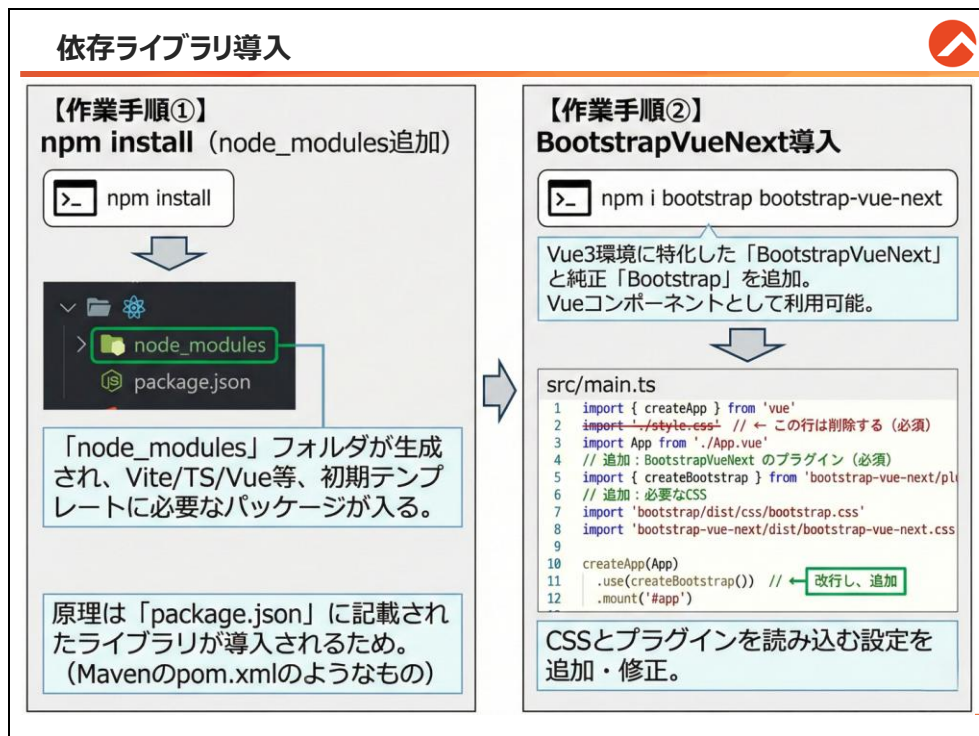
以下の内容を入力します。

C:¥work¥vue3¥ch01-setup¥src¥env.d.ts

```
/// <reference types="vite/client" />
declare module '*.vue' {
  import type { DefineComponent } from 'vue'
  const component: DefineComponent<{}, {}, any>
  export default component
}
```

この設定をしないと、App.vue 以外の vue ファイルを作成時にエラーが起きる可能性があります。

1.3. 依存ライブラリ導入



依存ライブラリ導入

「create vite」コマンドにより、Vue3 プロジェクトの雛形は作成されたものの、それだけでは不十分です。Vite プロジェクトを動かすためのライブラリー式もダウンロードする必要があります。

【作業手順①】 npm install (node_modules 追加)

1. 引き続きターミナル上に、「npm install」を入力します。

```

問題 出力 デバッグ コンソール ターミナル ポート

npm install
npm run dev

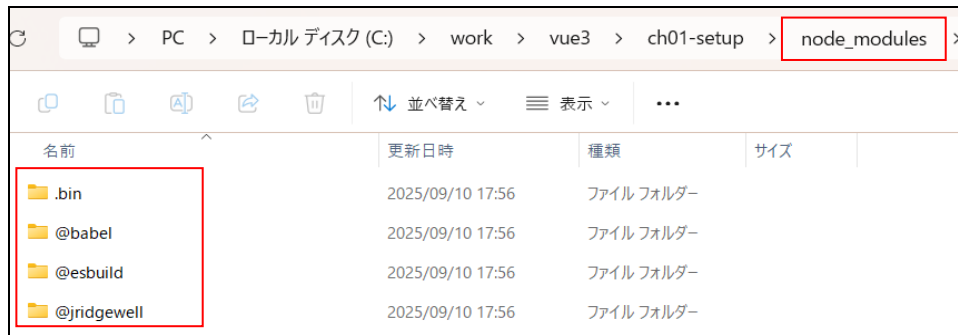
● PS C:\work\vue3\ch01-setup> npm install

added 94 packages, and audited 95 packages in 9s

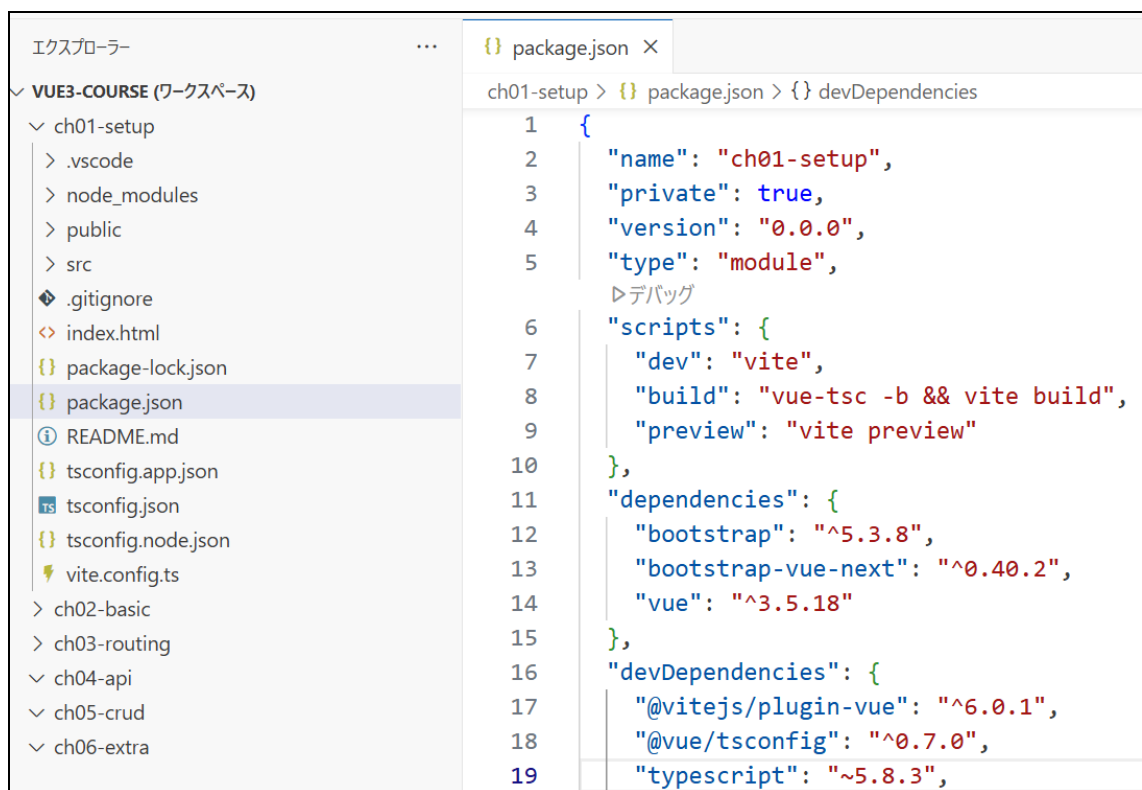
6 packages are looking for funding
  run `npm fund` for details

found 0 vulnerabilities
○ PS C:\work\vue3\ch01-setup>

```



2. 「node_modules」フォルダが生成され、Vite/TypeScript/Vue など、初期テンプレートに必要なパッケージが入ります。その原理は、vite create コマンドによって生成された、「package.json」のおかげです。



package.json の中に、Vite で環境構築する際の必要ライブラリが記載されていますので、「npm install」するだけで、適切なライブラリが導入されたわけです。

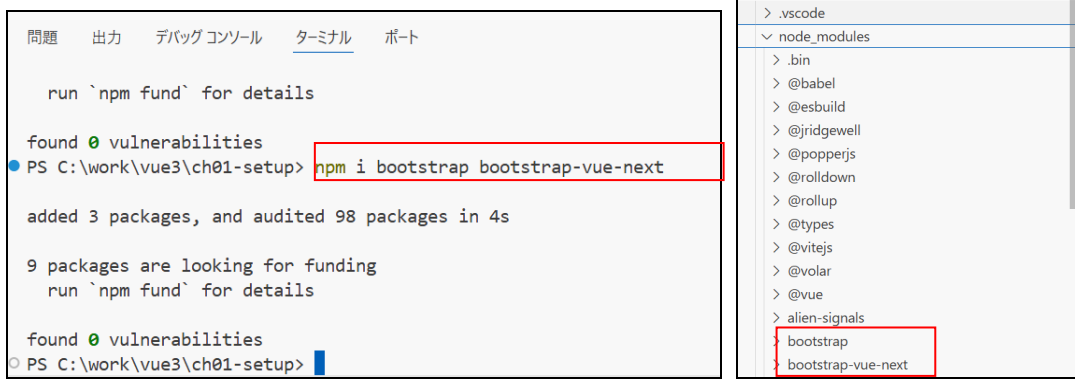
【作業手順②】 BootstrapVueNext の依存ライブラリ導入

本研修では、簡単に UI デザインができる Bootstrap を利用しますが、純正の Bootstrap のみでは、Vue3 内の JavaScript と競合する恐れがあります。

そのため、Vue3 環境に特化した、「**BootstrapVueNext**」を導入します。BootstrapVueNext なら Vue コンポーネントとして自然に組み込めます。

1. Bootstrap と BootstrapVueNext を追加します。

```
npm i bootstrap bootstrap-vue-next
```



2. CSS とプラグインを読み込む (src/main.ts)

以下のように追記・修正します。

C:\work\vue3\ch01-setup\src\main.ts

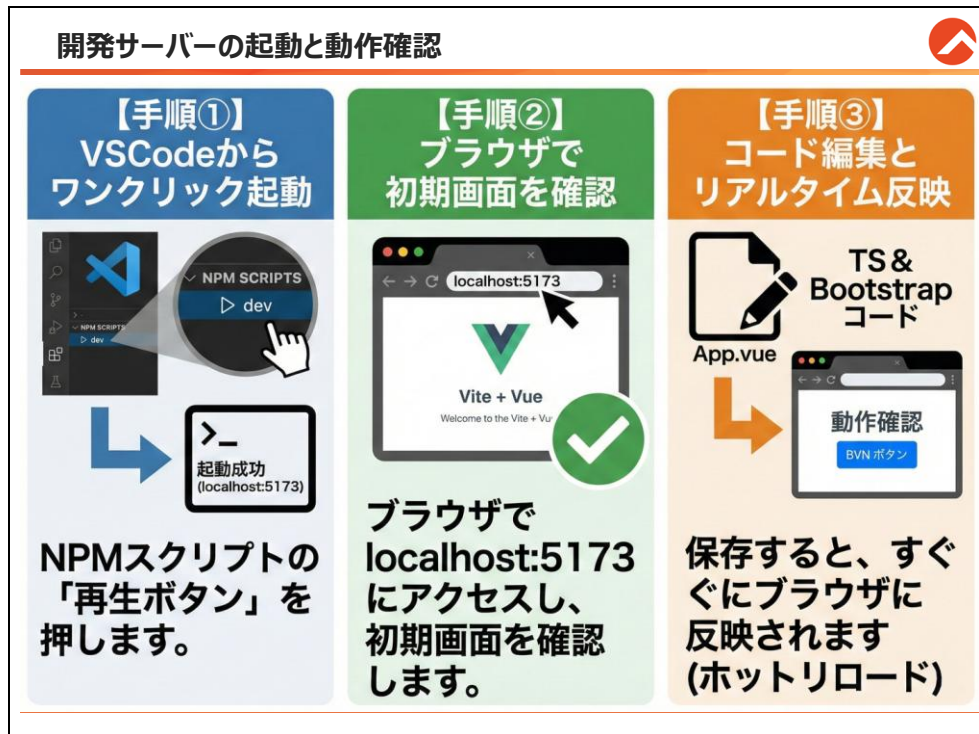
```
import { createApp } from 'vue'
import './style.css' // この行は削除する (必須)
import App from './App.vue'

// 追加 : BootstrapVueNext のプラグイン (必須)
import { createBootstrap } from
'bootstrap-vue-next/plugins/createBootstrap'

// 追加 : 必要な CSS
import 'bootstrap/dist/css/bootstrap.css'
import 'bootstrap-vue-next/dist/bootstrap-vue-next.css'

createApp(App)
  .use(createBootstrap()) // 改行し、追加
  .mount('#app')
```

1.4. 開発サーバーの起動と動作確認



Vue3 プロジェクトをローカルで起動する方法

Vue3 プロジェクトの動作確認をするためには、ローカルの開発サーバーを起動する必要があります。ローカルの開発サーバーは、環境構築の時に利用していた、「Vite」を使います。

【作業手順①】VSCode の左サイドバーの Npm Script を表示

Vite はコマンド入力で操作することもできますが、「Npm Script」を利用して、より簡単に操作することができます。

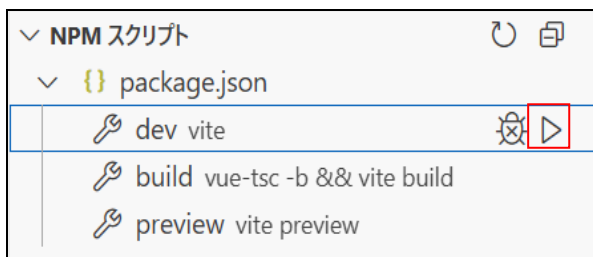


「…」をクリックし、「Npm スクリプト」にチェックを入れてください。



VSCode の左サイドバーに、NPM スクリプトメニューが表示されました。事前に Vite の環境構築が完了していますので、package.json に記載されているコマンドも認識されるようになっています。

【作業手順②】開発サーバーを起動



Npm Script メニューから、「dev」行の「▶」ボタンをクリックすると、自動でサーバー実行コマンドが入力され、Vite ローカルサーバーが起動します。



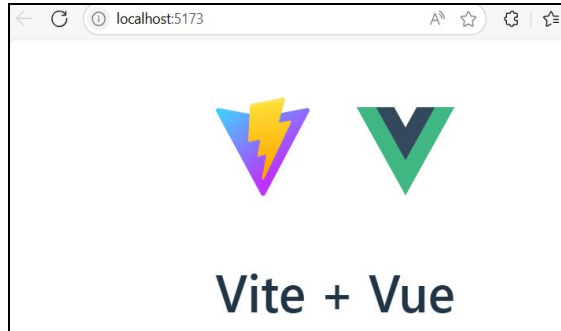
※「npm run dev」をターミナルに直接入力しても、同様の操作ができます。

この Vite サーバーは、ホットリロードサーバーです。つまり、コードを保存すると即座にブラウザに反映されるため、開発効率が大幅に上がります。

【作業手順③】ブラウザで初期起動確認

サーバーが起動すると、ブラウザから接続できるようになります。

`http://localhost:5173` をブラウザから開いてください。



このように表示されると、Vite で生成した Vue3 の初期開発環境が無事に生成され、正常に動いていることが確認できます。

※ポート番号は、5173 がデフォルトですが、サーバーを追加で起動すると、:5174、:5175 のように割り当てられます。

【作業手順④】TypeScript + Bootstrap の導入確認

Vue3 と同時に導入した TypeScript と Bootstrap も正常に動作するのか、ソースコードを修正しながら確認を行います。

一般的な Web アプリケーションの場合、<http://localhost:5173/> の URL を入力すると、ウェルカムページとして設定されている「index.html」が初期画面になることが多いです。しかし、Vue3 では、初期画面のソースコードを index.html に書きません。

Vue3 では、index.html に直接ソースコードを記載するのではなく、「App.vue」ファイルに記載します。詳細は次の節でご説明します。

では、`src/App.vue` の既存のソースコードを削除し、以下のソースコードに置き換えましょう。

C:\work\vue3\ch01-setup\src\App.vue

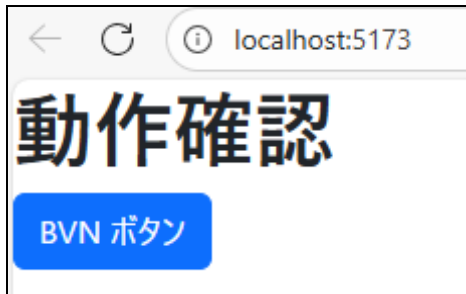
```
<script setup lang="ts">
const pageTitle: string = '動作確認'
</script>

<template>
  <h1>{{ pageTitle }}</h1>
  <button class="btn btn-primary">BVN ボタン</button>
</template>
```

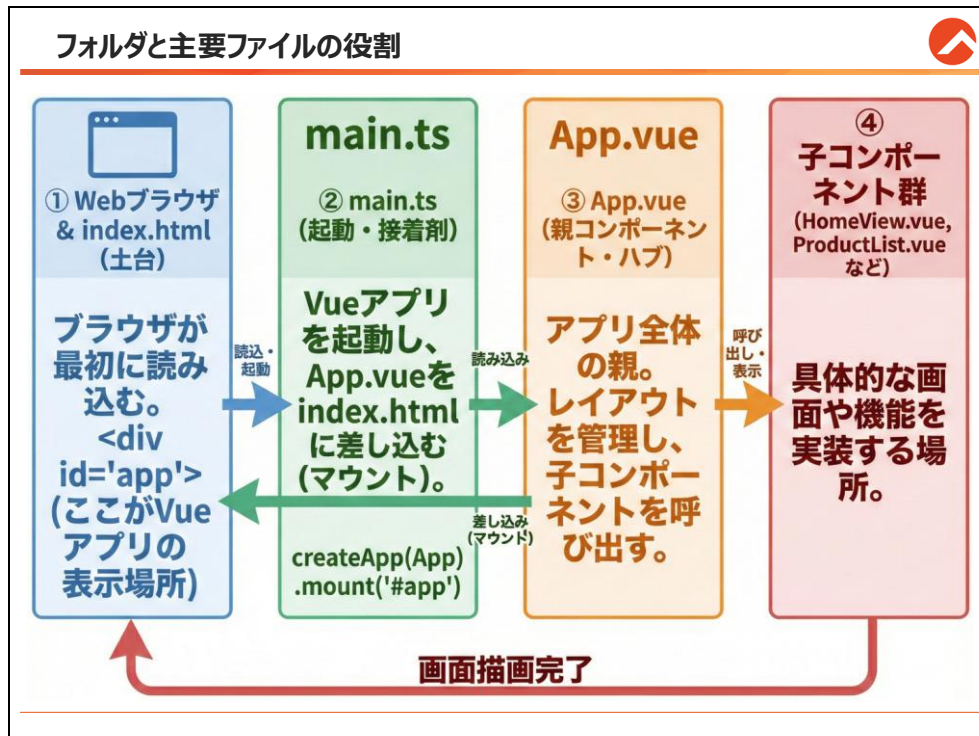
修正後、再度ブラウザで確認します。

以下のように表示されると TypeScript と Bootstrap の構文が正常に動作していることが確認できます。

ソースコードの詳しい意味は、別章でご説明します。



1.5. フォルダと主要ファイルの役割



主要ファイルの役割と実行の流れ

Vue3 プロジェクトの環境構築が終わると、以下のように、複数のフォルダとファイルが生成されます。



その中で、Web ブラウザが、**最初に読み込むのは、①index.html** です。

```
<> index.html ●
ch01-setup > <> index.html > ...
  2  <html lang="ja">
      8  </head>
      9  <body>
      10  <div id="app"></div>
      11  <script type="module" src="/src/main.ts"></script>
      12  </body>
      13 </html>
```

このファイルには、`<div id = "app"></div>`が記載されており、Vue アプリ全体を描画するための土台となります。

そして、②main.ts が読み込まれます。

```
<> index.html ● TS main.ts ●
ch01-setup > src > TS main.ts
  1  import { createApp } from 'vue'
  2  import App from './App.vue'
  3
  4  // 追加: BootstrapVueNext のプラグイン (必須)
  5  import { createBootstrap } from 'bootstrap-vue-next/plugins/createBootstrap'
  6
  7  // 追加: 必要なCSS
  8  import 'bootstrap/dist/css/bootstrap.css'
  9  import 'bootstrap-vue-next/dist/bootstrap-vue-next.css'
 10
 11  createApp(App)
 12    .use(createBootstrap()) // 改行し、追加
 13    .mount('#app')
```

次に読み込まれる②main.ts の2行では、③App.vue を import で読み込んでいます。

そして、13行では `createApp(App).mount( '#app' )` で、①index.html 内の `id = "app"` の部分に App.vue を差し込んで表示します。

つまり、App.vue に書いたコードが、Web 画面(index.html)の中身に差し込まれ、反映される仕組みになっています。

そして、App.vue は画面全体の「親コンポーネント」としての役割を持ちます。

App.vue では、ページ全体のレイアウト(ヘッダーやフッターなど)や、ルーティングの設定を記述し、実際の画面処理は ④子コンポーネント(例: HomeView.vue や ProductList.vue) の中に書いていくのが一般的です。

つまり、App.vue 単体に全ての処理を書くのではなく、

App.vue は「子コンポーネントを呼び出すためのハブ」として位置づけられ、実際の機能や画面表示の中心は、それぞれの子コンポーネント側で実装するということです。

では、次の章で Vue の本格的な構文を説明します。

章のまとめ

章のまとめ



- 開発環境の準備
- Viteでプロジェクト作成
- 依存ライブラリ導入
- 開発サーバーの起動と動作確認
- フォルダと主要ファイルの役割

この章では、Vite を利用し Vue の初期環境を構築する方法と、動作確認を行う方法について学習しました。

第2章 Vue の基礎知識

章の目標

- Vue 3 がどのようなフレームワークかを理解できる
- `ref`・`reactive`・`computed` を用いてデータを管理できる
- テンプレート構文（`{{ }}`）で変数を画面に表示できる
- `v-bind` で属性を動的に変更できる
- `v-on` でイベント処理（クリックなど）を実装できる
- `v-model` でフォーム入力を双方向バインディングできる

章の概要

章の概要



- Vueとは
- Vueコンポーネントの基本
- リアクティブなデータ管理
- ディレクティブ構文
 - v-bindディレクティブ
 - v-if/v-else/v-showディレクティブ
 - v-forディレクティブ
- イベント処理(v-on)
- 双方向バインディング(v-model)

この章では、Vue 3 の基本を体系的に学びます。最初に「Vue とは何か」というテーマで、Vue がどのようなフレームワークなのか、その歴史的背景や特徴について理解します。この節では、なぜ多くの開発者が Vue を選び、実務で利用しているのかを掴むことが目的です。

次に「Vue コンポーネントの基本」では、Vue 3 の基本的な書き方である `<script setup>` 構文や、テンプレート構文 `{{ }}` を用いたデータ表示の方法を学びます。HTML と JavaScript が自然に結びつく仕組みを体感できます。

続いて「リアクティブなデータ管理」では、Vue の核となる `ref` や `reactive` を利用して、データの変化に応じて画面が自動的に更新される仕組みを学びます。さらに、値を加工して表示するための `computed` プロパティについても理解を深めます。これにより、「データが変われば画面が変わる」という Vue の基本的な考え方を習得します。

「ディレクティブの基礎」として、`v-bind` による属性の動的変更、`v-on` によるイベント処理、そして `v-model` を利用したフォーム入力の双方向バインディングを学習します。これらを組み合わせることで、ユーザ入力に応答するインタラクティブな画面を作成できるようになります。

この章を終えることで、Vue 3 の主要な文法を一通り体験し、シンプルなアプリケーションを自分の手で実装できる基礎力が身につきます。

2.1. Vueとは

Vueとは


■ 従来のJavaScript（DOM直接操作）の課題

- 大規模開発になると保守が困難
- DOM操作のコードが複雑化しやすい
- 動的な画面更新のたびにDOM取得・更新を記述する必要があり非効率

■ フロントエンドフレームワークが登場

- データと画面の自動同期
- コンポーネント指向による再利用性の向上

React Angular Vue





■ Vue(最新はVue3)

- 2014年に公開。2020年にVue3公開
- TypeScript対応が強化され、大規模開発に適用しやすくなった
- Composition APIと<script setup>構文が導入され、旧バージョンよりシンプルで柔軟な記述が可能

動的な画面更新の必要性と課題

Webアプリケーションの開発では、画面を動的に更新する仕組みが欠かせません。従来は、JavaScriptで直接DOM(ドキュメントオブジェクトモデル)を操作しながら開発していましたが、コードが複雑になりやすく、大規模になるほど保守が難しくなるという課題がありました。

```

<script>
  // 初期値
  let count = 0;
  // ボタンをクリックする度にDOMを毎回取得しないといけない
  const countEl = document.getElementById("count");
  const btnEl = document.getElementById("btn");
  // イベントリスナー登録も明示的に記載が必要
  btnEl.addEventListener("click", function() {
    count++;
    countEl.textContent = count; // DOMを書き換える
  });
</script>
<h1 id="count">0</h1>
<button id="btn">カウントアップ</button>

```

上記のコードでは、画面を更新する度に、DOM を毎回取得し、イベントリスナーを明示的に書き、画面の更新を自分で制御しなければなりません。

規模が大きくなると「どこで DOM を更新したか」が追いつらくなり、保守性が低下してしまうデメリットがありました。

フロントエンドフレームワークの登場

この課題を解決するために登場したのが **フロントエンドフレームワーク**です。

これらを利用すると、**データと画面が自動で同期**されるようになるので、開発の効率が高くなります。なお、画面をパーツごとに分離する、「**コンポーネント指向**」になるので、再利用性が高まります。



代表的なものとして **React** (Meta(旧 Facebook)社)、**Angular** (Google 社)、そして **Vue** (個人開発から始まり、今では世界中で利用) があります。

2026 年の時点では、**React と Vue がシェアの多くを占め**、Angular は企業システムで使われることがあるものの、全体のシェアは低くなっています。

Vue の歴史と進化

Vue は 2014 年に Evan You 氏によって公開され、シンプルで分かりやすい構文を特徴として急速に広まりました。特に、React や Angular と比べて、日本語ドキュメントがとても充実していたこともあり、日本で特に広がりました。

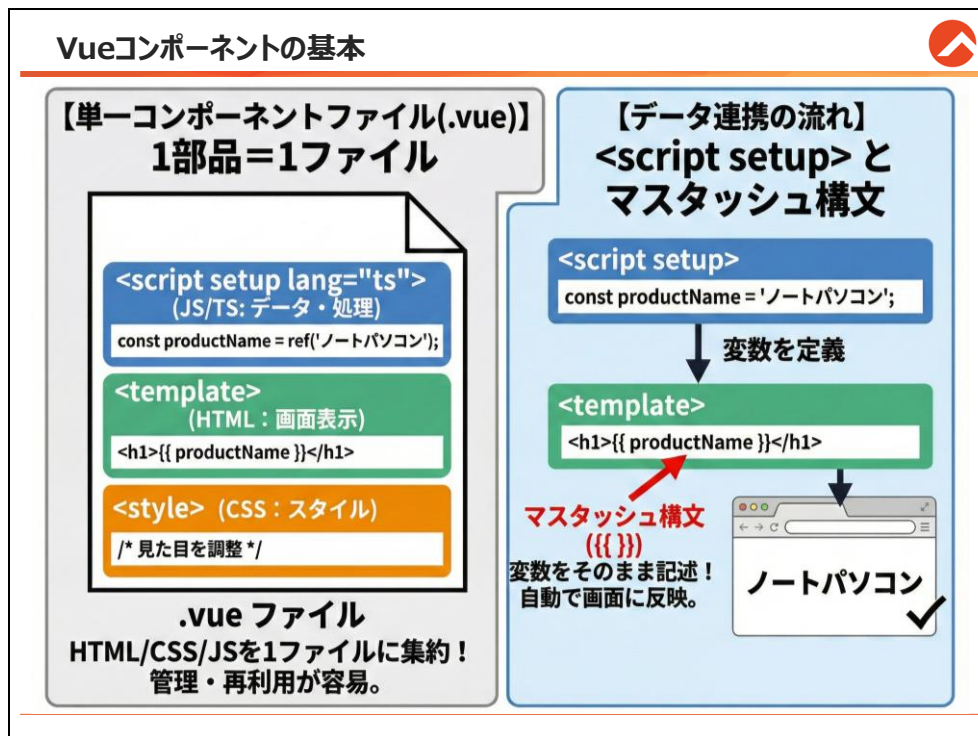
2020 年に登場した **Vue 3** では、内部処理が刷新され、パフォーマンスが向上しただけでなく、**TypeScript への対応**が強化されました。

TypeScript とは、JavaScript に「型(型注釈)」の仕組みを追加した言語です。JavaScript は、変数にどんな値が入るかを実行するまで厳密にチェックできないため、思わぬバグが起こりやすいという弱点があります。

TypeScript を使うと、開発中に「この変数は文字列型」、「この関数の引数は数値型」といったルールを定義でき、エディタ上でエラーを検知できるため、大規模開発や複数人での開発に強くなります。本テキストでも積極的に使っていきます。

また、Vue 3 では `<script setup>` 構文や **Composition API** といった新しい記述方法も導入され、従来(Vue2)よりもシンプルで柔軟な開発が可能になっています。

2.2. Vue コンポーネントの基本



単一コンポーネントファイル

Vue では、画面を「部品 (コンポーネント)」に分けて作ります。

但し、この章ではまず 1 つのコンポーネント(App.vue)だけを使って学習します。

コンポーネントは、「**〇〇.vue**」という拡張子のファイルを作り、

1 ファイルに HTML+JavaScript+CSS をまとめます。

Vue ファイルの雛形

```

<script setup lang="ts">
  // データや処理を書く部分 (script 部分)
</script>

<template>
  <!-- 画面に表示する部分 (html 部分) -->
</template>

<style>
  /* 必要ならスタイルをここに書く (css 部分) */
</style>
        
```

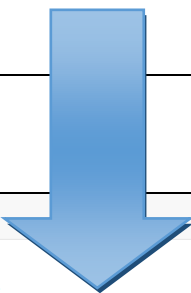
従来の開発では、1つの画面を作る時、HTML/CSS/JSファイルを別々のファイルで分けていましたが、

Vue3は「1部品=1ファイル」にすることで、どこを直せば良いかが分かりやすく、再利用もしやすくなります。

◆【補足】App.vueの初期修正

Viteで生成されるApp.vueは、以下のようなデフォルトのコードが記載されているため、
<script> <template> <style>のタグのみ残して、**削除**しておく必要があります。

```
▼ App.vue ×
ch02-basic_q > src > ▼ App.vue > {} script setup
1  <script setup lang="ts">
2  import HelloWorld from './components/HelloWorld.vue'
3  </script>
4
5  <template>
6    <div>
7      <a href="https://vite.dev" target="_blank">
8        
9      </a>
10     <a href="https://vuejs.org/" target="_blank">
11       
12     </a>
13   </div>
14   <HelloWorld msg="Vite + Vue" />
15 </template>
16
17 <style scoped>
18 .logo {
```



```
▼ App.vue ●
ch02-basic_q > src > ▼ App.vue > ...
1  <script setup lang="ts">
2    // データや処理を書く部分(script部分)
3
4  </script>
5
6
7  <template>
8    <!-- 画面に表示する部分 (html部分)-->
9
10 </template>
11
12
13 <style scoped>
14   /* 必要ならスタイルをここに書く (css部分)*/
15
16 </style>
```

<script setup lang="ts"> 構文

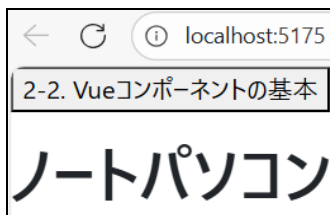
Vue 3 からは、コンポーネントのコードをよりシンプルに書ける <script setup> 構文が導入され、Vue3+TypeScript を使う場合の標準的な書き方になりました。

lang="ts" を追記すると、TypeScript を書くことができるようになり、スペルミスや型の間違いをVSCode が自動で教えてくれるので、コーディングミスを減らすことができます。

ch02-basic¥src¥components¥Sample2_2.vue

```
1:<script setup lang="ts">
2:  const productName = 'ノートパソコン'
3:</script>
4:
5:<template>
6:  <h1>{{ productName }}</h1>
7:</template>
8:
9:<style scoped>
10:
11:</style>
```

<script setup> 内で定義した JavaScript 変数は、そのまま <template> の中で使うことができます。{{ productName }} と記載するだけで、画面に「ノートパソコン」と表示されます。



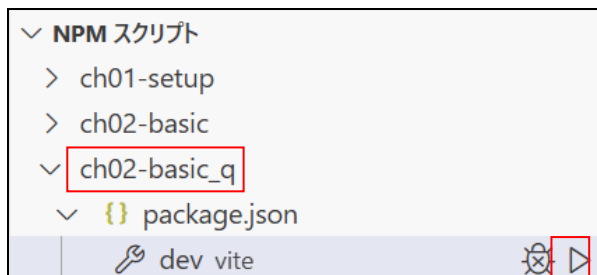
JavaScript のように document.getElementById.value などを使って HTML を直接操作する必要はありません。Vue が自動的に「**変数の中身を画面に反映**」してくれます。

✦【補足】マスタッシュ構文

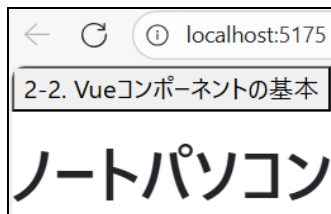
{{ productName }}のような構文を、「マスタッシュ構文」といいます。{{ }} の形が髭と似ているため、そのように名付けられました。

《 📄 構文チェック演習 》

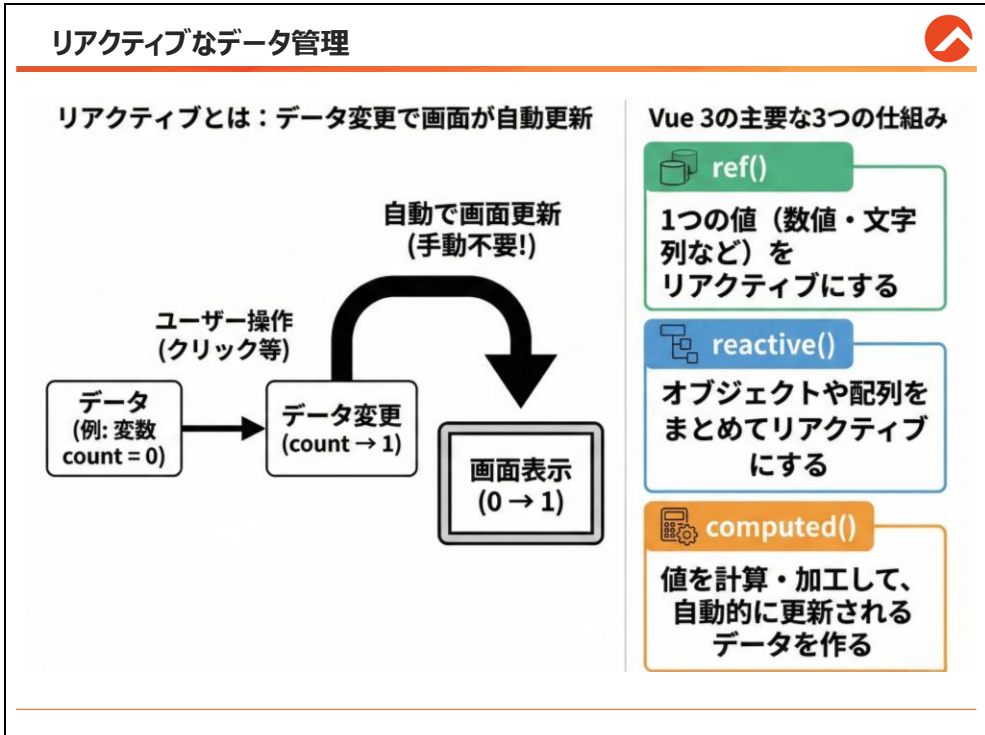
1. `ch02-basic_q¥src¥components¥Sample2_2.vue` を開きます。
2. `<script>` `<template>` `<style>` タグに何を記述すべきか、確認してください。
3. ①～②のコメントに従って、実装をしてください。
4. 保存し、Npm スクリプトから、「`ch02-basic_q`」の `dev` を実行し、Vite サーバーを起動してください。



5. Vite サーバーのターミナルに表示されている URL を参照し、ブラウザから、<http://localhost:5175/> へアクセスします。
6. 「2-2. Vue コンポーネントの基本」をクリックします。
7. 設定した変数が以下のように画面に表示されると、OK です。



2.3. リアクティブなデータ管理



リアクティブなデータ管理とは

Vue 3 の大きな特徴のひとつとして、**リアクティブシステム**（反応型データ管理）があります。

リアクティブとは、「**データが変わると、それに応じて画面も自動で変わる**」仕組みのことです。

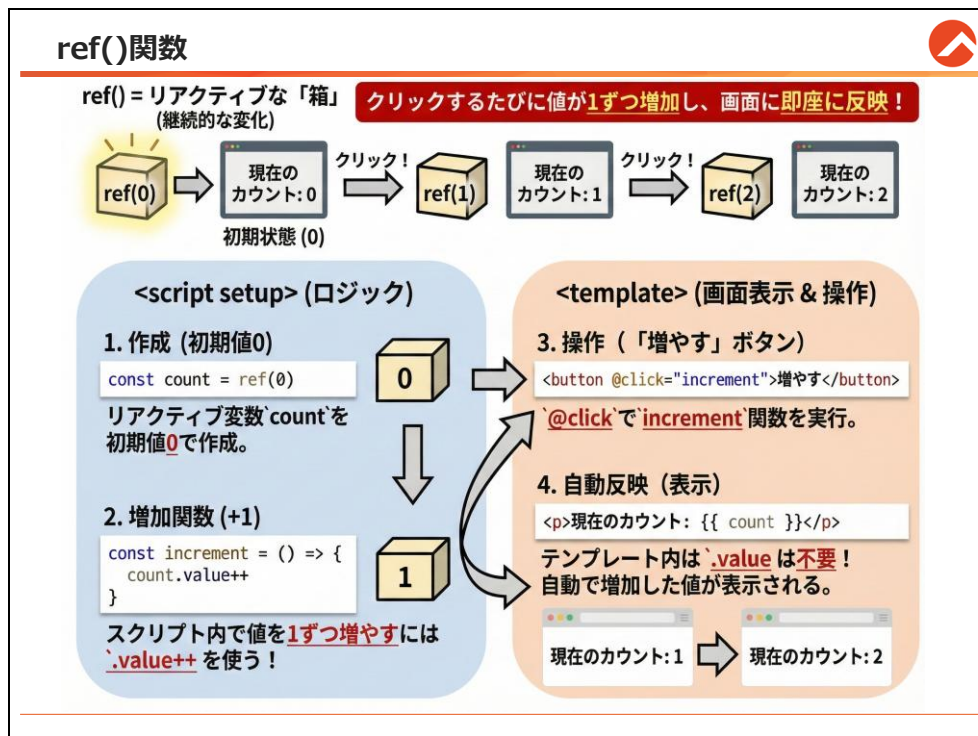
たとえば、変数 `count` に入っている値を表示していて、その値がクリックなどのユーザの操作により変化があると、

手動で画面を更新しなくても、自動で表示が変わる——これがリアクティブな動きです。

Vue 3 では、このリアクティブなデータを扱うために、主に次の 3 つの仕組みを使います。

| 仕組み | 主な役割 |
|------------|---------------------------|
| ref() | 1 つの値（数値・文字列など）をリアクティブにする |
| reactive() | オブジェクト・配列をまとめてリアクティブにする |
| computed() | 値を計算・加工して、自動的に更新されるデータを作る |

2.3.1. ref()関数



ref()関数とは

ref() は、1つの値をリアクティブに管理できる関数です。

まるで「反応する箱」のようなもので、中の値が変わると、画面の表示も自動で変わります。

まずは、ref()の動きを理解するために、以下のサンプルで説明します。

ch02-basic¥src¥components¥Sample2_3_1.vue

```

1:<script setup lang="ts">
2:import { ref } from 'vue' // ref 関数をインポート (毎回必須)
3:
4:const count = ref(0) // ref 関数でリアクティブな変数を作成
5:
6:const increment = () => { // カウントを増やす関数
7:  count.value++ // ref で作成した変数の値を変更する場合は.valueを使う
8:}
9:</script>
10:

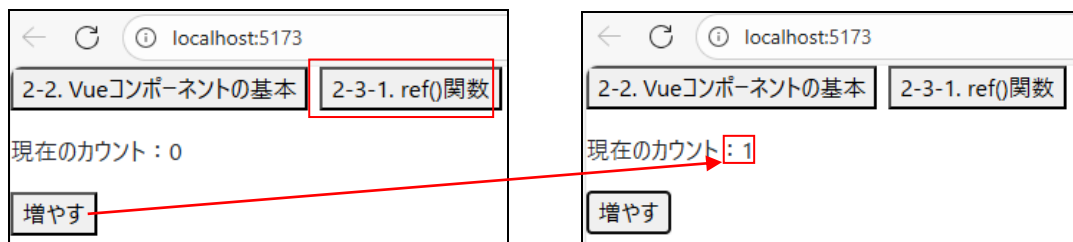
```

```

11:<template>
12: <!-- ref で作成した変数の値を参照する場合も.value は不要 -->
13: <p>現在のカウント : {{ count }}</p>

14: <!-- ボタンをクリックしたら increment 関数を実行 -->
15: <button @click="increment">増やす</button>
16:</template>

```



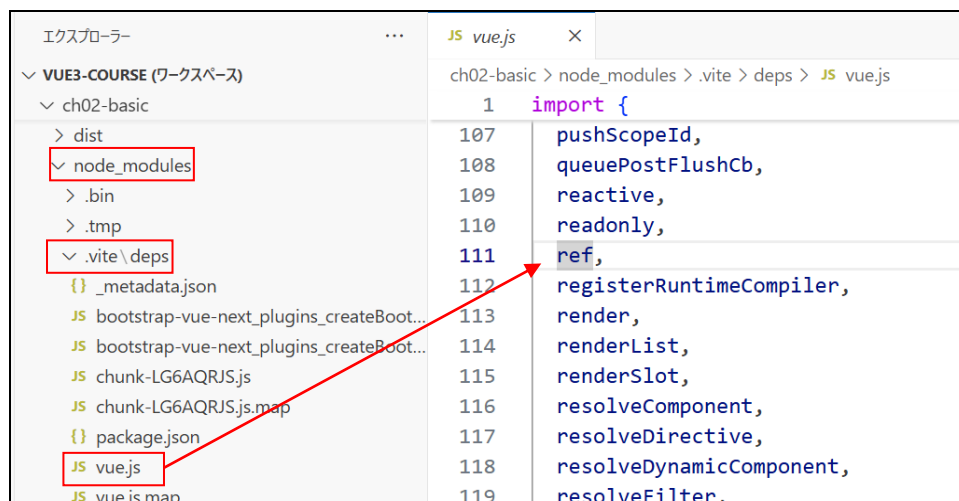
通常の JavaScript では、「増やす」ボタンをクリックする度に、カウント変数を加算し、その結果を HTML 上に更新させないといけませんでした。が、**ref()**を使うと、**変数に変化があったら、自動的に HTML の{{count}}の部分も変化している**のがわかります。

💡 構文の解説

```
2:import { ref } from 'vue' // ref 関数をインポート(毎回必須)
```

Vue の中で `ref()` を使うためには、まずは **vue モジュールから関数をインポート**する必要があります。モジュールとは、Java のライブラリのように開発に必要な部品を別のファイルで用意し、読み込ませるためのファイルです。必要な部品があれば、vue ファイルの上段に `import` して読み込むことができます。

Vue モジュールは、1 章で「`npm install`」コマンドを実行することで、以下の場所に格納されます。その中に、`ref` 関数も定義されているわけです。



```
4:const count = ref(0) // ref 関数でリアクティブな変数を作成
```

4行目の関数は、1つの値をリアクティブに管理できる関数、ref()関数です。引数として入れている「0」は、ただの初期値です。文字列を入れることもできます。

リアクティブな関数を設定した変数は、**リアクティブ変数**と呼ばれ、中身が変わると画面の表示も自動で変わるようになります。

通常、リアクティブ変数を宣言する際は、変数の定義方法が「let」ではなく「const」になります。

リアクティブ変数の変化を確認する関数

```
6:const increment = () => { // カウントを増やす関数
7:  count.value++ // ref で作成した変数の値を変更する場合は.valueを使う
8:}
```

この部分は、リアクティブ変数の動きを確認するために作った関数式です。

この関数が呼ばれる度、count 変数を1加算するようにし、画面の表示もリアルタイムで変わるのかを確認できます。

リアクティブ変数をスクリプト内で参照する際は、「.value」を利用します。

✦【補足】アロー関数

JavaScript の関数式は、本来以下のような書き方です。

```
const increment = function () {
  count.value++
}
```

Vue では、関数を定義する際に、function の部分を省略する、「アロー関数」の書き方が主流となっていますので、本書では、関数をアロー関数で表記します。

```

11:<template>
12:  <!-- refで作成した変数の値を参照する場合も.valueは不要 -->
13:  <p>現在のカウント : {{ count }}</p>
14:  <!-- ボタンをクリックしたら increment 関数を実行 -->
15:  <button @click="increment">増やす</button>
16:</template>

```

<template> は、画面 (HTML 部分) を書く場所です。13 行目のように、**{{ count }}** と書くと、**リアクティブ変数の count の値が連動され、そのまま画面に表示されます。**

スクリプトとは異なり、.value は不要です。Vue が自動的に中の値を取り出してくれます。

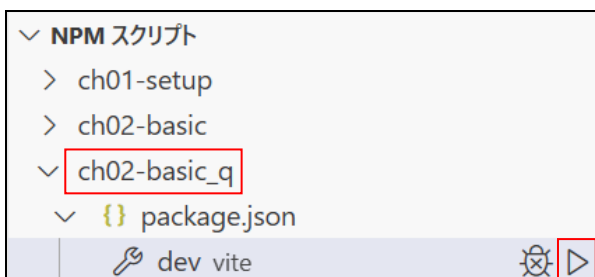
<button @click="increment"> は「ボタンをクリックしたら increment 関数を動かす」という意味です。

@click は「クリックイベント」の略で、Vue ではよく使われる構文です。この構文についての詳細は、別の節でご説明します。

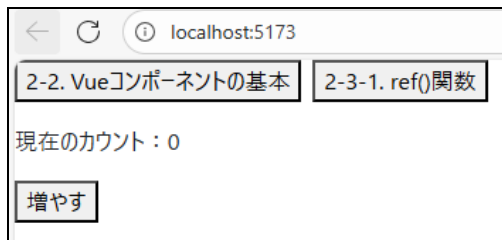
では、次の構文チェック演習で、動作を確認してください。

《📁 構文チェック演習》

1. **ch02-basic_q**をsrc¥components¥Sample2_3_1.vueを開きます。
2. ①～⑤のコメントに従って実装してください。**コメント解除の指示になっていて、実装しない箇所も必ずコードを確認し、意味を自分の言葉で説明できる状態になってから先へ進みましょう。**
3. 保存し、NPM スクリプトから、「**ch02-basic_q**」の dev を実行し、Vite サーバーを起動してください。



1. ブラウザから、<http://localhost:5173/> へアクセスします。
2. 「2-3-1. ref()関数」をクリックします。
3. 設定した変数が以下のように、画面に表示されます。



4. 「増やす」ボタンをクリックする度に、カウントが増えていくことを確認してください。

2.3.2. reactive() 関数

reactive()関数

reactive()関数とは
 オブジェクトや配列をまとめて**リアクティブ (反応的)**にする関数。
 複数の値 (プロパティ) を一つの塊として扱い、変更を自動的に画面へ反映させる入れ物。

ref()が「1つの値の箱」なら、**reactive()**は「複数の値の入れ物」

```
reactive({
  id: 1,
  name: 'ノートパソコン',
  price: 50000
})
```

reactive({ ... })
(リアクティブな塊)

自動反映
(同期)

名前を更新 更新 (rename関数)

価格 +1000 更新 (raisePrice関数)

```
<ul>
  <li>商品番号: {{product.id}}</li>
  <li>商品名: {{ product.name }}</li>
  <li>価格: {{ product.price }}</li>
</ul>
```

構文の解説・ポイント

- インポートが必要
- オブジェクトを一括で包む
- **.value は不要で直接アクセス**
- プロパティの変更が**即座に画面に反映される。**

商品情報

- 商品番号: 1
- 商品名: ノートパソコン
- 価格: 50000

価格 +1000
名前を更新

商品情報

- 商品番号: 1
- 商品名: ノートパソコン (New)
- 価格: 51000

価格 +1000
名前を更新

reactive()関数とは

reactive() は、オブジェクトや配列をまとめてリアクティブにするための関数です。

ref() が「1つの値に使う箱」だとすると、reactive() は「**複数の値(プロパティ)を一つの塊として反応させる入れ物**」です。

プロパティを変更すると、その変更が自動的に画面へ反映されます。

以下のサンプルコードでは、商品データをまとめてリアクティブ化して、関連がある複数のデータを画面と連動しています。

商品情報

- 商品番号: 1
- 商品名: ノートパソコン
- 価格: 50000

価格 +1000
名前を更新

商品情報

- 商品番号: 1
- 商品名: ノートパソコン (New)
- 価格: 51000

価格 +1000
名前を更新

ch02-basic¥src¥components¥Sample2_3_2.vue

```
1:<script setup lang="ts">
2:import { reactive } from 'vue' // reactive 関数をインポート
3:
4:// reactive 関数で複数プロパティをまとめてリアクティブ化
5:const product = reactive({
6:  id: 1,
7:  name: 'ノートパソコン',
8:  price: 50000
9:})
10:
11:const raisePrice = () => { // 価格を上げる関数
12:  product.price += 1000 // reactive で作成した変数は、.value 不要
13:}
14:
15:const rename = () => { // 名前を更新する関数
16:  product.name += ' (New)';
17:}
18:</script>
19:
20:<template>
21:  <h3>商品情報</h3>
22:  <ul>
23:    <li>商品番号 : {{ product.id }}</li>
24:    <li>商品名   : {{ product.name }}</li>
25:    <li>価格     : {{ product.price }}</li>
26:  </ul>
27:
28:  <!-- ボタン操作でオブジェクトの中身を更新 -->
29:  <button @click="raisePrice">価格 +1000</button>
30:  <button @click="rename">名前を更新</button>
31:</template>
```

💡 構文の解説

```
2:import { reactive } from 'vue' // reactive 関数をインポート
```

reactive 関数を利用するため、vue モジュールから関数をインポートする必要があります。

reactive 関数の定義

```
4:// reactive 関数で複数プロパティをまとめてリアクティブ化
5:const product = reactive({
6:  id: 1,
7:  name: 'ノートパソコン',
8:  price: 50000
9:})
```

product オブジェクトを reactive({...}) で包み、一括でリアクティブ化しています。こうすることで、id, name, price の複数プロパティを1つのリアクティブな塊として扱えます。<script>内でアクセスする際は、.value は不要で、「**product.プロパティ名**」でアクセスできます。

リアクティブ変数の変化を確認するための関数

```
11:const raisePrice = () => { // 価格を上げる関数
12:  product.price += 1000 // reactive で作成した変数は、.value 不要
13:}
14:
15:const rename = () => { // 名前を更新する関数
16:  product.name += ' (New)';
17:}
```

この2つのアロー関数は、前節の ref()と同様に、リアクティブ変数の変化を確認するための関数です。クリックする度に、product のプロパティを更新する関数です。プロパティにアクセスする際に、.value を使わず、product.price、product.name のように書いています。

商品情報の表示と、変化確認用のボタンの配置

```
23:  <li>商品番号 : {{ product.id }}</li>
24:  <li>商品名   : {{ product.name }}</li>
25:  <li>価格     : {{ product.price }}</li>
26: </ul>
27:
28: <!-- ボタン操作でオブジェクトの中身を更新 -->
29: <button @click="raisePrice">価格 +1000</button>
30: <button @click="rename">名前を更新</button>
```

リアクティブ変数は、{{ }}(マスタッシュ構文)でそのまま参照できます。

29 行目と 30 行目に割り当てているボタンのクリックイベントがある度に、11 行目と 15 行目の関数が実行され、画面も即時に反映されるようになります。

このように、reactive 関数を利用すると、オブジェクト全体が「見張られて」いて、どのプロパティを変えても自動で画面に反映されます。

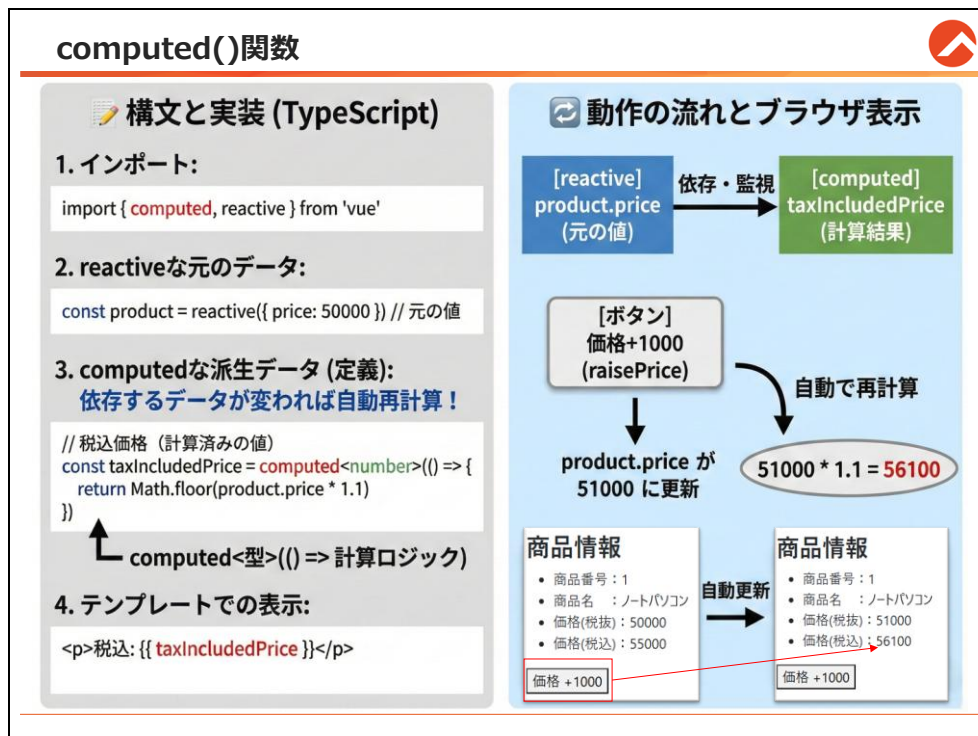
ただ、別章で後述しますが、ref も配列を扱うことができます。

「新しい配列全体を既存の変数にまるごと代入する」場合は、reactive 関数より、ref[] という書き方が好まれますので、使い分けましょう。

《📄 構文チェック演習》

1. ch02-basic_q¥src¥components¥Sample2_3_2.vue を開きます。
2. コメント ①～⑤ に従って、reactive で product を作成し、raisePrice と rename を実装してください。
3. 保存後、NPM スクリプトから「ch02-basic_q」→ dev を実行して Vite サーバーを起動します。
4. ブラウザで <http://localhost:5173/> にアクセスします。
5. 「2-3-2. reactive() 関数」をクリックします。
6. 画面に 商品番号 / 商品名 / 価格 が表示されることを確認します。
7. 「価格 +1000」「名前を変更」ボタンを押し、表示が自動更新されることを確認してください。

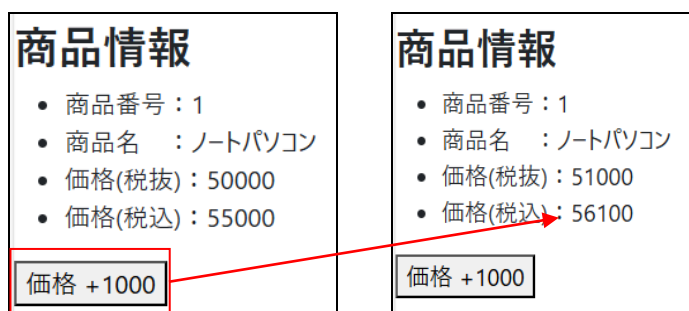
2.3.3. computed()関数



computed()関数とは

computed() は、他のリアクティブ変数をもとに、自動で計算・加工を行う関数です。ref() や reactive() の値が変わると、それに併せて computed() も自動的に再計算されます。テンプレートで表示したい「派生データ」や「加工済みの値」を作るときに利用します。

以下のサンプルでは、product の価格(price)から「税込価格」を自動で計算して表示しています。ボタンを押して価格を変えると、税込計算結果も自動で更新されることが確認できます。



ch02-basic¥src¥components¥Sample2_3_3.vue

```
1:<script setup lang="ts">
2:import { reactive, computed } from 'vue' //computed を同時に import
3:
4:// reactive 関数でオブジェクトをリアクティブ化
5:const product = reactive({
6:  id: 1,
7:  name: 'ノートパソコン',
8:  price: 50000
9:})
10:
11:// computed 関数で、price が変わると自動再計算
12:// TypeScript の型注釈として <number> を指定
13:const taxIncludedPrice = computed<number>(() => {
14:  return Math.floor(product.price * 1.1) // 税込価格を計算（小数点以下切り捨て）
15:})
16:
17:const raisePrice = (): void => { // 価格を上げる関数。戻り値型を明示的に指定
18:  product.price += 1000
19:}
20:</script>
21:
22:<template>
23:  <h3>商品情報</h3>
24:  <ul>
25:    <li>商品番号 : {{ product.id }}</li>
26:    <li>商品名   : {{ product.name }}</li>
27:    <li>価格(税抜) : {{ product.price }}</li>
28:    <li>価格(税込) : {{ taxIncludedPrice }}</li>
29:  </ul>
30:
31:  <!-- 価格を更新すると、税込価格も自動で再計算 -->
32:  <button @click="raisePrice">価格 +1000</button>
33:</template>
```

 構文の解説

```
2:import { reactive, computed } from 'vue'
```

computed()を利用したい場合は、前節と同様に、import をします。上記のようにまとめてimport することもできます。

```
11:// computed 関数で、price が変わると自動再計算
12:// TypeScript の型注釈として <number> を指定
13:const taxIncludedPrice = computed():number => {
14:   return Math.floor(product.price * 1.1) // 税込価格を計算
15:})
```

computed() 関数は括弧内の書き方が少し複雑で、computed(): 計算結果のデータ型 => {} で記述します。

関数内でリアクティブ変数 (product.price) を参照すると、**値の変化に合わせて自動で再計算**されます。computed 関数の戻り値が代入される変数のことを、「**算術プロパティ**」といいます。Math.floor 関数は、小数点の切り捨てをする関数です。

「:number は」 TypeScript の型注釈です。「この計算結果は数値型です」と明示しておくことで、型エラーを防げます。

```
27:<li>価格(税抜) : {{ product.price }}</li>
28:<li>価格(税込) : {{ taxIncludedPrice }}</li>
```

28 行で定義した算術プロパティ名を指定しています。product.price が変動すれば、自動的に変動した値で再計算を行い、表示するようになっています。

✦【補足】TypeScript の型注釈について

Vue 3 の `<script setup lang="ts">` を使うと、TypeScript による**型チェック**が自動で働きます。たとえば `const price = 1000` と書けば、TypeScript は自動で「number 型」と推論し、もし `price = '文字列'` のように代入すれば、エラーを検出してくれます。

ただし、**推論に任せず明示的に型を指定**することで、より安全で分かりやすいコードになります。

```
const price: number = 1000
const raise = (): void => {...}
```

このように、変数名や関数名の後ろに **:型名** を記述します。

(: number は「数値型」、(): void は「戻り値がない関数」を意味します。)

<実務での一般的なルール>

- ・簡単な変数(`const count = ref(0)` など)は型推論に任せることが多い
- ・外部 API のレスポンスや、関数の戻り値など、**重要な部分には型注釈を明示**する
- ・チーム開発では、読みやすさのために**型を明示するスタイル**が好まれる

本書では、理解しやすさを重視し、TypeScript の型推論に基本的に任せる方針とします。ただし、型注釈の書き方や意味もあわせて紹介しますので、実務で使う際には「どこまで明示するか」を判断できるように身につけていきましょう。

《📄 構文チェック演習》

1. `ch02-basic_q¥src¥components¥Sample2_3_3.vue` を開きます。
2. コメント①～④に従って実装してください。
3. 保存後、NPM スクリプトから「`ch02-basic_q → dev`」を実行し、Vite サーバーを起動します。
4. ブラウザで `http://localhost:517●/` にアクセスします。
5. 「2-3-3. computed() 関数」をクリックします。
6. 「価格 +1000」ボタンをクリックし、税抜価格と税込価格の両方がリアルタイムに連動して変化することを確認してください。

2.4. ディレクティブ構文

ディレクティブ構文

ディレクティブ構文とは？ 「v-」で始まる、HTML要素への「動的な命令」

```
<p>,</p>
<img></img>
<p>
```

HTMLだけではできない「変化」「条件」「繰り返し」を実現！

例1：表示・非表示の「条件」 (v-if)

```
<p v-if="isVisible">
  表示されます
</p>
```

isVisible = true
→ 表示 (見える)

isVisible = false
→ 非表示 (消える)

例2：属性を「動的」に変更 (v-bind)

```

```

imagePath変数の値に応じて、
画像ソース (src) が自動で切り替わる！

主なディレクティブと用途		
v-bind (動的バインド)	v-if / v-show (条件分岐)	v-for (繰り返し)
HTML属性 (src, href, classなど) を変数や計算結果で動的に変更	条件によって要素の表示・非表示を制御	配列・オブジェクトデータからリストを自動生成

ディレクティブ構文とは

ディレクティブ構文とは、JavaScriptの値をHTML要素や属性に埋め込むためのVue独自の記述方法です。

Vueでは、通常のHTMLだけではできなかった「変化するデータの表示」や「条件による切り替え」「繰り返し表示」などを、特別な構文を使って実現します。

Vueのテンプレート内では、**v-**で始まる特別な属性をディレクティブと呼びます。

「ディレクティブ」は、英語で「命令」という意味で、

HTML要素に“動きを命令する”Vue独自の仕組みです。たとえば、

```
<p v-if="isVisible">この文章は条件が true のときだけ表示されます</p>

```

このように、v-で始まる属性を使うことで、HTMLタグの中でも動的な処理を実現できます。

ディレクティブ	主な用途	学習のポイント
v-bind	HTML 属性(src, href, class など)を動的に変更する	変数や計算結果を属性に反映できる
v-if / v-else / v-show	要素の表示・非表示を切り替える	条件によって表示を制御する
v-for	配列・オブジェクトの繰り返し表示	データをもとにリストを自動生成する

2.4.1. v-bind ディレクティブ

v-bind ディレクティブ

JavaScript (reactiveデータ)

```
const location = reactive({
  name: '東京会場',
  img: '/images/tokyo.jpg',
  url: '...',
  btnClass: 'btn-primary' //青
})
// ボタンクリックで情報を切り替え
```

HTML テンプレート (v-bindの使用)

```
<h3>{{ location.name }}</h3>
<!-- 画像を動的に -->

<!-- リンクを動的に -->
<a :href="location.url">詳細</a>
<!-- クラス(色)を動的に -->
<button :class="location.btnClass">切替</button>
```

データをバインド → 動的に描画・更新

ブラウザ表示 (動作イメージ)

東京会場の情報

会場の詳細サイトを開く

別の会場を確認する

→

大阪会場の情報

会場の詳細サイトを開く

別の会場を確認する

→

名古屋会場の情報

会場の詳細サイトを開く

別の会場を確認する

→

東京会場の情報

会場の詳細サイトを開く

別の会場を確認する

ボタンクリックで、「画像・リンク・ボタン色」が連動して切り替わる！

構文とポイント

- 基本構文: `v-bind:属性名="変数名"`
- 省略形 (推奨): `:属性名="変数名"`
- ポイント: データと表示が同期するため、JavaScriptでHTMLを直接操作する必要 ✕

v-bind ディレクティブとは

v-bind ディレクティブは、HTML 属性を動的に変更するための構文です。

通常の HTML では、属性値 (画像のパス、リンク先、class 名など) は固定になりますが、v-bind を使うことで、JavaScript 側で管理している変数や計算結果を結び付け (バインド) し、そのまま属性に反映させることができます。

たとえば、`v-bind:src="変数名"` と書くと、その変数の値が変わるたびに、画像のパス・リンク先・デザインも自動的に切り替わります。



■ 構文

`v-bind:属性名="JavaScript の変数名"`

属性名としては、`src`、`href`、`class`、`style`などを指定できます。

また、次のように `v-bind:` を **:** に省略することもできます。

`:属性名="JavaScript の変数名"`

以下のサンプルコードでは、
ボタンをクリックする度に、「画像」「リンク先」「ボタンデザイン」が順番に切り替わるようになっています。

`ch02-basic¥src¥components¥Sample2_4_1.vue`

```
1:<script setup lang="ts">
2:import { reactive } from 'vue'
3:
4:// 1つのオブジェクトにまとめて管理
5:const locationInfo = reactive({
6:  name: '東京会場',
7:  imagePath: '/images/sample1.jpg',
8:  linkUrl: 'https://www.trainocate.co.jp/guide/tokyo.html',
9:  buttonClass: 'btn btn-primary',
10: state: 1
11:})
12:
13:// ボタンを押すたびに状態を切り替える関数
14:const toggle = (): void => {
15:  if (locationInfo.state === 1) {
16:    locationInfo.name = '大阪会場'
17:    locationInfo.imagePath = '/images/sample2.jpg'
18:    locationInfo.linkUrl = 'https://www.trainocate.co.jp/guide/osaka.html'
19:    locationInfo.buttonClass = 'btn btn-success'
20:    locationInfo.state = 2
21:  } else if (locationInfo.state === 2) {
22:    locationInfo.name = '名古屋会場'
23:    locationInfo.imagePath = '/images/sample3.jpg'
24:    locationInfo.linkUrl = 'https://www.trainocate.co.jp/guide/nagoya.html'
25:    locationInfo.buttonClass = 'btn btn-warning'
26:    locationInfo.state = 3
```

```
27: } else {
28:   locationInfo.name = '東京会場'
29:   locationInfo.imagePath = '/images/sample1.jpg'
30:   locationInfo.linkUrl = 'https://www.trainocate.co.jp/guide/tokyo.html'
31:   locationInfo.buttonClass = 'btn btn-primary'
32:   locationInfo.state = 1
33: }
34:}
35:</script>
36:
37:<template>
38:
39:   <h3>{{ locationInfo.name }}の情報</h3>
40:
41:   <!-- v-bind:src → :src で画像を動的に変更 -->
42:   
43:
44:   <!-- v-bind:href → :href でリンク先を動的に変更 -->
45:   <p>
46:     <a :href="locationInfo.linkUrl" target="_blank">
47:       会場の詳細サイトを開く
48:     </a>
49:   </p>
50:
51:   <!-- v-bind:class → :class で Bootstrap ボタンの色を動的に変更 -->
52:   <button :class="locationInfo.buttonClass" @click="toggle">
53:     別の会場を確認する
54:   </button>
55:</template>
```

💡 構文の解説

5～11 行目: `reactive()` でオブジェクトをリアクティブ化

```
5: const locationInfo = reactive({
6:   name: '東京会場',
7:   imagePath: '/images/sample1.jpg',
8:   linkUrl: 'https://www.trainocate.co.jp/guide/tokyo.html',
9:   buttonClass: 'btn btn-primary',
10:  state: 1
11:})
```

ここでは、前節で学習した通り、1つのオブジェクトに複数のデータをまとめてリアクティブ化しています。

画面上で動的に変化する内容(画像・リンク・ボタン色など)の初期値を設定しています。

14～34 行目: ボタンを押した時の処理

```
14: const toggle = (): void => {
15:   if (locationInfo.state === 1) {
16:     locationInfo.name = '大阪会場'
17:     //イメージパス・リンク先・ボタンデザインの置き換え処理が入る
20:     locationInfo.state = 2
21:   } else if (locationInfo.state === 2) {
22:     locationInfo.name = '名古屋会場'
23:     //イメージパス・リンク先・ボタンデザインの置き換え処理が入る
26:     locationInfo.state = 3
27:   } else {
28:     locationInfo.name = '東京会場'
29:     //イメージパス・リンク先・ボタンデザインの置き換え処理が入る
32:     locationInfo.state = 1
33:   }
34: }
```

クリックイベントで `toggle()` が呼ばれるたびに、現在の `state` の値をもとに会場を順番に切り替えています。

各分岐の中では、`imagePath`、`linkUrl`、`buttonClass` の値を変更しており、Vue が自動的に画面の表示を更新してくれます。HTML を直接書き換える必要はありません。

42 行目:画像の切り替え

```
42: 
```

:src は v-bind:src の省略形です。

locationInfo.imagePath の値が変わるたびに、画像も自動で切り替わります。

46～48 行目:リンク先の切り替え

```
46: <a :href="locationInfo.linkUrl" target="_blank">
47:   会場の詳細サイトを開く
48: </a>
```

:href は v-bind:href の省略形です。

クリックすると、各会場に応じたリンク先が新しいタブで開きます。

target="_blank" は、「新しいタブで開く」ための設定です。

52 行目:ボタンデザインの切り替え

```
52: <button :class="locationInfo.buttonClass" @click="toggle">
53:   別の会場を確認する
54: </button>
```

:class は v-bind:class の省略形です。

クリックで toggle() 関数を呼び出し、ボタンのデザインを切り替えています。

ところで、リンクやイメージの属性とは異なり、クラス名を変えることで何ができるか、疑問に思われるかもしれません。class 属性は、様々な使い道がありますが、その一つは **Bootstrap** を利用することです。

Bootstrap では、**クラス名を指定するだけでボタンの色や形を簡単に変えられます。**

たとえば以下のように書くと、それぞれ異なる色のボタンが適用されます。

```
9:   buttonClass: 'btn btn-primary', (青色のボタンのデザインを適用)
19: locationInfo.buttonClass = 'btn btn-success' (緑色のボタンのデザインを適用)
25: locationInfo.buttonClass = 'btn btn-warning' (黄色のボタンのデザインを適用)
```

このように、v-bind:class と Bootstrap を組み合わせると、**Vue のデータの変化に合わせて見た目をリアルタイムに切り替えることができます。**

もし直接細かいスタイルを修正したい場合は、

「v-bind:style="プロパティ名: 値"」の形式で CSS を直接指定することも可能です。

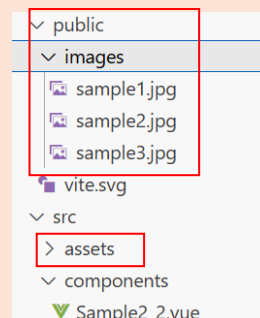
ただし、デザインを簡単に管理したい場合は、Bootstrap の class を動的に切り替える方法が実用的です。

◆【補足】画像ファイルの配置について

画像ファイルは **public/images/ フォルダ**に入れてください。
ブラウザ上では ‘/images/sample1.jpg’ のように参照できます。

実際の開発現場では、
アプリ内部で使用する**固定画像**は **src/assets** に置き、
ユーザーが動的に切り替える画像や外部リンクは **public/** に**配置**するのが一般的です。

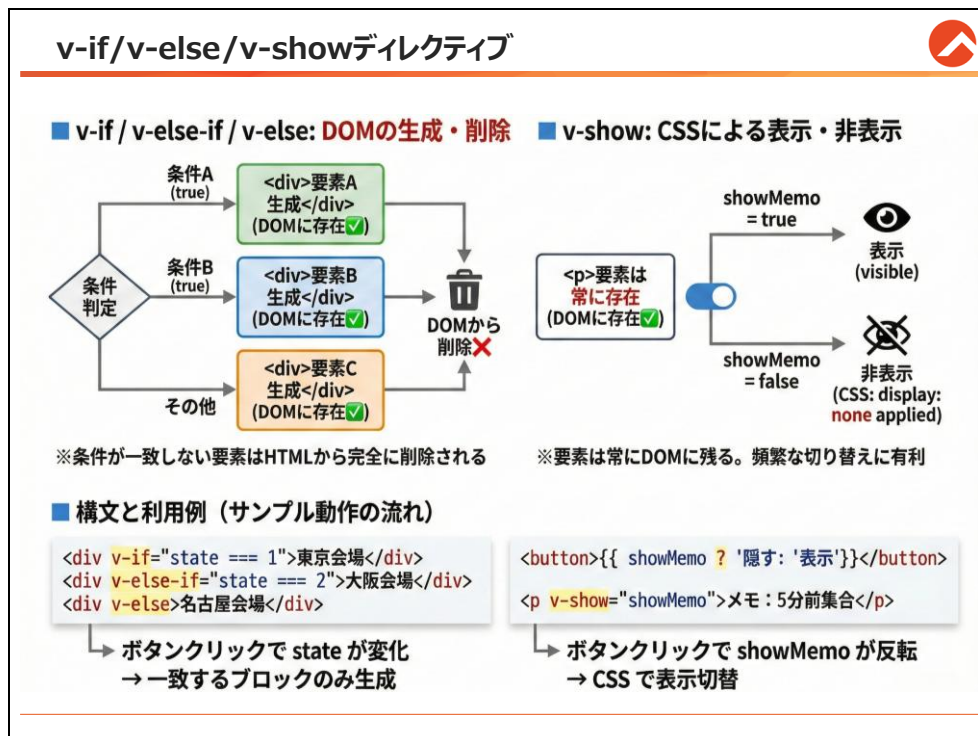
本テキストでは理解しやすさを優先し、
すべて public/images フォルダを利用します。



《📄 構文チェック演習》

1. `ch02-basic_q¥src¥components¥Sample2_4_1.vue` を開きます。
2. コメント①～⑥に従って、`:src`、`:href`、`:class` を使用して
画像・リンク・ボタン色を動的に変更する機能を実装してください。
3. 保存後、NPM スクリプトから「`ch02-basic_q → dev`」を実行し、Vite サーバーを起動します。
4. ブラウザで <http://localhost:5173/> にアクセスします。
5. 「2-4-1. v-bind」をクリックします。
6. 「別の会場を確認する」ボタンを押して、
画像・リンク・ボタンの色が順番に切り替わることを確認してください。

2.4.2. v-if/v-else/v-show ディレクティブ



v-if / v-else / v-show とは

v-if、v-else-if、v-else は、条件に応じて HTML 要素を表示・非表示に切り替えるためのディレクティブです。

さらに v-show は、要素を削除せずに「非表示 (display:none)」にする仕組みです。

どちらも見た目は同じように動作しますが、内部の処理 (DOM の扱い方) が異なります。

■ 構文

```
<div v-if="条件">条件が true のときに、囲っている要素が生成される</div>
<div v-else-if="別の条件">別の条件のときに囲っている要素が生成される</div>
<div v-else>どの条件にも当てはまらないときに、囲っている要素が生成される</div>

<p v-show="条件">条件が true の間だけ表示される (false でも要素は残す) </p>
```

条件式は、通常の if~else if~else 式と書き方は一緒です。条件に一致する場合は、v-if ディレクティブを付与している要素全体が生成されます。条件が一致しない場合は、HTML が生成されません。

v-show は、「生成/非生成」ではなく、要素はいつも存在していて、「表示/非表示」に近いです。

以下のサンプルコードは、会場ごとに案内メッセージを切り替えしています。

補足メモは、v-show で表示・非表示を実装しています。

ch02-basic¥src¥components¥Sample2_4_2.vue

```
1:<script setup lang="ts">
2:import { ref } from 'vue'
3:
4:// 会場の状態 (1:東京 / 2:大阪 / 3:名古屋)
5:const state = ref(1)
6:
7:// v-show 用 : メモの表示・非表示
8:const showMemo = ref(false)
9:
10:// 次の会場へ
11:const next = (): void => {
12:  if (state.value === 1) {
13:    state.value = 2
14:  }
15:  else if (state.value === 2) {
16:    state.value = 3
17:  }
18:  else {
19:    state.value = 1
20:  }
21:}
22:
23:// メモの表示切り替え
24:const toggleMemo = (): void => {
25:  showMemo.value = !showMemo.value
26:}
27:</script>
28:
29:<template>
30:  <h3>会場案内</h3>
31:
32:  <!-- v-if / v-else-if / v-else : 状態に合わせて表示ブロックを切替 -->
33:  <div v-if="state === 1"> 東京会場 </div>
34:  <div v-else-if="state === 2"> 大阪会場</div>
35:  <div v-else> 名古屋会場</div>
```

```

36:
37: <button class="btn btn-primary mt-2" @click="next">次の会場</button>
38:
39: <hr class="my-3" />
40:
41: <!-- v-show : 要素は残したまま見た目だけ隠す/見せる -->
42: <button class="btn btn-outline-secondary mb-2" @click="toggleMemo">
43:   メモを {{ showMemo ? '隠す' : '表示' }}
44: </button>
45: <p v-show="showMemo">メモ : 当日は 5 分前集合</p>
46: </template>

```

💡 構文の解説

基本的に、前節のサンプルコードと似ているため、異なる部分のみ解説します。

```

7:// v-show 用 : メモの表示・非表示
8:const showMemo = ref(false)

```

8行目では、trueかfalseの値を持つリアクティブ変数を宣言しています。下記のv-showの切り替えで使われます。

```

23:// メモの表示切り替え
24:const toggleMemo = (): void => {
25:  showMemo.value = !showMemo.value
26:}

```

24行～26行目もv-showのための関数です。「!」は「反転」を意味します。

showMemoの値がtrueなら、falseが代入されますし、falseなら、trueが代入されるので、ボタンを押すたびにshowMemoの値が反対となり、表示・非表示の切り替えを可能にします。

42～45 行目: v-show と 三項演算子

```

41: <!-- v-show : 要素は残したまま見た目だけ隠す/見せる -->
42: <button class="btn btn-outline-secondary mb-2" @click="toggleMemo">
43:   メモを{{ showMemo ? '隠す' : '表示' }}
44: </button>
45: <p v-show="showMemo">メモ : 当日は 5 分前集合</p>

```

42 行目のボタンをクリックすると、toggleMemo 関数が呼び出され、showMemo の値が true か false に切り替わります。

43 行の構文は、**三項演算子**と呼ばれる書き方で、**短い if 文**のような書き方です。

「:」を境界にし、?の直後の変数が true なら左辺が適用、false なら右辺が適用されます。

つまり、「もし showMemo が true なら '隠す'、そうでなければ '表示'を出力」という条件を 1 行で書けるようにしたものが三項演算子です。Vue では、ボタンやメッセージを状態に応じて切り替える時によく使われます。

45 行の v-show は、要素を削除せずに CSS で非表示にする構文です。条件が true のときだけ見えるようになります。

条件が頻繁に変わるような場合に便利です。(例:開閉ボタン、タブの切り替えなど)

【v-show の条件が false の時】

会場案内

名古屋会場

次の会場

メモを表示

```

<!-- v-show : 要素は残したまま見た目だけ隠す/見せる -->
<button class="btn btn-outline-secondary mb-2"> メモを表示</button>
<p style="display: none;">メモ : 当日は5分前集合</p> == $0
</div>

```

v-show の条件が false の場合は、描画後の HTML に display:none のスタイルが適用され、表示されなくなります。

【v-show の条件が true の時】

会場案内

名古屋会場

次の会場

メモを隠す

```

<!-- v-show : 要素は残したまま見た目だけ隠す/見せる -->
<button class="btn btn-outline-secondary mb-2"> メモを隠す</button>
<p style>メモ : 当日は5分前集合</p> == $0
</div>

```

v-show の条件が true の時は、display:none が無効になるため、表示されるようになります。

40～42 行目 : v-if / v-else-if / v-else

```
30: <h3>会場案内</h3>
32: <!-- v-if / v-else-if / v-else : 状態に合わせて表示ブロックを切替 -->
33: <div v-if="state === 1"> 東京会場 </div>
34: <div v-else-if="state === 2"> 大阪会場</div>
35: <div v-else> 名古屋会場</div>
36:
37: <button class="btn btn-primary mt-2" @click="next">次の会場</button>
```

v-show とは異なり、v-if/v-else-if/v-else は、条件が true の時だけ要素ブロックを表示します。上記の例だと、<div>～</div>までのブロックが表示されるという意味です。false の場合は、ブロックごと DOM 上から削除され、表示されなくなります。

以下のように、3 つのうち、どれかの条件が true の<div>ブロック 1 つしか表示されません。

会場案内

東京会場

次の会場

メモを表示

会場案内

東京会場

次の会場

メモを表示

会場案内

大阪会場

次の会場

メモを表示

会場案内

大阪会場

次の会場

メモを表示

```

<h3>会場案内</h3>
<!-- v-if / v-else-if / v-else : 状態に合わせて表示ブロックを切替 -->
<div>東京会場</div> == $0
<button class="btn btn-primary mt-2">次の会場</button>

```

```

<h3>会場案内</h3>
<!-- v-if / v-else-if / v-else : 状態に合わせて表示ブロックを切替 -->
<div>大阪会場</div> == $0
<button class="btn btn-primary mt-2">次の会場</button>

```

v-if の場合、1 つ目のボタンをクリックすると、関数により state が変動し、state によって異なる要素が生成されるようになります。

《📄 構文チェック演習》

1. ch02-basic_q¥src¥components¥Sample2_4_2.vue を開きます。
2. コメント①～④に従って、ソースコードの意味を把握してください。
3. コメント⑤と⑦の実装を行ってください
4. 保存後、NPM スクリプトから「ch02-basic_q → dev」を実行します。
5. ブラウザで <http://localhost:517> にアクセスします。
6. 「2-4-2. v-if」をクリックします。
7. 2 つボタンを押して、会場名とメモのメッセージがに変化することを確認してください。

2.4.3. v-for ディレクティブ

v-forディレクティブ

■ 配列やオブジェクトの要素を繰り返し表示できる

■ 構文

<要素 v-for="任意の変数*i* in 配列名" :key="*i*.id">

- *in*を境に、*右辺*は配列名、*左辺*は任意の変数名
- *:key*を付けることで、各要素の識別ができる

```

<tr v-for="p in products" :key="p.id">
  <td>{{ p.id }}</td>
  <td>{{ p.name }}</td>
  <td>{{ p.price }}円</td>
</tr>
          
```

➡

商品一覧

ID	商品名	価格
1	ノートパソコン	120000円
2	マウス	2500円
3	キーボード	4800円

v-for とは

v-for ディレクティブは、**配列やオブジェクトの要素を繰り返し表示**するための構文です。たとえば、複数の商品データやユーザー一覧などを画面に並べたいときに使います。

実際の業務では、v-for で表示するデータは **データベース(DB)やAPI から取得**することが多いですが、ここではまず、Vue の中で定義したサンプル配列を使って仕組みを理解します。

■ 構文

<要素 v-for="任意の変数 *i* in 配列名" :key="*i*.id">

v-for は、他言語の拡張 for 文と似ています。

「*in*」を境に、**右辺には配列名**を記載します。

左辺には、配列に入っているコンテンツを意味する簡略な変数名を決め、記載します。

そして、「v-bind:key」、略して「**:key**」を**一緒に並べる**ことが**推奨**されます。:key を指定することで、Vue が各要素を一意に識別できるので、データの順番が変わった時でも正しく再描画できるようになるためです。

それでは、商品データを v-for を使って一覧表示するサンプルを紹介します。

```
1:<script setup lang="ts">
2:import { reactive } from 'vue'
3:
4:// 商品の配列（本来は DB や API から取得することを想定）
5:const products = reactive([
6:  { id: 1, name: 'ノートパソコン', price: 120000 },
7:  { id: 2, name: 'マウス', price: 2500 },
8:  { id: 3, name: 'キーボード', price: 4800 }
9:])
10:</script>
11:
12:<template>
13:  <h3>商品一覧</h3>
14:
15:  <!-- v-for で配列の中身を順に表示 -->
16:  <table class="table table-bordered w-auto">
17:    <thead class="table-light">
18:      <tr>
19:        <th>商品 ID</th>
20:        <th>商品名</th>
21:        <th>価格</th>
22:      </tr>
23:    </thead>
24:    <tbody>
25:      <tr v-for="p in products" :key="p.id">
26:        <td>{{ p.id }}</td>
27:        <td>{{ p.name }}</td>
28:        <td>{{ p.price }}円</td>
29:      </tr>
30:    </tbody>
31:  </table>
32:</template>
```

💡 構文の解説

5～9 行目: 配列データを reactive で定義

```

5: const products = reactive([
6:   { id: 1, name: 'ノートパソコン', price: 120000 },
7:   { id: 2, name: 'マウス', price: 2500 },
8:   { id: 3, name: 'キーボード', price: 4800 }
9: ])

```

配列の中に、3 件の商品オブジェクトを入れています。配列は、`[]` で囲むことで表現できます。各商品オブジェクトは `id`、`name`、`price` の 3 つの情報を持っています。

実際の業務では、このようなデータをサーバーの DB や REST API から取得して表示します。今回は仕組みを理解するために、画面側で直接定義しています。

25～29 行目: `v-for` で繰り返し表示

```

25: <tr v-for="p in products" :key="p.id">
26:   <td>{{ p.id }}</td>
27:   <td>{{ p.name }}</td>
28:   <td>{{ p.price }}円</td>
29: </tr>

```

テーブルの行を表す、`<tr>` 要素に `v-for` ディレクティブを入れています。
`v-for="p in products"` は、`products` 配列の要素を 1 つずつ取り出して、`p` という変数に入れています。繰り返しの回数は、配列の要素数(今回は 3 件)になります。

`<tr>～</tr>` が 3 回生成されると、以下のように画面に出力されます。

商品一覧		
ID	商品名	価格
1	ノートパソコン	120000円
2	マウス	2500円
3	キーボード	4800円

`:key="p.id"` は、Vue が繰り返し要素を区別できるようにするための**識別子(ID)**です。

同じような要素が並ぶ場合でも、`key` があると高速で正確に再描画できますので、記載するのが推奨されます。

◆【補足】アプリケーション内で v-for が使われるイメージ

通常のアプリケーションでは、商品一覧や社員リストなどを DB に保存し、バックエンド(例:Spring Boot や Node.js)から **JSON 形式**でデータを取得します。

フロントエンド(Vue 側)では、axios や fetch() を使って API と通信し、DB からのデータを取得します。その後、同じように v-for で画面に並べて表示します。

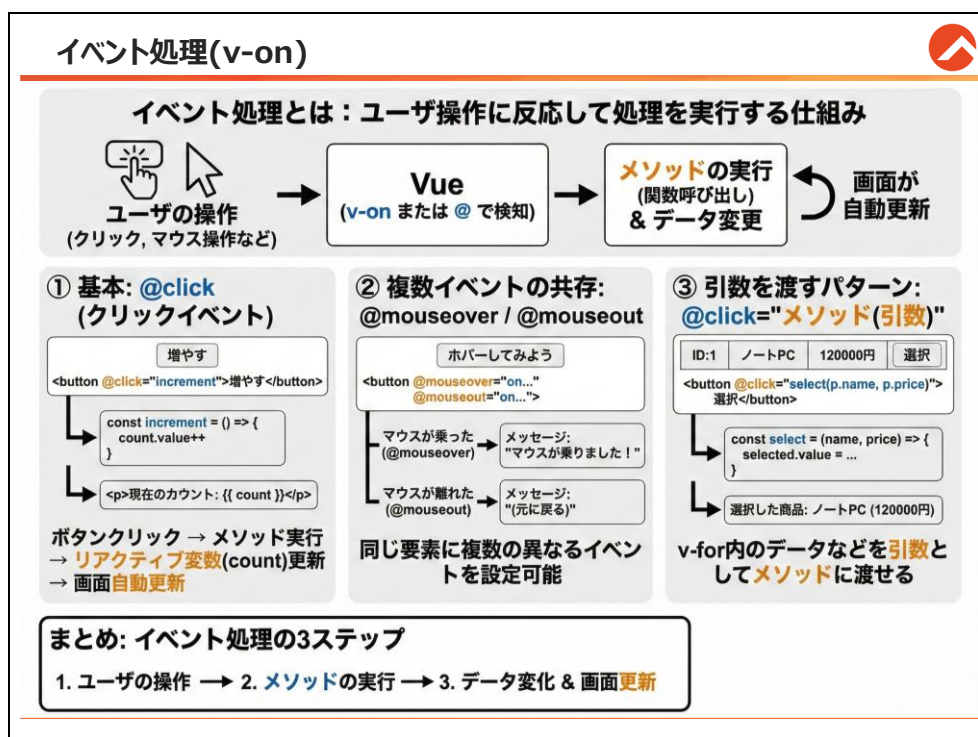
このように、v-for の使い方を理解しておくと、「データベースの内容を一覧表示する画面」を簡単に作ることができます。

詳細は、5 章を参照してください。

《📄構文チェック演習》

1. ch02-basic_q¥src¥components¥Sample2_4_3.vue を開きます。
2. コメント①～③に従って、ソースコードの確認と実装を行ってください。
3. 保存後、NPM スクリプトから「ch02-basic_q → dev」を実行します。
4. ブラウザで <http://localhost:517●/> にアクセスします。
5. 「2-4-3. v-for」をクリックします。
6. 3 つの商品データの一覧が表示されるかを確認してください。

2.5. イベント処理(v-on)



イベント処理とは

イベント処理とは、ボタンのクリックやマウス操作など、ユーザの操作に反応して処理を実行する仕組みのことです。

Vue では **v-on** または省略形の **@** を使って、HTML 要素にイベントを登録します。

たとえば、ボタンをクリックしたり、マウスを乗せる動作をきっかけに、変数の値を変更したり、関数(メソッド)を呼び出すことができます。

まずは実際に動かしてイメージを掴みましょう。

ch02-basic プロジェクトを起動し、「**2-5. イベント処理**」の画面を開いて操作してみてください。

💡 構文の解説

【① @click: クリックイベント】

ch02-basic¥src¥components¥Sample2_5.vue

```

4: // カウント値
5: const count = ref(0)

20: // カウントを増やすメソッド
21: const increment = (): void => {
22:   count.value++
23: }
24:
25: // カウントをリセットするメソッド
26: const reset = (): void => {
27:   count.value = 0
28: }

52: <p>現在のカウント: {{ count }}</p>
53:   <button class="btn btn-primary mx-1" @click="increment">増やす</button>
54:   <button class="btn btn-secondary mx-1" @click="reset">リセット</button>

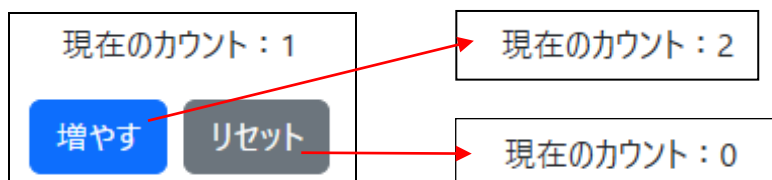
```

v-on ディレクティブは、要素に「**どんな操作を検知するか**」を設定する構文です。最も基本的なイベントは、マウスクリック(click)です。

正式な構文は **v-on:click="メソッド名"** ですが、通常は省略形 **@click="メソッド名"** を使います。

53 行と 54 行目のボタンをクリックすると、該当するメソッド(increment または reset)が実行されます。その結果、リアクティブ変数 count の値が更新され、52 行の {{ count }} の表示も自動的に変わります。

※引数を渡さない場合、() は省略できます。



【② @mouseover と @mouseout : 1つの要素に、複数のイベントを付与できる】

ch02-basic¥src¥components¥Sample2_5.vue

```

10:// マウスオーバー時のメッセージ
11:const hoverMsg = ref(' (ボタンにマウスを乗せてみてください) ')

30:// マウスが乗ったときのメソッド
31:const onMouseOver = (): void => {
32:  hoverMsg.value = ' マウスが乗りました ! '
33:}
34:
35:// マウスが離れたときのメソッド
36:const onMouseOut = (): void => {
37:  hoverMsg.value = ' (ボタンにマウスを乗せてみてください) '
38:}

59:<h5>マウス操作のイベント</h5>
60:<button class="btn btn-success" @mouseover="onMouseOver"
                                     @mouseout="onMouseOut">
61:    ホバーしてみよう
62:</button>
63:<p class="mt-2">{{ hoverMsg }}</p>

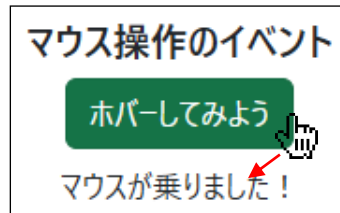
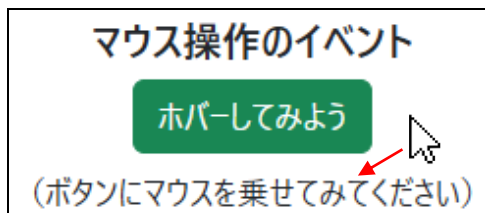
```

イベントは、@click イベント以外にも多数あります。

@mouseover はマウスが要素に乗ったとき、@mouseout はマウスが離れたときに反応します。これらは正反対の動作なので、**同じ要素に2つ同時に設定**することができます。

このサンプルでは、60行目のボタンにマウスを乗せると onMouseOver が呼ばれ、離すと onMouseOut が呼ばれ、63行の表示メッセージが切り替わります。

※mouseover / mouseout は、div、p、img などにも使用できます。



◆【補足】v-on イベントの種類（一部）

イベント名	用途	対応する要素	例
@click	クリック時	button, div など	ボタンを押したとき
@dblclick	ダブルクリック時	全要素	ダブルクリックで画像拡大
@mouseover	マウスが乗ったとき	全要素	ボタンにマウスを重ねたら色変更
@mouseout	マウスが離れたとき	全要素	ボタンからマウスを離したら元の色に
@change	値が変更されたとき	input, select	セレクトボックス変更時
@submit	フォーム送信時	form	入力フォームの送信を処理

【③ @click=“メソッド名(引数)”：引数を渡すパターン】

ch02-basic¥src¥components¥Sample2_5.vue

```

7:// 選択された商品の情報
8:const selected = ref('')

13:// 商品リスト
14:const products = reactive([
15:  { id: 1, name: 'ノートパソコン', price: 120000 },
16:  { id: 2, name: 'マウス', price: 2500 },
17:  { id: 3, name: 'キーボード', price: 4800 }
18:])

40:// 商品がクリックされたとき（引数あり）のメソッド
41:const selectProduct = (name: string, price: number): void => {
42:  selected.value = `選択した商品 : ${name} (${price} 円) `
43:}

77:<tbody>
78:  <tr v-for="p in products" :key="p.id">
79:    <td>{{ p.id }}</td>
80:    <td>{{ p.name }}</td>
81:    <td>{{ p.price }}円</td>
82:    <td>

```

```

83:      <!-- 引数つきでメソッドを呼び出し -->
84:      <button class="btn btn-outline-primary"
           @click="selectProduct(p.name, p.price)">
85:          選択
86:      </button>
87:  </td>
88: </tr>
89:</tbody>

```

41 行の selectProduct メソッドは、引数として「商品名」と「価格」を受け取ります。

42 行は、**文字列の中で JavaScript の変数を簡単に埋め込むことができる、テンプレートリテラル**の構文です。**バッククォート(`)`** で囲って、**`${変数名}`** の書き方で埋め込みます。

```
41:const selectProduct = (name: string, price: number): void => { ~処理~ }
```

メソッドに仮引数がある場合、型を指定することが推奨されます (TypeScript の型注釈)。

v-for の中で @click に p.name と p.price を渡すことで、ボタンをクリックした行の商品データをメソッドに渡すことができます。

これにより、**複数のボタンで同じ関数を使いまわす**ことができます。

(例: 商品を選択・削除・編集するなど)

商品一覧			
ID	商品名	価格	操作
1	ノートパソコン	120000円	選択
2	マウス	2500円	選択
3	キーボード	4800円	選択

選択した商品: ノートパソコン (120000円)

商品一覧			
ID	商品名	価格	操作
1	ノートパソコン	120000円	選択
2	マウス	2500円	選択
3	キーボード	4800円	選択

選択した商品: キーボード (4800円)

ここまで、v-on ディレクティブを使い、イベント処理を行うサンプルを解説しました。

イベント処理の基本は、次の 3 ステップで成り立ちますので、意識しておきましょう。

① ユーザの操作(クリック・マウス操作など)

↓

② メソッドの実行(関数を呼び出す)

↓

③ データが変化し、画面が自動更新される

構文チェック演習

1. `ch02-basic_q¥src¥components¥Sample2_5.vue` を開きます。
2. コメント①～③のソースコードを確認してください。
3. コメント④～⑥の部分の実装を行ってください。
4. 保存後、NPM スクリプトから「`ch02-basic_q → dev`」を実行し、ブラウザで動作を確認してください。
5. イベントが全て問題なく動作するかを確認してください。

2.6. 双方向バインディング(v-model)

双方向バインディング(v-model)

双方向バインディングとは

フォームの入力値と変数の値を **リアルタイムで同期**！ユーザー入力で変数が変わり、変数が変われば入力欄も自動で変わる。

基本構文

```
<input v-model="同期させたい変数名">
```

対応要素: input, textarea, select など。
入力と変数が **常に連動** します。

【実装例 & 動作イメージ】

① テキスト入力

```
<script>
const name = ref('')
</script>
<template>
<input v-model="name"> {{ name }} </template>
```

入力⇔変数が同期

② セレクトボックス

```
<script>
const selectedPrice = ref('3000')
</script>
<template>
<select v-model="selectedPrice">...</select>
</template>
```

選択したvalueが変数に入る

③ ラジオボタン

```
<script>
const category = ref('PC')
</script>
<template>
<input type="radio" value="PC" v-model="category">...
</template>
```

選ばれた項目のvalueが変数に入る

双方向バインディングとは

これまでの v-bind や v-on では、
「データ → 画面」または「画面 → データ」のどちらか一方の連動しかできませんでした。

Vue では、v-model を使うことで、フォームの入力値と変数の値をリアルタイムで同期させることができます。これを「**双方向バインディング**」と呼びます。

つまり、ユーザが入力すれば変数が変わり、変数が変われば入力欄も自動で変わります。

■ 基本構文

```
<input type = "xxxxx" v-model="同期させたい変数名">
```

- v-model は、input、textarea、select、などのフォーム要素で使えます。
- input type = "text"、input type = "radio"、input type = "password"などに使えます。
- 入力内容が変化するたびに、指定した変数の値も自動で変わります。
- 逆に、変数をスクリプト側で変更すると、フォームの値も変わります。

以下のサンプルは、v-model を使って、テキスト入力・セレクトボックス・ラジオボタンを連動させる簡単な例です。

ch02-basic¥src¥components¥Sample2_6.vue

```
1:<script setup lang="ts">
2:import { ref } from 'vue'
3:
4:// 入力フォームと連動するリアクティブ変数
5:const name = ref('')           // テキスト入力
6:const selectedPrice = ref('3000') // セレクトボックス
7:const category = ref('PC')      // ラジオボタン
8:</script>
9:
10:<template>
11:  <div class="text-center">
12:    <h3>v-model の基本</h3>
13:
14:    <!-- テキスト入力 -->
15:    <p>① テキスト入力</p>
16:    <input type="text" v-model="name" placeholder="お名前を入力"
17:           class="form-control w-50 mx-auto" />
18:
19:    <p>こんにちは、{{ name }} さん！</p>
20:
21:    <hr>
22:
23:    <!-- セレクトボックス -->
24:    <p>② セレクトボックス（価格帯）</p>
25:    <select v-model="selectedPrice" class="form-select w-50 mx-auto">
26:      <option value="0">指定なし</option>
27:      <option value="3000">3,000 円以上</option>
28:      <option value="10000">10,000 円以上</option>
29:    </select>
30:    <p>選択中の価格条件：{{ selectedPrice }} 円以上</p>
31:
32:    <hr>
33:
34:    <!-- ラジオボタン -->
35:    <p>③ ラジオボタン（カテゴリ）</p>
36:    <div>
```

```

35:   <input type="radio" value="PC" v-model="category" /> PC
36:   <input type="radio" value="Accessory" v-model="category" /> アクセサリ
37:   <input type="radio" value="Display" v-model="category" /> ディスプレイ
38: </div>
39: <p>選択中のカテゴリ : {{ category }}</p>
40: </div>
41:</template>

```

💡 構文の解説

5～6 行目: 入力フォームと連動するリアクティブ変数の定義

```

5:const name = ref('')           // テキスト入力
6:const selectedPrice = ref('3000') // セレクトボックス
7:const category = ref('PC')      // ラジオボタン

```

まず、フォームと変数を結びつけるために、リアクティブ変数をあらかじめ定義します。

3つのフォームがあるため、それぞれに対応する変数を用意します。

16 行目: v-model で連動し、画面に表示

```

16:   <input type="text" v-model="name" placeholder="お名前を入力"
      class="form-control w-50 mx-auto" />
17:   <p>こんにちは、{{ name }} さん!</p>

```

ユーザが入力フォームに文字を入力すると、リアクティブ変数 name が即座に更新され、その値が {{ name }} にもリアルタイムで反映されます。

① テキスト入力 お名前を入力 こんにちは、 さん!		① テキスト入力 山田太郎 こんにちは、山田太郎 さん!
--	--	--

「入力した瞬間に表示が変わる」。これが双方向バインディングの特徴です。

23-28 行目 : select 要素内から v-model で連動し、画面に表示

```

23: <select v-model="selectedPrice" class="form-select w-50 mx-auto">
24:   <option value="0">指定なし</option>
25:   <option value="3000">3,000 円以上</option>
26:   <option value="10000">10,000 円以上</option>
27: </select>
28: <p>選択中の価格条件 : {{ selectedPrice }} 円以上</p>

```

select 要素の場合、選択した option の value がリアクティブ変数に代入されます。
メニューを切り替えると、選択内容が即座に反映されます。

② セレクトボックス (価格帯) 3,000円以上 ▼ 選択中の価格条件 : 3000円以上		② セレクトボックス (価格帯) 10,000円以上 ▼ 選択中の価格条件 : 10000円以上
---	--	---

35-39 行目 : input radio と v-model で連動し、画面に表示

```

34: <div>
35:   <input type="radio" value="PC" v-model="category" /> PC
36:   <input type="radio" value="Accessory" v-model="category" /> アクセサリ
37:   <input type="radio" value="Display" v-model="category" /> ディスプレイ
38: </div>
39: <p>選択中のカテゴリ : {{ category }}</p>

```

複数のラジオボタンを同じ v-model に指定すると、
1 つだけ選ばれた項目の value が変数に入ります。
この場合、選択したカテゴリ名 (PC、Accessory など) が category に自動的に入ります。

③ ラジオボタン (カテゴリ) ☒ PC ☐ アクセサリ ☐ ディスプレイ 選択中のカテゴリ : PC		③ ラジオボタン (カテゴリ) ☐ PC ☐ アクセサリ ☒ ディスプレイ 選択中のカテゴリ : Display
---	--	--

 構文チェック演習

1. ch02-basic_q¥src¥components¥Sample2_6.vue を開きます。
2. コメント①～④のソースコードを確認・実装してください。
3. 保存後、NPM スクリプトから「ch02-basic_q → dev」を実行し、ブラウザで動作を確認します。
4. 3つのフォームの入力値を変える度に、リアルタイムでテキストが変動するかを確かを確認してください。

 【補足】双方向バインディングでできること

v-model による双方向バインディングを使うと、
ユーザ操作とアプリの状態を完全に一致させることができます。

たとえば次のような場面で活躍します。

用途	説明
フォーム入力画面	名前・住所・価格などを入力しながら即時プレビュー表示が可能
検索・絞り込み機能	入力内容をリアルタイムで反映し、リストを動的に更新
設定画面	トグルスイッチやチェックボックスと設定値を連動
編集フォーム	既存データを表示して、そのまま変更内容を反映

章のまとめ

章のまとめ



- Vueとは
- Vueコンポーネントの基本
- リアクティブなデータ管理
- ディレクティブ構文
 - v-bindディレクティブ
 - v-if/v-else/v-showディレクティブ
 - v-forディレクティブ
- イベント処理(v-on)
- 双方向バインディング(v-model)

この章では、Vue3 の基本となる構文について学習しました。

第3章 コンポーネント分割とデータの受け渡し

章の目標

- Vue のアプリを複数のコンポーネントに分けて管理できる
- 親子間でデータをやり取りする方法を理解し、使い分けられる
- 複数の子コンポーネントを持つ画面を実装できる

章の概要

章の概要



- コンポーネントの分割
- 親から子へ値を渡す(props)
- 【参考】子から親へイベントを送る(emit)
- 複数の子コンポーネントを配置する

この章では、Vue アプリを複数の「コンポーネント」に分けて管理する方法を学びます。Vue では、画面全体を 1 つのファイルで作るのではなく、Header、Sidebar、Main、Footer のように役割ごとに分割して再利用するのが一般的です。

まず、コンポーネントを分けて構成する方法を学びます。次に、親コンポーネントから子コンポーネントへ値を渡す props、そして子から親へイベントを送る emit の使い方を理解します。

この章を学ぶことで、Vue の親子コンポーネントのデータの流れを理解し、アプリを構造的に設計できるようになります。

3.1. コンポーネントの分割

コンポーネントの分割

コンポーネント分割とは？

- 画面を機能ごとの部品（コンポーネント）に分けて管理。
- **コード整理、保守性・再利用性向上。**

```

src/
├── App.vue           (親コンポーネント)
└── components/
    ├── HeaderMenu.vue (子コンポーネント)
    ├── SideMenu.vue   (子コンポーネント)
    ├── MainContent.vue (子コンポーネント)
    └── FooterInfo.vue  (子コンポーネント)
          
```

各領域が独立したコンポーネントとして機能

実装の流れ (App.vue - 親コンポーネント)

- 子コンポーネントの **import** (<script setup>)

```

<script setup lang="ts">
// 重要: importと名前定義
import HeaderMenu from './components/HeaderMenu.vue'
import SideMenu from './components/SideMenu.vue'
// ...他も同様
</script>
          
```

- template** での配置 (要素として使用)

```

<template>
<div class="container...">
  <!-- 重要: 要素名として配置 -->
  <HeaderMenu />
  <div class="row">
    <SideMenu /> <!-- 配置場所を指定 -->
    <MainContent />
  </div>
  <FooterInfo />
</div>
</template>
          
```

結果: 子コンポーネントの内容が指定場所に挿入される

コンポーネントを分割すること

これまでの第2章では、Vueの基本構文を学ぶために、複数のサンプルを1つのプロジェクト内で切り替えて表示していました。

その時実は、App.vueの中で各サンプルを呼び出す仕組みとして、**コンポーネントの分割**をすでに使っていました。

Vueでは、**画面の一部分を1つのコンポーネントとして扱い、必要な場所で読み込んで使う**ことができます。

たとえば、Header、Sidebar、Main、Footerのように画面を分けておくと、コードが整理され、保守や再利用がしやすくなります。

この節では、App.vueを親コンポーネントとして、HeaderMenu.vueやMainContent.vueなどの子コンポーネントを作り、画面を組み立てる基本構成を確認します。

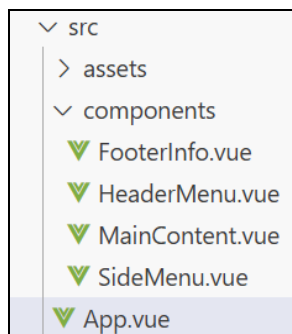
この構成を理解することで、Vueアプリ全体を「役割ごとに整理された部品の集合」として設計できるようになります。

コンポーネント分割の画面例

以下の画面は、1 つの HTML ファイルに全ての HTML が記述されているわけではなく、

App.vue という親画面(親コンポーネント)に、

「HeaderMenu」子コンポーネント、「SideMenu」子コンポーネント、「MainContent」子コンポーネント、「FooterInfo」子コンポーネントを呼び出して配置されている画面です。



Vue3 では、親コンポーネントの役割は、src フォルダの直下の「App.vue」が担当し、子コンポーネントは、src/components フォルダ内に作成します。

※もちろん、子コンポーネントの中に孫コンポーネントを配置することもできます。

親コンポーネントの作成例

親コンポーネントが子コンポーネントを読み込むためには、

以下のサンプルのように、**import**を行い、**子コンポーネントの vue ファイルを指定**する必要があります。

その後、import した名前と同じ要素名を配置すれば、子コンポーネントの HTML が挿入される仕組みです。

ch03-components¥src¥App.vue

```
1:<script setup lang="ts">
2:import HeaderMenu from './components/HeaderMenu.vue'
3:import SideMenu from './components/SideMenu.vue'
4:import MainContent from './components/MainContent.vue'
5:import FooterInfo from './components/FooterInfo.vue'
6:</script>
7:
8:<template>
9:  <div class="container-fluid bg-light">
10:    <HeaderMenu />
11:
12:    <!-- グリッドシステムで横並び -->
13:    <div class="container my-4">
14:      <div class="row">
15:        <div class="col-md-3">
16:          <SideMenu />
17:        </div>
18:        <div class="col-md-9">
19:          <MainContent />
20:        </div>
21:      </div>
22:    </div>
23:
24:    <FooterInfo />
25:  </div>
26:</template>
27:<style scoped>
28:
29:</style>
```

💡 構文の解説

2～5 行目 : 子コンポーネントの Vue ファイルの import

ch03-components¥src¥App.vue

```
2: import HeaderMenu from './components/HeaderMenu.vue'
3: import SideMenu from './components/SideMenu.vue'
4: import MainContent from './components/MainContent.vue'
5: import FooterInfo from './components/FooterInfo.vue'
```

App.vue の先頭には、子コンポーネントを記述している、各 vue ファイルの格納場所を import しておく必要があります。

import の直後の名称は、vue ファイル名と一致させ、クラス名のように記述することが推奨されます。

9～25 行目 : 子コンポーネントの挿入

```
15: <div class="col-md-3">
16:   <SideMenu />
17: </div>
```

import 後には、import 時に指定したクラス名がそのまま要素名となります。

16 行目のように、<子コンポーネントの要素名 /> を記載すると、子コンポーネント (SideMenu.vue) 内のコードが読み込まれます。

※今回のサンプルでは、見栄えのために若干の Bootstrap の class を指定しています。Bootstrap の class の説明については、割愛します。

子コンポーネントの例

ch03-components¥src¥components¥SideMenu.vue

```

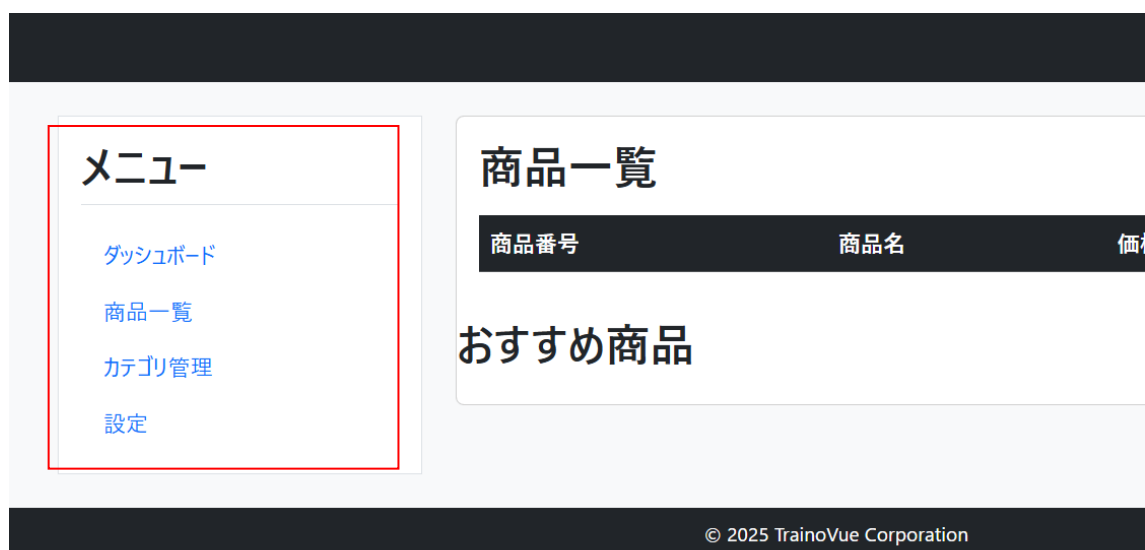
1:<script setup lang="ts">
2:
3:</script>
4:
5:<template>
6:   <!-- サイトメニューの子コンポーネント -->
7:<aside class="bg-white border p-3 h-100">
8:   <h2 class="border-bottom pb-2 mb-3">メニュー</h2>
9:   <ul class="nav flex-column">
10:    <li class="nav-item"><a href="#" class="nav-link">ダッシュボード</a></li>
11:    <li class="nav-item"><a href="#" class="nav-link">商品一覧</a></li>
12:    <li class="nav-item"><a href="#" class="nav-link">カテゴリ管理</a></li>
13:    <li class="nav-item"><a href="#" class="nav-link">設定</a></li>
14:  </ul>
15:</aside>
16:</template>

```

子コンポーネントは、src/components フォルダ内に格納するのが推奨されています。

vue ファイルですので、<script>と<template>要素で構成されます。

親コンポーネントである App.vue の 16 行で<SideMenu/>と指定した場所に、この SideMenu.vue 内の内容が呼び出され、以下のように配置されます。



他の子コンポーネントである、「HeaderMenu.vue」「MainContent.vue」「FooterInfo.vue」も同様に画面内に読み込まれますので、他の子コンポーネントの説明は割愛します。

《📄 構文チェック演習》

1. ch03-components_q¥src¥App.vue を開きます。
2. コメント【3.1 ①～④】に従って、HeaderMenu・SideMenu・MainContent・FooterInfo を作成し、読み込みましょう。
3. 保存後、NPM スクリプトから ch03-components_q → dev を実行し、Vite サーバーを起動します。
4. ブラウザで <http://localhost:5177/> にアクセスします。
5. 「商品管理システム」のヘッダーと、サイドメニュー、フッター、商品一覧が Bootstrap のテーブルで表示されることを確認してください。

3.2. 親から子へ値を渡す (props)

親から子へ値を渡す (props)

■ コンポーネントは、**親子の役割が異なる**

- 親コンポーネント：データの管理
- 子コンポーネント：データの表示、操作

■ props

- 親→子へデータを渡すために使う仕組み
- 子コンポーネント要素に**v-bind**でデータを付与すると渡せる
- 単体のデータを複数渡すことも、配列ごと渡すことも可能

```
const pageTitle = '商品管理システム'
const userName = '管理者'
```

親コンポーネント

```
<HeaderMenu :title="pageTitle" :user="userName" />
```

```
//TypeScriptを使う場合、以下のように型の定義が必要
type HeaderData = {
  title: string
  user: string
}
```

子コンポーネント

```
// 親から受け取るpropsの定義
const props = defineProps<HeaderData>()
```

props とは

Vue では、画面を部品 (コンポーネント) ごとに分けて開発します。

ただし、それぞれが独立していると、お互いデータを参照できません。そして、独立して動いているだけでは、アプリ全体の整合が取れません。

たとえば、ヘッダーには「画面タイトル」や「ログイン中のユーザ名」を表示し、メイン部分には「商品一覧」を表示したいとき、これらの**情報は App.vue (親) で一元管理し、各子コンポーネントに渡すことが推奨**されます。

データの管理は親で、見た目と操作の担当は子に任せることで、整合性・再利用・役割の明確化が向上され、開発効率と保守性が向上するメリットがあります。

このように、**親 → 子 にデータを渡す仕組みが「props (プロップス)」**といいます。props は Vue の最も基本的なコンポーネント間通信であり、アプリ設計の基礎となる考え方です。

単体のデータを子コンポーネントへ渡す時

最初に、単体のデータを親コンポーネントに定義し、そのデータを子コンポーネントへ渡すサンプルコードを説明します。

商品管理システム

ようこそ、管理者さん

ch03-components¥src¥App.vue

```
8: // 親で管理するデータ
9: const pageTitle = '商品管理システム'
10: const userName = '管理者'
.....
.....
.....
25: <!--3.2 親 → 子: タイトルとユーザー名を渡す -->
26: <HeaderMenu :title="pageTitle" :user="userName" />
```

9 行と 10 行目では、親コンポーネントで単純な文字列データを定義しています。
26 行目では、子コンポーネントを呼び出す `<HeaderMenu />` のタグ内に、
`v-bind` を使って値を渡しています。

構文は `v-bind:属性名="変数名"` ですが、省略形として `:title="pageTitle"` のように書けます。
`:user="userName"` も同様の書き方です。

ここで、**属性名の title や user は、キーの役割**をしていますので、任意で決めて大丈夫です。
このように、**親コンポーネントの変数を子コンポーネントに渡す**ときは、
子コンポーネントの要素(タグ)に `v-bind` を使って変数をバインドします。

単体のデータを親コンポーネントから受け取る方法

親コンポーネントから渡されたデータは、Vue 内部で「props(プロップス)」という仕組みで受け取られます。

子コンポーネント側では、この props を定義して利用できるようにします。

ch03-components¥src¥components¥HeaderMenu.vue

```

1:<script setup lang="ts">
2://TypeScript を使う場合、以下のように型の定義が必要
3:type HeaderData = {
4:  title: string
5:  user: string
6:}
7:
8:// 親から受け取る props の定義
9:const props = defineProps<HeaderData>()
10:</script>
.....
15:  <div class="col-6">
16:    <h1 class="h5 m-0">{{ props.title }}</h1>
17:  </div>
18:  <div class="col-6 text-end">
19:    <span class="small">ようこそ、{{ props.user }} さん</span>
20:  </div>

```

子コンポーネントで props を受け取る際は、**defineProps()** 関数を使います。(9 行) 構文は次の通りです。

```
const props = defineProps<型名>()
```

ここで **<型名>** には、親から送られてくるデータ構造を表す型を指定します。

この例では、親から ヘッダー向けの title と user の 2 つの文字列データが送られてくるので、HeaderData という名前の型を定義して、それぞれの型を string としています(3～6 行)。

もし、型の定義を省略して const props = defineProps<HeaderData>() だけを書くと、TypeScript が HeaderData の中身を認識できず、型エラーになります。そのため、**あらかじめ HeaderData 型を定義**しておく必要があります。(3～6 行)。

定義が完了すれば、16 行と 19 行のように、props の中にある値を **{{ props.title }}** や **{{ props.user }}** の形で HTML に埋め込むことができます。

🔗【補足】 props の型定義を行う理由

TypeScript 式の定義をしておくことで、props の型チェックやコードの補完ができます。誤った型のデータを親から渡した場合でも、コンパイル時点でエラーを検出できるようになります。

配列・オブジェクトデータを子コンポーネントへ渡す時

単体の文字列や数値と違い、**配列やオブジェクトを渡す場合**は、子コンポーネント側の型指定と v-for の使い方がポイントになります。

以下のサンプルでは、商品一覧(配列)を MainContent に渡して表示する例を説明します。

ch03-components¥src¥App.vue

```
12:// 商品一覧データ (このあと MainContent に渡す)
13: const products = ref([
14: { id: 1, name: 'ノートパソコン', price: 120000, category: '電子機器' },
15: { id: 2, name: 'スマートフォン', price: 90000, category: '電子機器' },
16: { id: 3, name: 'オフィスチェア', price: 25000, category: '家具' },
17: { id: 4, name: 'ボールペンセット', price: 800, category: '文房具' }
18: // 【ここまでは、3.2 で利用するデータ】
19:])
.....
34:<div class="col-md-9">
35:   <!-- このあと MainContent にも商品一覧を渡す -->
36:   <MainContent :products="products" />
37:</div>
```

13-19 行目では、親コンポーネントに **商品配列 products** を定義します。今回のサンプルでは固定値ですが、実務では API から取得する想定 of 配列です。更新時に描画も同時に変えられるよう、ref でリアクティブ変数化しています。

36 行目では、<MainContent />の中に、「:products="products"」で、**配列全体を props** として子へ渡します。属性名(キー)と、値(変数名)を一致させています。

単体データのとときと同じく、v-bind の省略形です。

子コンポーネントから配列の props を受け取る

商品一覧

商品番号	商品名	価格	カテゴリ
1	ノートパソコン	120000 円	電子機器
2	スマートフォン	90000 円	電子機器
3	オフィスチェア	25000 円	家具
4	ボールペンセット	800 円	文房具

ch03-components¥src¥components¥MainContent.vue

```
1:<script setup lang="ts">
2:// 商品配列の型定義
3:type Product = {
4:   id: number;
5:   name: string;
6:   price: number;
7:   category: string
8:}
9:
10:// 親から受け取る props を定義
11:const props = defineProps<{ products: Product[] }>()
12:</script>
.....
29:<tbody>
30:  <tr v-for="p in props.products" :key="p.id">
31:    <td>{{ p.id }}</td>
32:    <td>{{ p.name }}</td>
33:    <td>{{ p.price }} 円</td>
34:    <td>{{ p.category }}</td>
35:  </tr>
36: </tbody>
```

3-8 行では、単体のデータと同じく、任意の型である、Product 型を定義しています。渡される配列がどんな型を持っているのか、中身の要素まで型を指定しておく必要があります。こうしておくと、誤った型ミスを防げます。

11 行では、「`defineProps<{ products: Product[] }>()`」で配列を受け取っています。

「受け取るのは「Product 型の配列で、受け取るキー名は products」という意味のコードです。

const props に代入していますので、30 行の `v-for` で指定する配列名は、`props.products` と指定しています。

`:key="p.id"` は、項目ごとの特定やパフォーマンス向上のためにつけるオプションだと思ってください。

《📄 構文チェック演習》

1. `ch03-components_q$src$App.vue` を開きます。

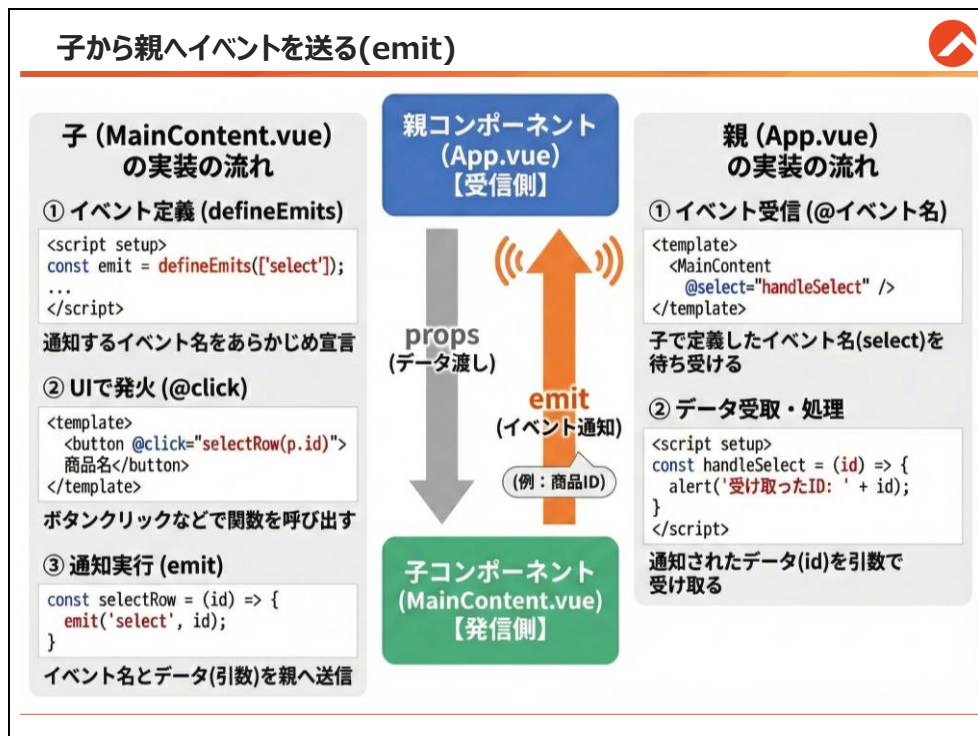


2. 上記の図を参照し、コメント【3.2 ①】に従って、HeaderMenu に渡すデータを定義してください。
3. コメント【3.2 ②】のソースコードを確認してください。
4. コメント【3.2 ③～④】の指示に従い実装し、保存します。
5. `ch03-components_q$src$components$HeaderMenu.vue` を開きます。
6. コメント【3.2 ⑤～⑧】指示通りに、実装し、保存します。
7. `ch03-components_q$src$components$MainContent.vue` を開きます。
8. 配列のデータの型の定義、props の受け取り方、`v-for` で表示しているソースコードを確認してください。

9. NPM スクリプトから `ch03-components_q → dev` を実行し、Vite サーバーを起動します。
10. ブラウザで <http://localhost:5173/> にアクセスします。
11. ヘッダーとメインコンテンツが正しく表示されていることを確認してください。

※商品名がリンク風になっているのは、後続の問題のためですので、無視しても大丈夫です

3.3. 【参考】 子から親へイベントを送る(emit)



emit とは

前節で学習した「props」は、親 → 子 にデータを渡す仕組みでした。

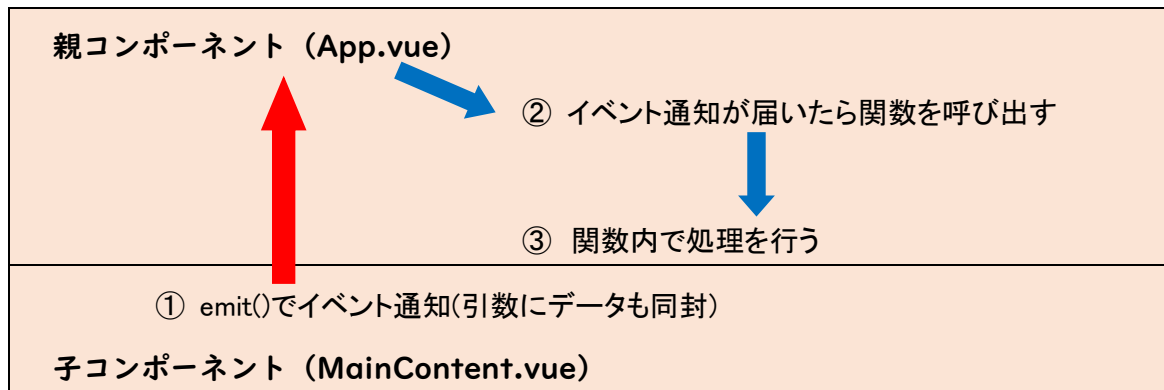
一方で、**子 → 親 に何かを伝えたいとき**もあります。

たとえば、「子のボタンをクリックしたら、親に知らせたい」「一覧の中でどの商品が選ばれたかを親に伝えたい」といった場面です。

このように、**子コンポーネントから親にイベントを通知する仕組みを emit(エミット)** と呼びます。

「emit＝発信する」という意味で、子から親に“何か起きた”と知らせる仕組みです。

Vue のデータの流れは、常に 一方向(親→子) です。子が親のデータを直接書き換えることはできません。その代わりに、子で起きたイベントを emit で知らせて、親がその通知を受け取って処理を行う、という構成を取るわけです。



子コンポーネント内の emit の定義とイベントの通知方法

今回は、MainContent.vue に各行(商品名)をリンク風にしてクリックができるように修正しました。

商品名をクリックすると、子から親に「どんな商品がクリックされたのか」を知らせるサンプルで説明します。

ch03-components¥src¥components¥MainContent.vue

```

14: // 親へ通知するイベントを定義
15: const emit = defineEmits(['select'])
16:
17: // ボタンが押されたときに emit で親に ID を通知
18: const selectRow = (id: number) => {
19:   emit('select', id)
20: }
.....
39: <tbody>
40: <tr v-for="p in props.products" :key="p.id">
41:   <td>{{ p.id }}</td>
42:   <td>
43:     <!-- 商品名をリンク風にして選択可能に -->
44:     <button class="btn btn-link"
45:               @click="selectRow(p.id)">{{ p.name }}</button>
46:   </td>
47:   <td>{{ p.price }} 円</td>
48:   <td>{{ p.category }}</td>

```

```
48: </tr>
49: </tbody>
.....
```

15行目の `defineEmits([...])` は「子コンポーネントが親へ通知してよいイベント名の一覧」をあらかじめ宣言する関数です。配列やオブジェクト化する必要があり、今回は `['select']` と登録しています。

```
17: // ボタンが押されたときに emit で親に ID を通知
18: const selectRow = (id: number) => {
19:   emit('select', id)
20: }
```

18行-20行目は、親へ通知を行う、`emit` 関数を呼び出す関数の定義ですが、関数の仮引数として、数値を入れるように定義しています。TypeScript 式で、型の指定も行っています。

19行目の `emit` 関数の第1引数には、イベント名。第2引数通知の際に一緒に送りたい値を記載できます。

イベント名の文字列 `'select'` は、15行で `defineEmits()` で宣言した名前と一致している必要があります。

商品一覧

商品番号	商品名	価格	カテゴリ
1	ノートパソコン	120,000 円	電子機器
2	スマートフォン	90,000 円	電子機器
3	オフィスチェア	25,000 円	家具
4	ボールペンセット	800 円	文房具

```
40: <tr v-for="p in props.products" :key="p.id">
41:   <td>{{ p.id }}</td>
42:   <td>
43:     <!-- 商品名をリンク風にして選択可能に -->
44:     <button class="btn btn-link"
45:               @click="selectRow(p.id)">{{ p.name }}</button>
45:   </td>
```

44行は、`v-for` の中で繰り返し商品名を表示するコードでしたが、そこにボタンを追記し、リンク風にしてクリックイベントを付与しました。

クリックしたら、`id` の引数を入れて、`selectRow` 関数を実行させます。

親子コンポーネントで emit のイベント通知を受け取る方法

ch03-components¥src¥App.vue

```

21:// 【3.3】子から通知された「選択 ID」を受け取る
22:const handleSelect = (id: number) => {
23:  alert(`親で実行しています。¥n 選択された商品 ID は ${id} です。`)
24:}
25:
26:</script>
.....
43:<!-- 【3.3 のサンプルコード】 -->
44: <MainContent :products="products" @select="handleSelect"/>

```

既存の App.vue に、コードを追記しました。

まず、44 行目の子コンポーネントの配置の部分のコードに、select イベントを付与します。

「select」は、子コンポーネントで設定していたイベント名と合わせています。

子コンポーネントからのイベントの通知があれば、handleSelect 関数を実行しますが、その際は以下のコードで emit の空間に格納していた部分と連動しているため、22 行目では仮引数を設定しているわけです。

ch03-components¥src¥components¥MainContent.vue

```

19: emit('select', id)

```

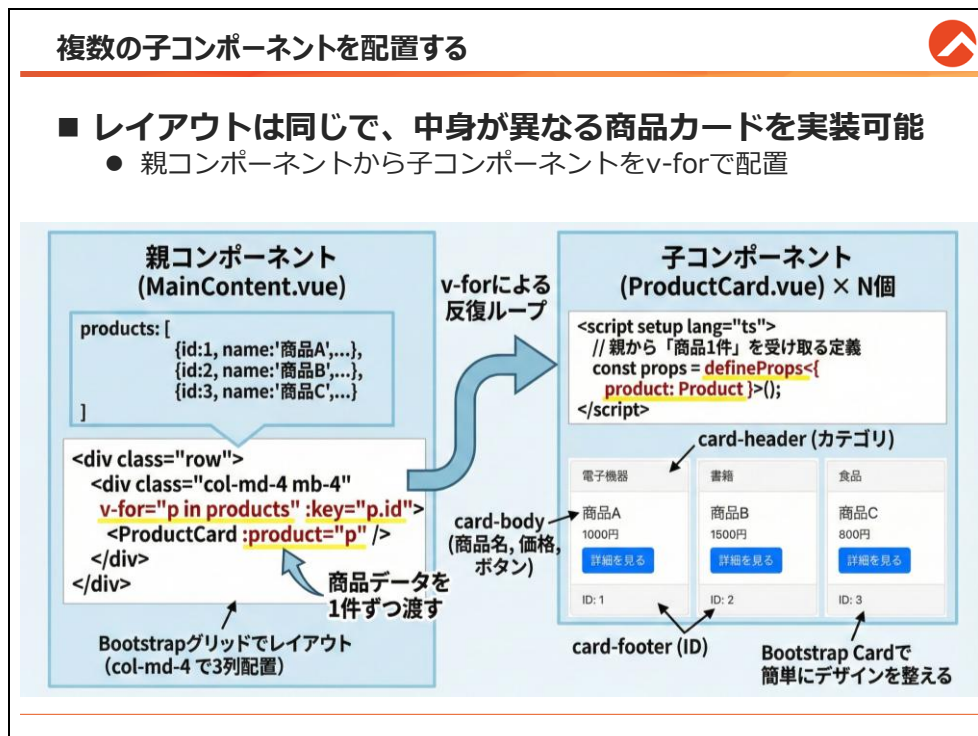


よって、子コンポーネントのどの商品名をクリックしたかを、親コンポーネントの方から特定できるようになり、データベースへの検索といった、後続の処理に繋がられるわけです。

《📄 構文チェック演習》

1. `ch03-components_q¥src¥components¥MainContent.vue` を開きます。
2. コメント【3.3 ①～③】指示通りに、実装し、保存します。
3. `ch03-components_q¥src¥App.vue` を開きます。
4. コメント【3.3 ④～⑤】指示通りに、実装し、保存します。
5. NPM スクリプトから `ch03-components_q → dev` を実行し、Vite サーバーを起動します。
6. ブラウザで <http://localhost:5173/> にアクセスします。
7. 商品名をクリックすると、該当する商品番号が正しくメッセージ表示されるか確認してください。

3.4. 複数の子コンポーネントを配置する

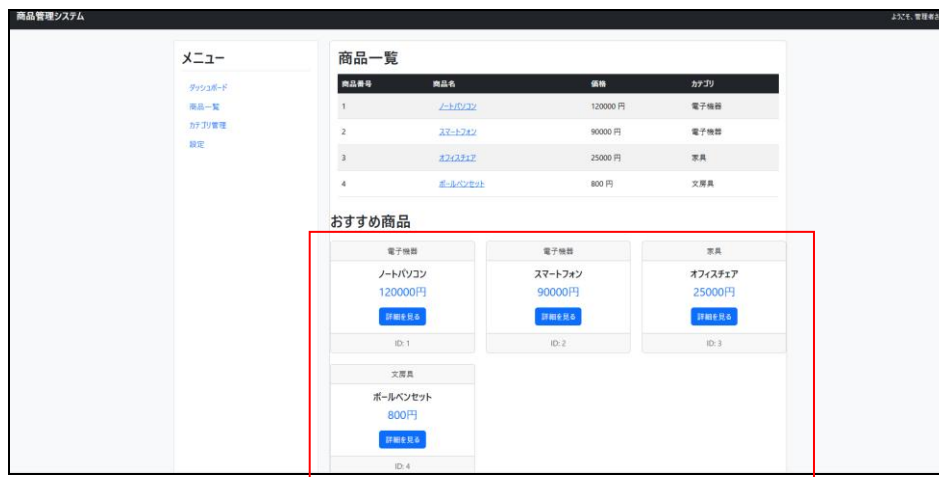


コンポーネントの反復レンダリング

第2章では、HTML要素(<tr>や)に対してv-forを使用して一覧表示を行いました。Vue.jsの強力な機能として、このv-forはコンポーネント自体に対しても使用できます。

例えば、ECサイトの商品カードのように「レイアウトは同じだが、中身のデータ(商品画像や名前)だけが異なる」場合、1つの「カード部品(子コンポーネント)」を作成し、それを親コンポーネント側で配列の数だけ繰り返し表示することで、効率的に画面を構築できます。

親コンポーネントが「データの配列」を持ち、v-forでループしながら、1件ずつのデータを子コンポーネントのpropsに渡す仕組みです。



子コンポーネントの作成(ProductCard.vue)

まず、商品 1 件分の表示を担当する「カード型」の子コンポーネントを作成します。このコンポーネントは、親から「商品データ 1 件(オブジェクト)」を受け取り、Bootstrap の Card スタイルで表示します。

ch03-components¥src¥components¥ProductCard.vue

```
1:<script setup lang="ts">
2:  // 商品単体の型定義
3:  type Product = {
4:    id: number;
5:    name: string;
6:    price: number;
7:    category: string;
8:  }
9:
10:  // 親から「商品 1 件」を受け取る定義
11:  const props = defineProps<{ product: Product }>();
12:</script>
13:
14:<template>
15:  <div class="card h-100 text-center">
16:    <div class="card-header">{{ props.product.category }}</div>
17:    <div class="card-body">
18:      <h5 class="card-title">{{ props.product.name }}</h5>
19:      <p class="card-text text-primary fs-4">{{ props.product.price }}円</p>
20:      <button class="btn btn-primary">詳細を見る</button><!--形だけ実装-->
21:    </div>
22:    <div class="card-footer text-muted">ID: {{ props.product.id }}</div>
23:  </div>
24:</template>
```



💡 構文の解説

親から商品 1 件を受け取る

```
11: const props = defineProps<{ product: Product }>();
```

ここが重要です。親から for 文で商品データが送信されますが、コンポーネントは配列全体ではなく、「分解された 1 件分の商品データ(Product 型)」を受け取るように定義しています。

受け取った商品データを当てはめる

```
15: <div class="card h-100 text-center">
16:   <div class="card-header">{{ props.product.category }}</div>
17:   <div class="card-body">
18:     <h5 class="card-title">{{ props.product.name }}</h5>
19:     <p class="card-text text-primary fs-4">{{ props.product.price }}円</p>
20:     <button class="btn btn-primary">詳細を見る</button><!--形だけ実装-->
21:   </div>
22:   <div class="card-footer text-muted">ID: {{ props.product.id }}</div>
23: </div>
```

EC サイトの商品一覧や、ブログの記事一覧のように、「画像・タイトル・説明・ボタン」がセットになった情報を並べる際、ゼロから CSS でデザインするのは非常に手間がかかります。枠線を書き、余白(パディング)を調整し、タイトルと本文のフォントサイズを変え……といった作業が必要です。

Bootstrap の **Card コンポーネント** を使う理由は、「情報のまとまりを、手間をかけずに美しく見せるため」です。クラス名を指定するだけで、適切な余白や境界線が自動的に適用され、整った見た目の「情報の箱」を一瞬で作ることができます。

Card は、基本的に 3 つの階層(ヘッダー・ボディ・フッター)で構成されています。今回のコードと照らし合わせると、以下ようになります。

クラス名	役割	今回のコードでの用途
.card	外枠(コンテナ)	商品 1 件分の枠組みを作ります。

.card-header	ヘッダー(見出し帯)	カテゴリ名(例:「電子機器」)を表示し、背景色をつけて区別しやすくしています。
.card-body	メインコンテンツ	商品名、価格、ボタンなど、主要な情報を配置する場所です。
.card-footer	フッター	ID などの補足情報を下部に控えめに表示します。
.h-100	高さを均等に揃える	要素(グリッドの列)の高さいっぱいカードを広げ、横並びになった時の高さを綺麗に揃えています。

親コンポーネントへ配置(MainContent.vue が親)

これまでは App.vue (親) の中に MainContent.vue (子) を配置していましたが、さらにその中に ProductCard.vue (孫) を配置することで、より階層的な画面作りを行います。

この構成にすることで、将来的に「商品データ」が API から取得されるようになって、修正範囲を MainContent.vue だけに限定でき、保守性が向上します。

ch03-components¥src¥components¥MainContent.vue

```
2: import ProductCard from './ProductCard.vue';
.....
52: </table>
53: </div>
54: <h2 class="mb-4">おすすめ商品</h2>
55: <div class="row">
56:   <div class="col-md-4 mb-4" v-for="product in products" :key="product.id">
57:     <ProductCard :product="product" />
58:   </div>
59: </div>
```

💡 構文の解説

2 行目では、ProductCard.vue を配置するため、import しています。

そこから、52 行目までは商品テーブルのソースコードが続き、54 行目に「おすすめ商品」という部分を追記しました。

56-57 行目では、v-for の中で子コンポーネント(ProductCard)を配置しています。

商品データが 1 件ずつ、v-bind され、子コンポーネントの方に送信されます。

56 行目の「col-md-4」は、1 行に 4:4:4 の割合で 3 つずつ配置するデザインにしました。この数値を調整することで、1 行に何個の子コンポーネントを配置するかが決まります。



《📄 構文チェック演習》

1. `ch03-components_q¥src¥components¥ProductCard.vue` を開きます。
2. コメント【3.4 ①～⑤】指示通りに、実装し、保存します。
3. `ch03-components_q¥src¥MainContent.vue` を開きます。
4. コメント【3.4 ⑥～⑧】指示通りに、実装し、保存します。
5. NPM スクリプトから `ch03-components_q → dev` を実行し、Vite サーバーを起動します。
6. ブラウザで <http://localhost:5177/> にアクセスします。
7. 複数の商品カードコンポーネントが正しく配置されているか確認してください。

章のまとめ

章のまとめ



- コンポーネントの分割
- 親から子へ値を渡す(props)
- 【参考】子から親へイベントを送る(emit)
- 複数の子コンポーネントを配置する

この章では、Vue3 のコンポーネントについて学びました。

アプリを複数の部品に分けて管理する「コンポーネント分割」から始まり、親から子へデータを渡す props、子から親へイベントを通知する emit の仕組みを学習しました。

これらを組み合わせることで、親がデータを管理し、子が表示や操作を担当するという Vue らしいコンポーネント間通信の流れを学習しました。

今回学習した内容は、今後のアプリ開発で非常に重要な考え方です。慣れてきたら、props や emit を使って、画面間でのやり取りや複雑な UI の制御にも挑戦してみましょう。

第4章 ルーティングと SPA

章の目標

- Vue Router を導入して、ページ遷移の仕組みを理解できる
- ルートとコンポーネントを対応づけて表示を切り替えられる
- メニュー操作でページを動的に切り替える SPA 構成を実装できる
- ページ間のデータ共有ができる
- ルートパラメータを使って、ページ間で値を受け渡せる

章の概要

章の概要

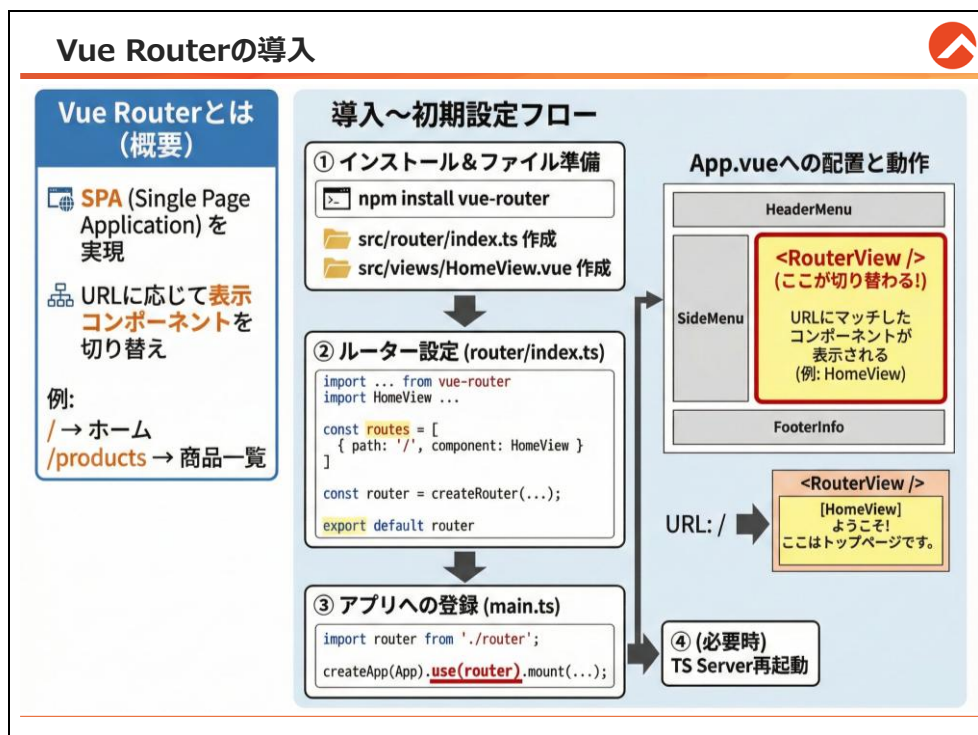


- Vue Routerの導入
- ページ（ルート）とコンポーネントの違い
- ルート設定によるページ切り替え
- ナビゲーションメニューの作成
- provideとinjectによるデータ共有
- パラメータ付きルート

この章では、Vue Router を使って、複数のページを持つアプリを作成する方法を学びます。Vue では、画面を切り替えるたびにブラウザを再読み込みするのではなく、ページごとにコンポーネントを切り替えることで動作する SPA (Single Page Application) を構築できます。まず、Vue Router を導入してルート(URL)とコンポーネントを関連づける方法を学びます。次に、メニュー操作でページを切り替える方法や、ルートパラメータを使って詳細ページを表示する方法を確認します。

この章を学ぶことで、Vue アプリを複数ページ構成に拡張し、実践的な画面遷移を実装できるようになります。

4.1. Vue Router の導入



Vue Router とは

Vue アプリで複数のページを切り替えられるようにするには、**Vue Router(ビュー・ルーター)**という仕組みを使います。

Vue Router を使うと、URL に応じて表示するコンポーネントを切り替えられるようになり、SPA (Single Page Application)として動作できます。

たとえば、以下のように URL ごとに異なる画面を表示できるようになります。

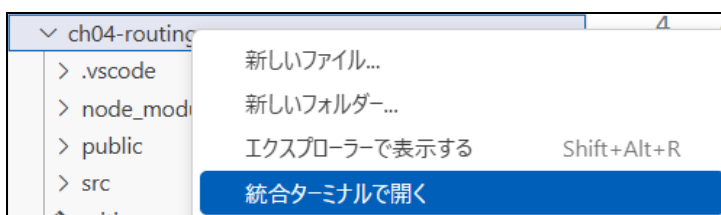
- / … トップページ(ホーム画面)
- /products … 商品一覧ページ
- /products/1 … 商品 id が 1 の詳細ページ

Vue Router のインストール

まずは、既存のプロジェクトに Vue Router を追加します。

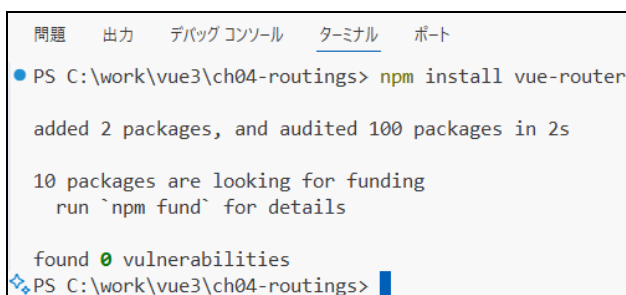
【作業手順①】 npm からインストール

今回は、ch04-routings プロジェクトを利用するため、ch04-routings フォルダを右クリックし、「総合ターミナルで開く」をクリックします。



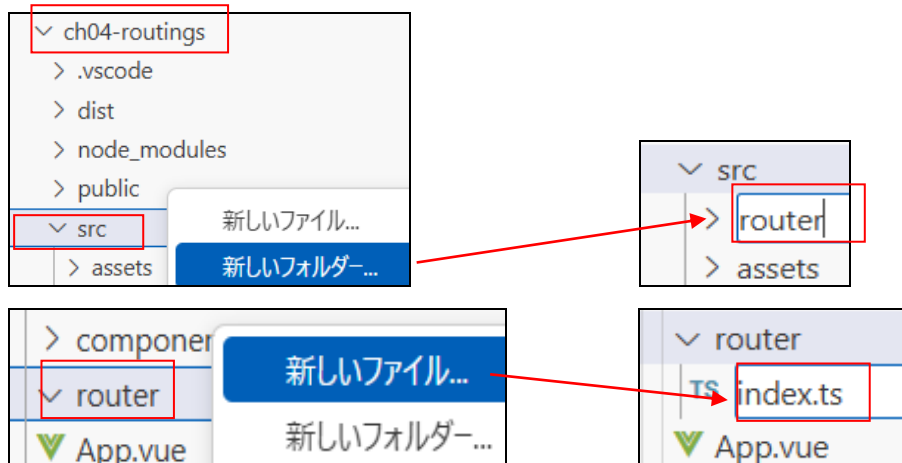
以下のコマンドを入力すると、インストールが行われます。

```
npm install vue-router
```



【作業手順②】 初期設定ファイルの生成

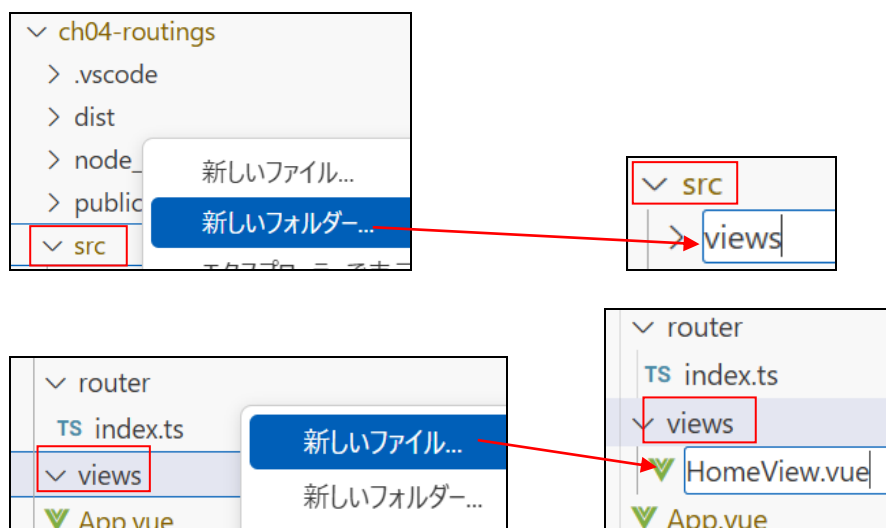
src フォルダの中に、**router** というフォルダを作成します。
その中に、ルート設定をまとめる **index.ts** ファイルを用意します。



【作業手順③】 views フォルダと HomeView の作成

Vue Router を使う場合は、画面ごとの Vue ファイルを src/views フォルダ内に作成することが推奨されています。

src/views フォルダを作成したあと、トップページを意味する、HomeView.vue を作成しておきましょう。



【作業手順④】 ルーター(index.ts)の初期ソースコード入力

作成した/src/router/index.ts に、以下のコードを追加します。

詳細は後ほど説明しますので、一旦初期設定を行っておきます。

ch04-routings¥src¥router¥index.ts

```

1:// src/router/index.ts
2:import { createRouter, createWebHistory } from 'vue-router'
3:import HomeView from '../views/HomeView.vue'
4:
5:// ルートの一覧を定義
6:const routes = [
7:  { path: '/', name: 'home', component: HomeView }
8:]
9:
10:// ルーターを作成
11:const router = createRouter({
12:  history: createWebHistory(import.meta.env.BASE_URL),
13:  routes
14:})
  
```

```

15:
16:// 他のファイルでも使えるようにエクスポート
17:export default router

```

【作業手順⑤】 TypeScript サーバーの再起動

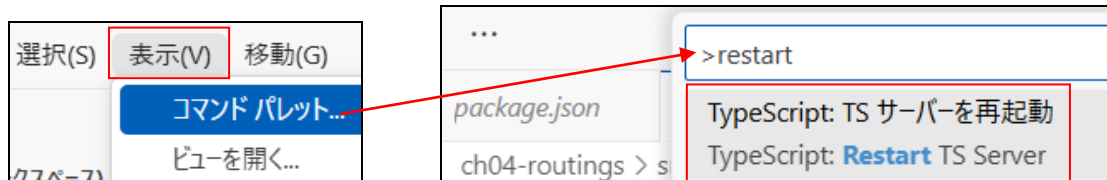
```

ch04-routings > src > router > TS index.ts > router > routes
1  // src/router/index.ts
2  import { createRouter, createWebHistory } from 'vue-router'

```

モジュールをインストールしても、以下のように、VSCode 上では認識されないことがありますので、以下の手順で再起動します。

コマンドパレットを開き、「restart」と入力し、TS Server を再起動してください。



【作業手順⑥】 main.ts にルーターを組み込む

作成したルーターを、アプリ全体で使えるように登録する必要があります。

以下のように追記してください。

ch04-routings¥src¥main.ts

```

1:import { createApp } from 'vue'
2:import App from './App.vue'
3:import { createBootstrap } from
    'bootstrap-vue-next/plugins/createBootstrap'
4:import 'bootstrap/dist/css/bootstrap.css'
5:import 'bootstrap-vue-next/dist/bootstrap-vue-next.css'
6:import router from './router' // 追加：ルーターのインポート
7:
8:createApp(App)
9:  .use(createBootstrap())
10:  .use(router) // 追加：ルーターをアプリに登録
11:  .mount('#app')

```

ここで行うと、基本的な設定は完了です。

【作業手順⑦】 初期画面の作成

ch04-routings¥src¥views¥HomeView.vue

```
1:<script setup lang="ts">
2:</script>
3:
4:<template>
5:  <main>
6:    <h2>ようこそ！</h2>
7:    <p>ここはトップページです。</p>
8:  </main>
9:</template>
```

3章で学習した通り、App.vue には複数のコンポーネントを配置することができ、ヘッダー・サイドバー・メインコンテンツ・フッターに分けて配置していました。

今回は、初期画面の中でも、「メインコンテンツ」の部分に初期表示されるための、HomeView.vue を簡単なコードで実装しておきました。

【作業手順⑦】 App.vue に RouterView を配置



4章では、3章のサンプルと同じようなサンプルで説明していきます。既知のソースコードの部分は割愛します。

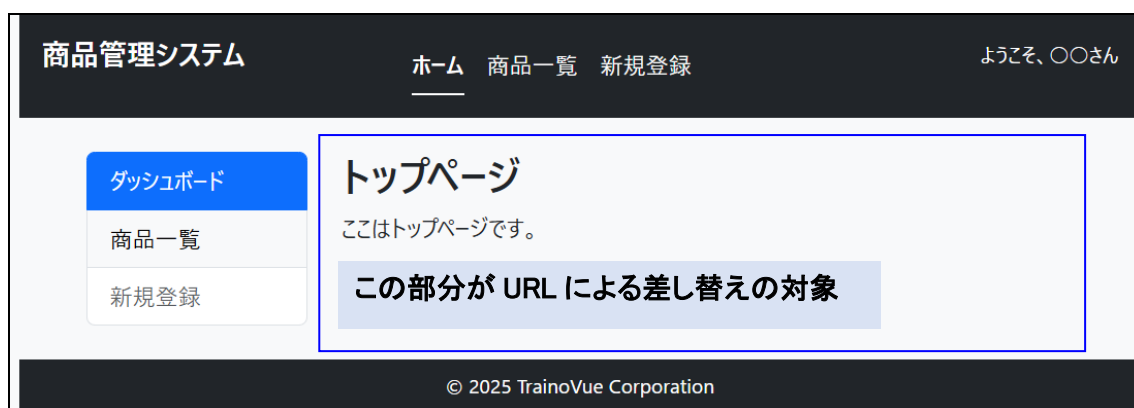
MainContent の部分が、RouterView になっていることに注目してください。

ch04-routings¥src¥App.vue

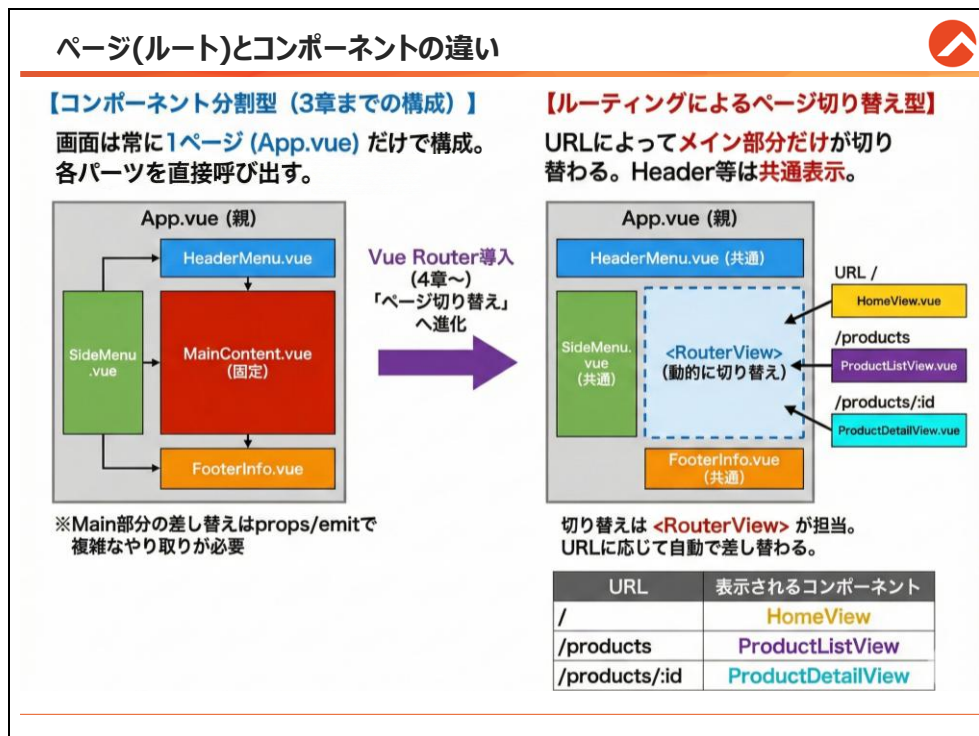
```
.....
20: <template>
21:   <div class="container-fluid bg-light">
22:
23:     <HeaderMenu :title="pageTitle" :user="userName" />
24:
25:     <div class="container my-4">
26:       <div class="row">
27:         <div class="col-md-3">
28:           <SideMenu />
29:         </div>
30:         <div class="col-md-9">
31:           <!-- ルーティングされたコンポーネントがここに表示される -->
32:           <RouterView />
33:         </div>
34:       </div>
35:     </div>
36:     <FooterInfo />
37:   </div>
38: </template>
```

これからルーティングによる、複数の画面を作っていきますが、SPA では基本的にヘッダー・サイドバー・フッターは変更されることがあまりないため、**主にメインコンテンツの部分が URL による画面切り替えの対象になる**と認識してください。

以下のように表示されると、動作確認完了です。



4.2. ページ(ルート)とコンポーネントの違い



コンポーネント分割型 (3 章までの構成)

第3章では、画面を構成する各パーツをコンポーネントとして分割しました。HeaderMenu.vue、SideMenu.vue、MainContent.vue、FooterInfo.vue などを組み合わせて、ひとつのページを作り上げる構造でした。

- ページ全体は App.vue が管理
- 子コンポーネントは <HeaderMenu> や <SideMenu> として直接呼び出す
- 画面の一部を差し替えたいときは、props や emit でデータのやり取り後に画面更新



このように、画面は常に1ページ(App.vue)だけで構成されていました。

ルーティングによるページ切り替え型

4章では、Vue Router を導入し、
「ページごとに表示するコンテンツ(Main 部分)を切り替える」仕組みに進化させます。

仕組みの違い:

- HeaderMenu・SideMenu・FooterInfo はすべてのページで共通表示
- **メイン部分(青枠)**は、URL(ルート)によって切り替わる
- 切り替えは <RouterView> が担当



URL が

- / のとき → HomeView
 - /products のとき → ProductListView
 - /products/:id のとき → ProductDetailView
- の vue ファイルに自動的に差し替わって表示されるようになります。

4.3. ルート設定によるページ切り替え

ルート設定によるページ切り替え

ルート設定ファイル (src/router/index.ts)

Vue Routerの設定はここに集約。「URLとページの対応表 (ルーティングテーブル)」を定義し、表示するコンポーネントを決定します。

【設定ファイル (index.ts) の重要部分】

```
// Vue Router の機能と各ビューコンポーネントをインポート
import { createRouter, createWebHistory } from 'vue-router'
import HomeView from '../views/HomeView.vue'
import ProductListView from '../views/ProductListView.vue'

// ルート (URLとコンポーネントの対応表) を定義 ★心臓部★
const routes = [
  { path: '/', name: 'home', component: HomeView },
  { path: '/products', name: 'products', component: ProductListView }
]

// (この後にルーター作成のコードが続きますが省略)
```

ルーティングテーブル (routes) の内容

path (URL)	name (ルート名)	component (表示するVueファイル)
'/'	'home'	HomeView
'/products'	'products'	ProductListView

【ページ切り替えの流れ (図解)】

- ① ユーザーがアクセス
http://localhost/products
- ② Vue Routerが 'routes' を確認
ルーティングテーブル
- ③ <RouterView> に表示
App.vue (親)
Header
Side-Menu
ProductListView (差し替え枠)

【動作確認 (イメージ)】

HomeView.vueの画面

ProductListView.vueの画面

ルート設定ファイル(index.ts)の構成

Vue Router では、ページの切り替え設定をすべて src/router/index.ts にまとめます。このファイルを理解すると、URL とページのつながりがイメージできるようになります。

以下のサンプルコードは、初期設定からコードを追記してトップページ (Home) と商品一覧ページ (Products) の 2 ページを切り替える設定にしたコードです。

ch04-routings¥src¥router¥index.ts

```
1:// Vue Router の機能をインポート
2:import { createRouter, createWebHistory } from 'vue-router'
3:
4:// 各ビューコンポーネントをインポート
5:import HomeView from '../views/HomeView.vue'
6:import ProductListView from '../views/ProductListView.vue'
7:
8:// ルート (URL とコンポーネントの対応表) を定義
9:const routes = [
```

```

10: { path: '/', name: 'home', component: HomeView },
11: { path: '/products', name: 'products', component: ProductListView }
12:]
13:
14:// ルーターを作成
15:const router = createRouter({
16:  // ブラウザの履歴管理を有効にする
17:  history: createWebHistory(),
18:  routes // 上で定義したルート情報を登録
19:})
20:
21:// 他のファイル (main.ts など) で使えるようにエクスポート
22:export default router

```

💡 構文の解説

```

1:// Vue Router の機能をインポート
2:import { createRouter, createWebHistory } from 'vue-router'

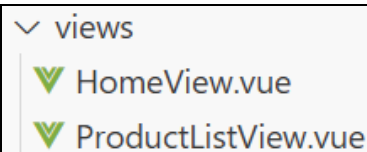
```

ルート設定のためには、vue-router から提供する 2 つのモジュールを import する必要があります。createWebHistory は、SPA でとても重要な設定で、#がない URL パスを扱うことができるようになります。

```

4:// 各ビューコンポーネントをインポート
5:import HomeView from '../views/HomeView.vue'
6:import ProductListView from '../views/ProductListView.vue'

```



views

- HomeView.vue
- ProductListView.vue

1 画面ごとに、views フォルダに vue ファイルを用意し、そのファイルを読み込むための import も必要です。今回は、画面切り替えを確認するため、ホーム画面以外に商品一覧画面も追加しました。

```
8:// ルート（URL とコンポーネントの対応表）を定義
9:const routes = [
10:  { path: '/', name: 'home', component: HomeView },
11:  { path: '/products', name: 'products', component: ProductListView }
12:]
```

ここがルーター設定の「心臓部」です。

この配列の中身は「**ルーティングテーブル**」と呼ばれるもので、それぞれの行は「1つのページ（URL）」を表しています。

どの URL が呼ばれたら、どの View ファイルに差し替えるかのルールを設定しています。

項目	説明
path	ブラウザでアクセスする URL のパス。例: /products
name	ルートの名前
component	表示したい Vue ファイル（ビューコンポーネント）

14行目以降の部分は、Vue Router の初期化や、別ファイルでの設定の読み込みを可能にする部分です。特別な理由がない限り、デフォルトのままでも大丈夫です。

このルート設定ができていれば、以下の流れで画面の差し替えが行われます。

1. ユーザが /products にアクセス
2. Vue Router が routes 配列を確認
3. /products に対応するコンポーネントを見つける
4. そのコンポーネントを <RouterView> の部分に表示

動作確認のための一覧ページのコード

ch04-routing\$src\$views\$ProductListView.vue

```
1:<script setup lang="ts">
2:</script>
3:
4:<template>
5:  <main>
6:    <h2>ようこそ！</h2>
7:    <p>ここは一覧ページです。</p>
8:  </main>
9:</template>
```

一覧ページは、画面差し替えの確認だけのための、簡単なコードにします。

動作確認

① ブラウザの URL が、「/」の時は、HomeView.vue の画面が表示



② ブラウザの URL が、「/products」の時は、ProductListView.vue の画面が表示



このように、ルーティングテーブルに定義した通りの画面差し替えが行われます。

《📄 構文チェック演習》

1. `ch04-routings-q¥src¥router¥index.ts` を開きます。
2. コメント【4.3 ①～②】の指示通りに、実装し、保存します。
3. `ch04-routings-q¥src¥views` の `vue` ファイルの中身を確認してください。
4. `ch04-routings-q¥src¥App.vue` を開きます。
5. コメント【4.3 ③】の指示通りに、実装し、保存します。
6. NPM スクリプトから `ch04-routings-q → dev` を実行し、Vite サーバーを起動します。
7. ブラウザで <http://localhost:517> にアクセスし、トップページが表示されることを確認してください。
8. ブラウザで <http://localhost:517/products> にアクセスし、一覧ページが表示されることを確認してください。

◆【補足】アプリのベース URL を変更する方法

通常、開発環境ではアプリの基本 URL (ベース URL) は `/` です。
 そのため、たとえば `/products` ルートを設定すると、
 ブラウザでは <http://localhost:517/products> にアクセスできます。

しかし、実際のシステムでは `/vueapp` などのサブディレクトリ配下にアプリを配置する場合があります。

その場合は、以下のように設定を変更します。

① `import` 先の URL を「`@/~~`」で設定します。

```
6: import ProductListView from '@/views/ProductListView.vue'
```

② `createWebHistory` の引数を以下に設定します。

```
17: history: createWebHistory(import.meta.env.BASE_URL),
```

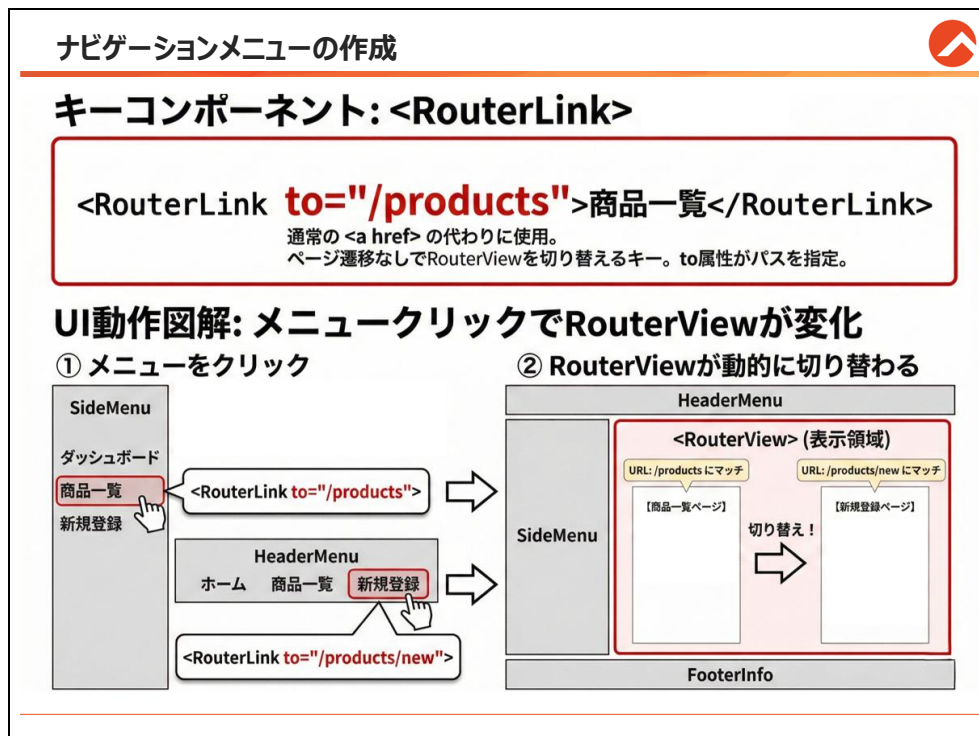
③ `vite.config.ts` の `defineConfig` を、以下のように修正します。

```
export default defineConfig({
  base: '/vueapp/', // ← 基本 URL を設定
  plugins: [vue()]
})
```

これらを設定すると、アプリの URL は次のようになります。

<http://localhost:517/vueapp/products>

4.4. ナビゲーションメニューの作成



ナビゲーションメニューの作成

この節では、Vue Router を使ってメニューからページを切り替える方法を学びます。
 前の節までで、URL に直接アクセスすればページが切り替わるようになりました。
 今度は、画面上の「メニュー（ナビゲーション）」をクリックしたときに、
 対応するページへ移動できるようにしていきます。



これまでの画面構成は、HeaderMenu や SideMenu は固定表示されていて、
 中央の <RouterView> 部分だけがページごとに切り替わる構成でした。

その構成なら、固定メニュー部分から「トップページ」や「商品一覧」をクリックして
 <RouterView> の中身（ページ）を切り替える仕組みを実装すれば、
 1 ページの中で複数の画面を切り替える SPA が完成できるわけです。

<RouterLink>

Vue では、通常の HTML リンク()の代わりに、専用の <RouterLink> コンポーネントを使います。

<RouterLink to= “/products” 商品一覧 </RouterLink>

このように書くだけで、ページを再読み込みせずに
ルーターが URL を切り替えて <RouterView> に対応するページを表示してくれます。

「to」属性に指定するパスは、router/index.ts の path と一致させます。

```
TS index.ts ×
ch04-routings > src > router > TS index.ts > ...
9   const routes = [
10    { path: '/', name: 'home', component: HomeView },
11    { path: '/products', name: 'products', component: ProductListView }
12  ]
```

サイドバーのメニュー作成の例

もう一つのリンク先として、「新規登録」の画面も増やします。

ch04-routings¥src¥router¥index.ts

```
import NewProductView from '../views/NewProductView.vue'

const routes = [
  { path: '/', name: 'home', component: HomeView },
  { path: '/products', name: 'products', component: ProductListView },
  { path: '/products/new', name: 'new-product', component: NewProductView }
]
```

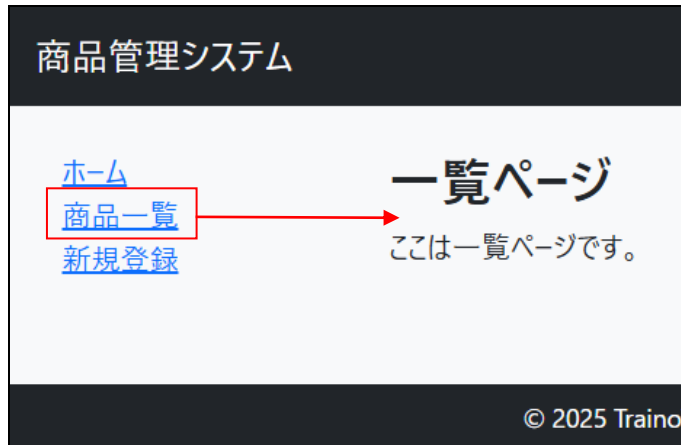
```

VUE3-C...  ch04-routings > src > views > NewProductView.vue > {} script setup
  components
    FooterInfo.vue
    HeaderMenu.vue
    MainContent.vue
    SideMenu.vue
  router
    TS index.ts
  views
    HomeView.vue
    NewProductView.vue
    ProductDetailView...
    ProductListView.vue

1  <script setup lang="ts">
2  </script>
3
4  <template>
5    <main>
6      <h2>登録ページ</h2>
7      <p>ここは登録ページです。</p>
8    </main>
9  </template>
10
```

そして、SideMenu.vue を以下のように修正すると、

```
<RouterLink to="/">ホーム</RouterLink><br>
<RouterLink to="/products">商品一覧</RouterLink><br>
<RouterLink to="/products/new">新規登録</RouterLink>
```

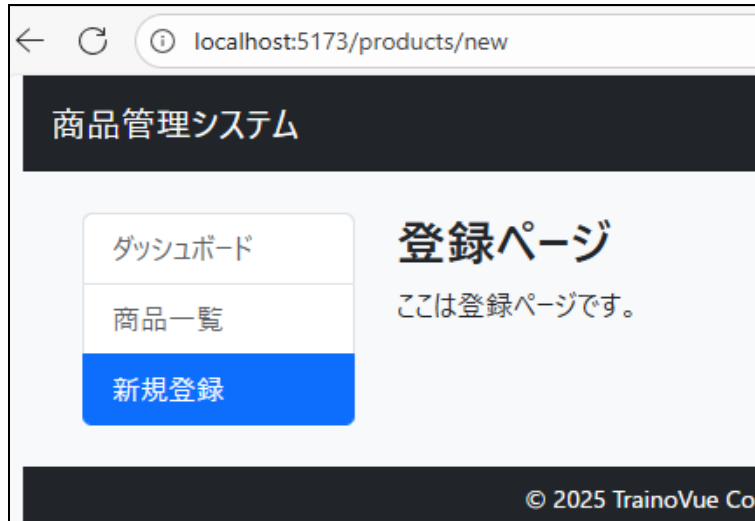


サイドバーに、views の各画面に切り替えできるリンクを挿入できます。

サイドバーのメニューを装飾した例

ch04-routings¥src¥components¥SideMenu.vue

```
1:<script setup lang="ts">
2:</script>
3:
4:<template>
5: <div class="list-group">
6:   <RouterLink to="/" class="list-group-item list-group-item-action"
     exact-active-class="active">ダッシュボード
     </RouterLink>
7:   <RouterLink to="/products"
     class="list-group-item list-group-item-action"
     exact-active-class="active">商品一覧
     </RouterLink>
8:   <RouterLink to="/products/new"
     class="list-group-item list-group-item-action"
     exact-active-class="active">新規登録
     </RouterLink>
9: </div>
10:</template>
```



Bootstrap を活用し、どこを選択しているのかが明確なサイドバーにすることもできます。

以下は、今回のサイドバーのサンプルで使われた Bootstrap の class です。

注目する部分は、**list-group** の class の要素で囲っておくところです。

対象	Bootstrap クラス / 設定	役割・効果	補足
コンテナ	list-group	縦並びのメニューコンテナを作成	サイドバー向きの基本レイアウト
各項目	list-group-item	リスト項目の標準スタイル	罫線・余白が付与され整う
各項目	list-group-item-action	ホバー時の反応(背景/カーソル)	クリック可能な見た目に
<RouterLink>	exact-active-class="active"	現在 URL と完全一致した項目に active を付与	Bootstrap が active を強調表示(背景色など)

あくまで参考ではありますが、真似するだけで簡単に汎用的なデザインのサイドバーを作ることができます。

ヘッダーのナビゲーションバー作成の例

縦方向が主流のサイドバー以外に画面の切り替えを行うメニューといえば、**ヘッダーのナビゲーションバー**を挙げられます。



ヘッダーに配置されるナビゲーションバーは、主に**横方向で配置**されますので、**<nav>タグ**を使います。

以下は、Bootstrap による装飾も含めたサンプルです。

ch04-routings¥src¥components¥HeaderMenu.vue

```
1: <!-- 中央 (6/12 幅) : ナビゲーションメニュー -->
2: <div class="col-6">
3:   <nav>
4:     <!-- 横方向のリストを構成し、下線を引く -->
5:     <ul class="nav nav-underline">
6:       <!-- ホームへのリンク -->
7:       <li class="nav-item">
8:         <RouterLink to="/" class="nav-link text-white
9:           exact-active-class="active">
10:           ホーム
11:         </RouterLink>
12:       </li>
13:       <!-- 商品一覧へのリンク -->
14:       <li class="nav-item">
15:         <RouterLink to="/products" class="nav-link text-white
16:           exact-active-class="active">
17:           商品一覧
18:         </RouterLink>
```

```

17:   </li>
18:   <!-- 新規登録へのリンク -->
19:   <li class="nav-item">
20:     <RouterLink to="/products/new" class="nav-link text-white"
21:                 exact-active-class="active">
22:       新規登録
23:     </RouterLink>
24:   </li>
25: </ul>
26: </nav>
27: </div>

```

3章で作成した、HeaderMenu.vue に、上記のコードを加えています。

<nav>で囲った中に、とを入れ、以下の Bootstrap を適用することで、ナビゲーションバーのデザインを適用できます。

対象	クラス / 設定	役割・効果	補足
コンテナ	nav	ナビゲーション領域	役割を明示
水平リスト	ul.nav	横並びのナビリスト	ヘッダーメニューの基本形
下線スタイル	nav-underline	アクティブなリンクに下線を付与	直感的に「今どこか」を示す
項目ラッパー	li.nav-item	各メニュー項目の枠	構造の明確化
リンク	nav-link	ナビ用リンクの標準スタイル	active が付与されると強調表示
<RouterLink>	exact-active-class="active"	現在 URL と完全一致で active 付与	下線付きスタイルが有効化

《📁 構文チェック演習》

1. ch04-routings-q¥src¥components¥SideMenu.vue を開きます。
2. コメント【4.4 ①】の指示通りに、実装し、保存します。
3. NPM スクリプトから ch04-routings-q → dev を実行し、Vite サーバーを起動します。
4. ブラウザで <http://localhost:5173/> にアクセスし、サイドバーのリンクをクリックするたびに、メイン画面が切り替わることを確認してください。
5. コメント【4.4 ②】の指示通りに、Bootstrap の class の意味を確認してください。

4.5. provide と inject によるデータ共有

provideとinjectによるデータ共有

■ ルーティング構造は広範囲のデータ共有が必要

- 親子構造ではなくなるので、propsではなく、**provide(提供)**と**inject(注入)**を利用する

```

graph TD
    App[App.vue (provide)] --> RouterView[RouterView]
    RouterView --> ProductListView[ProductListView (inject)]
    RouterView --> ProductDetailView[ProductDetailView (inject)]
  
```

■ provideの構文

キー名

配列データ

App.vue

```
provide('products', products)
```

■ injectの構文

- 受け取るデータの型の定義が前提

キー名

型指定

○○View.vue

```
const products = inject('products') as Ref<Product[]>
```

props はあくまで 1:1 通信

これまで(第3章)では、App.vue から MainContent.vue へデータを送るときに、props を使いました。

つまり、親から子へ値を渡し、子がそれを受け取るという“1 対 1”の通信でした。

HTTP で言えば、「リクエストとレスポンス」のようなやり取りです。

App.vue → **MainContent.vue(子)**

└─ v-bind で props を送る

ルーティング構造

しかし、今回のようにルーティングを使うと、構造が変わります。

App.vue には <RouterView> があるだけで、実際の画面 (ProductListView.vue や ProductDetailView.vue) はルーターによって動的に切り替わるようになっています。

App.vue (親)



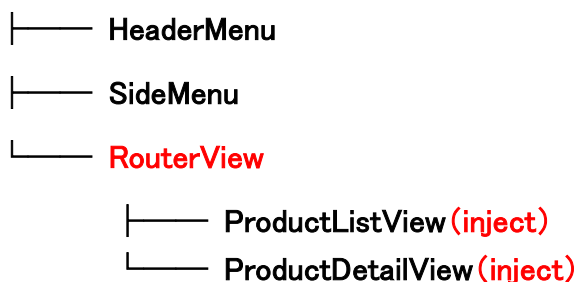
ここでは、すでに親子関係はなくなっており、`props` や `emit` を使ってデータを受け渡すことができません。

provide と inject

今回のように、どの画面(どの Vue ファイル)からでも共通のデータを参照したいという場面では、もう少し広い範囲でデータを共有する仕組みが必要になります。

そのときに使うのが、`provide`(提供)と `inject`(注入)です。

App.vue (provide)



App.vue がデータを「提供 (provide)」し、下の階層のコンポーネントはどこからでも「注入 (inject)」して使うことができます。この仕組みを使うと、RouterView をまたいでも同じデータを共有できます。

App.vue に provide を追加

ch04-routings¥src¥App.vue

```
5:import { ref, provide } from 'vue'
6:
7:// 商品一覧データ
8:const products = ref([
9:  { id: 1, name: 'ノートパソコン', price: 120000, category: '電子機器' },
10: { id: 2, name: 'スマートフォン', price: 90000, category: '電子機器' },
11: { id: 3, name: 'オフィスチェア', price: 25000, category: '家具' },
12: { id: 4, name: 'ボールペンセット', price: 800, category: '文房具' }
13:])
14:
15:// 子コンポーネント（画面全体）で参照できるように登録
16:provide('products', products)
17:</script>
```

3 章でも使われていた、固定値の products 配列データですが、16 行のように、**provide(‘キー名’ , データ名)** の構文で登録できます。

provide で登録されたデータは、アプリケーション全体で参照できるようになります。

もちろん、5 行のように、provide を import する必要があります。

ここで、**ref でリアクティブ変数にしたものを登録している部分**を意識してください。別画面で受け取る時も、**ref 型で受け取る理由**に繋がります。

provideしたデータを inject で受け取る

ch04-routings¥src¥views¥ProductListView.vue

```
2:import { inject } from 'vue'
3:import type { Ref } from 'vue'
4:
5:// 商品配列の型定義
6:type Product = {
7:  id: number;
8:  name: string;
9:  price: number;
10:  category: string
11:}
12:// App.vue で provide された「products」を受け取る
13:const products = inject('products') as Ref<Product[]>
.....
33:table class="table table-striped table-hover align-middle">
34:    <thead class="table-dark">
35:        <tr>
36:            <th>商品番号</th>
37:            <th>商品名</th>
38:            <th>価格</th>
39:            <th>カテゴリ</th>
40:        </tr>
41:    </thead>
42:    <tbody>
43:        <tr v-for="p in products" :key="p.id">
44:            <td>{{ p.id }}</td>
45:            <td>{{ p.name }}</td>
46:            <td>{{ p.price }} 円</td>
47:            <td>{{ p.category }}</td>
48:        </tr>
49:    </tbody>
```

💡 構文の解説

```
2:import { inject } from 'vue'
3:import type { Ref } from 'vue'
```

inject を利用できるように、2 行では **import** を行っています。

inject は、App.vue で **provide** されたデータを受け取るための関数です。

つまり、「上の階層から共有されているデータを取りに行く」ときに使います。

3 行の **Ref 型** の **import** は、App.vue で **provide** に登録されたデータが **ref** のリアクティブ変数だったので、受け取る時も型を合わせる必要があるためです。

```
5:// 商品配列の型定義
6:type Product = {
7:   id: number;
8:   name: string;
9:   price: number;
10:  category: string
11:}
12:// App.vue で provide された「products」を受け取る
13:const products = inject('products') as Ref<Product[]>
```

一番大事なのは、13 行です。App.vue の中で「**provide('products', products)**」と登録されていたデータを取りに行っています。

‘products’ という文字列は データの登録名(キー) です。

これが一致していれば、App.vue で共有したデータを受け取ることができます。

さらに、**as Ref<Product[]>** という部分は、

「この変数 **products** は、**Product** 型の配列を **ref()** で包んだものです」

ということを TypeScript に伝えています。そのデータがリアクティブな配列であることを型で保証できるわけです。

3 章の MainContent でも行ったように、6-11 行の **Product** 型の定義も事前に必要です。

```

43:     <tr v-for="p in products" :key="p.id">
44:         <td>{{ p.id }}</td>
45:         <td>{{ p.name }}</td>
46:         <td>{{ p.price }} 円</td>
47:         <td>{{ p.category }}</td>
48:     </tr>

```

13行で products という変数名で配列データを受け取ったので、43行ではその配列データを使って、中身を順番に表示しています。

ダッシュボード

商品一覧

新規登録

商品一覧

商品番号	商品名	価格	カテゴリ
1	ノートパソコン	120000 円	電子機器
2	スマートフォン	90000 円	電子機器
3	オフィスチェア	25000 円	家具
4	ボールペンセット	800 円	文房具

App.vue に定義したデータを、ProductListView.vue でも受け取れるので、サイドバーからのクリックでも商品一覧を表示できるようになりました。

《📄 構文チェック演習》

1. ch04-routings-q¥src¥App.vue を開きます。
2. コメント【4.5 ①】の指示通りに、実装し、保存します。
3. ch04-routings-q¥src¥views¥ProductListView.vue を開きます。
4. コメント【4.5 ②～⑤】の指示通りに、実装し、保存します。
5. NPM スクリプトから ch04-routings-q → dev を実行し、Vite サーバーを起動します。
6. ブラウザで <http://localhost:517> にアクセスし、サイドバーのリンクをクリックするたびに、一覧画面のデータが正しく表示されるかを確認してください。

4.6. パラメータ付きルート

パラメータ付きルート

- 詳細画面へ遷移する際は、パラメータをURLに追記する必要がある。
 - バスに、「:id」のように追記すると、動的URLを設定できる
 - 例) /products/1、/products/4

```
router/index.ts{ path: '/products/:id', name: 'product-detail', component: ProductDetailView}
```

- 遷移先では、useRoute関数でURL情報を取得
 - route.paramsでパラメータを取得可能

```
ProductDetailView.vueconst route = useRoute()const id = Number(route.params.id)
```

商品番号	商品名
1	ノートパソコン

商品詳細
商品ID 1の詳細ページです。

パラメータ付きルートとは

Vue Router では、URL の一部に値 (パラメータ) を埋め込んで画面を切り替えることができます。

例えば、一覧画面の「商品名」をクリックしたときに、該当商品の詳細ページへ移動することがよくありますが、その時利用されます。

/products/1 → id=1 の詳細ページ

/products/2 → id=2 の詳細ページ

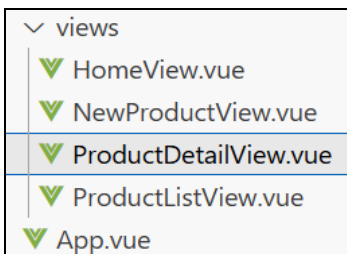
ルート設定にパラメータを追加する

パラメータ付きルートの設定は、通常のルーティングと同様に、router/index.ts 内で行います。

ch04-routings¥src¥router¥index.ts

```
1:// 各ビューコンポーネントをインポート
2:import HomeView from '../views/HomeView.vue'
3:import ProductListView from '../views/ProductListView.vue'
4:import NewProductView from '../views/NewProductView.vue'
5:import ProductDetailView from '../views/ProductDetailView.vue'
6:
7:// ルート (URL とコンポーネントの対応表) を定義
8:const routes = [
9:  { path: '/', name: 'home', component: HomeView },
10: { path: '/products', name: 'products', component: ProductListView },
11: { path: '/products/new', name: 'new-product', component: NewProductView },
12: { path: '/products/:id', name: 'product-detail', component: ProductDetailView }
```

新たな画面として、詳細画面、ProductDetailView.vue を追加していますので、5 行で import します。



そして、12 行の '/products/:id' を追記しました。

:id の部分は、変数のようなものです。

実際のアクセス時には、次のように数字などの値が入ります。

- /products/1
- /products/2
- /products/10

Vue Router は、この :id の部分を「ルートパラメータ」として扱います。まとめると、

「/products/〇〇 という URL に来たら」

「その〇〇を id として扱い」

「ProductDetailView を表示する」

というルールを、Vue Router に設定しています。

パラメータ付きルートを作成するリンクを作成

通常、パラメータ付きルートは、一覧画面のリンクやボタンに生成されることが多いです。

今回は、商品一覧画面に動的のパラメータを持つ RouterLink を配置します。

商品一覧		
商品番号	商品名	価格
1	ノートパソコン	120000 円
2	スマートフォン	90000 円
3	オフィスチェア	25000 円
4	ボールペンセット	800 円

クリックすると
詳細画面に切り替わる

ch04-routing\$src\$views\$ProductListView.vue

```

.....
43:<tr v-for="p in products" :key="p.id">
44:  <td>{{ p.id }}</td>
45:  <td>
46:    <RouterLink :to="'/products/${p.id}'">
47:      {{ p.name }}
48:    </RouterLink>
49:  </td>
50:  <td>{{ p.price }} 円</td>
51:  <td>{{ p.category }}</td>
52: </tr>

```

商品名が入る場所に、<RouterLink>要素で囲って画面の切り替えができるリンクを配置しました。

46 行に、リンク先を文字列と変数を組み合わせていますが、バッククォート(`)と `$p.id` を使って、文字列と変数を組み立てています。

例えば、p.id が 3 の場合、``/products/${p.id}`` → `"/products/3"` の URL が生成されます。

商品ごとにリンク先が変わるようになっているので、クリックすると、先ほど設定したルート { `path: '/products/:id'`, `component: ProductDetailView` } に一致するため、ProductDetailView.vue が表示されます。

🔗【補足】名前付きルート

名前付きルートを理解しておく、パスを文字列結合で書くより安全に画面遷移できます。

```
{ path: '/products/:id', name: 'product-detail', component: ProductDetailView }
```

index.ts に上記のようにルーティングがされている場合、以下のように遷移先を指定できます。

```
<RouterLink :to="{ name: 'product-detail', params: { id: p.id } }">
```

```
  {{ p.name }}
```

```
</RouterLink>
```

名前付きルートの書き方を採用すると、以下のメリットがありますので、Vue 公式でも推奨されています。

- URL 構造を後で変更しても影響が最小限
- 大規模 SPA で安全に使えるルーティングが書ける

パラメータを受け取る方法(useRoute 関数)

id を受け取って、詳細画面を表示するため、ProductDetailView.vue を作成し、以下のコードを記載しました。

ch04-routings¥src¥views¥ProductDetailView.vue

```
2:import { useRoute } from 'vue-router'
3:
4:// 現在の URL から、:id の部分を取得
5:const route = useRoute()
6:const id = Number(route.params.id)
7:
8:// 本来は、API へ id を渡してデータを取得する処理が続きます
9:</script>
11:<template>
12:  <h2 class="mb-3">商品詳細</h2>
13:  <p>商品 ID {{ id }} の詳細ページです。</p>
14:
15:  <!--本来は、API から取得した詳細データを表示するコードが入ります -->
16:</template>
```

5 行の `useRoute()` を使うと、

「今どの URL を開いているか」が分かるオブジェクトを取得できます。

`route` の中には、次のような情報が入っています。

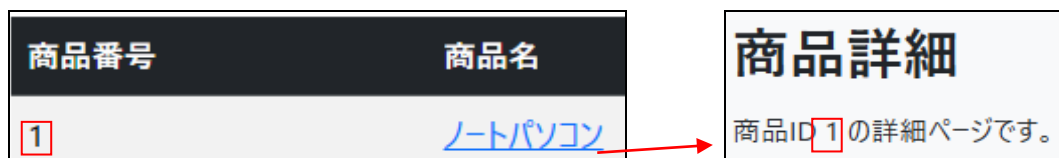
- `route.path` ... 実際のパス(例: `"/products/3"`)
- `route.params` ... `:id` などのパラメータ(今回の主役)

ですので、6 行では、`route.params.id` で、渡されたパラメータを取得しています。

id は通常、数値データになることが多いため、`Number` 型に変換しています。

id を取得して行う次の処理についてですが、7 行のコメントにもあるように、**本来はこの id(商品番号)を利用して、API を経由し、データベースから詳細データを取得するコードが入ります。**

しかし、この章ではまだ API を扱わないため、単純に 13 行で id を表示するようにしています。今回はあくまで、API 経由直前までのイメージを持っていただくだけで大丈夫です。



《📄 構文チェック演習》

1. `ch04-routings-q¥src¥router¥index.ts` を開きます。
2. コメント【4.6 ①】の指示通りに、コードを確認してください。
3. `ch04-routings-q¥src¥views¥ProductListView.vue` を開きます。
4. コメント【4.6 ②】の指示通りに、実装し、保存します。
5. `ch04-routings-q¥src¥views¥ProductDetailView.vue` を開きます。
6. コメント【4.6 ③-⑤】の指示通りに、実装し、保存します。
7. NPM スクリプトから `ch04-routings-q → dev` を実行し、Vite サーバーを起動します。
8. ブラウザで <http://localhost:5173/products> にアクセスし、一覧画面を表示します。
その後、任意の商品名をクリックすると、同じ商品 ID が詳細画面で表示されるかを確認してください。

章のまとめ

章のまとめ



- Vue Routerの導入
- ページ（ルート）とコンポーネントの違い
- ルート設定によるページ切り替え
- ナビゲーションメニューの作成
- provideとinjectによるデータ共有
- パラメータ付きルート

この章では、Vue Router を利用し、ページの切り替えをする方法を学習しました。
SPA を作るために必ず必要な知識ですので、使いこなせるようになりましょう。

4 章までの学習で、Vue.js の構文は一通り学んだことになり、Vue 単体は作成可能となりました。しかし、実際のアプリケーションでは、データベースと連携が必要なため、外部の API(バックエンド側)との連携が不可欠です。

オプションの内容ではありますが、5 章でその連携の方法も紹介していますので、目を通すことを推奨します。

第5章 【オプション】REST API 連携による CRUD 実装

章の目標

- Node.js と Express を用いて、Vue.js と連携する基本的な REST API サーバーを構築できる
- axios を用いた非同期通信により、データの検索・登録・更新・削除（CRUD）機能を実装できる
- ルーティングと API 連携を組み合わせ、データ操作に応じて画面が遷移する SPA を実装できる

章の概要

章の概要



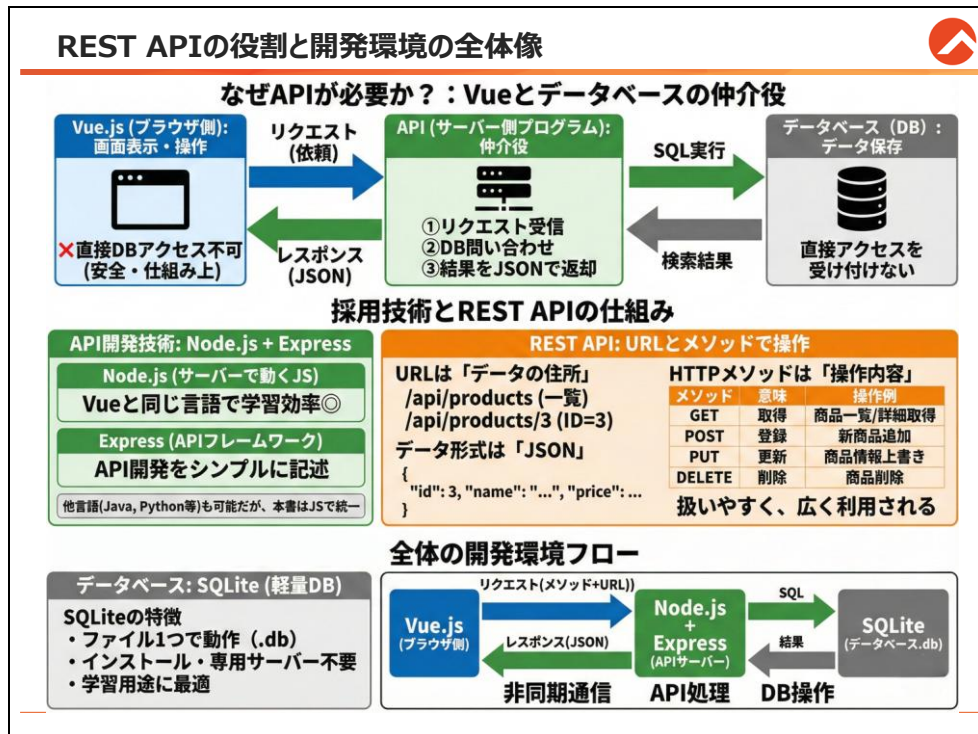
- REST APIの役割と開発環境の全体像
- REST API環境構築
- Expressで検索機能のAPI実装
 - 一覧画面の実装
 - キーワード検索機能の実装
 - 詳細検索機能の実装
- 登録処理の実装
- 更新処理の実装
- 削除処理の実装

第 5 章では、Vue.js とバックエンド (Node.js/Express) を連携させ、本格的な Web アプリ開発へステップアップします。

まず API サーバーを構築し、データの「参照 (一覧・検索・詳細)」から「操作 (登録・更新・削除)」まで、業務システムに不可欠な CRUD 機能 (Create/Read/Update/Delete) を網羅的に実装します。

画面操作がデータベースへ反映される一連の仕組みを理解し、実践的な開発スキルを習得します。

5.1. REST API の役割と開発環境の全体像



Vue とデータベースを繋ぐには

4 章までは、Vue を使って画面の表示や操作を行う方法を学びました。

しかし、実際の業務システムでは、画面に表示するデータはすべてデータベースに保存されています。

ここで多くの方が疑問に思うのが、

「Vue から直接データベースを操作できないのか？」

という点です。

結論として、ブラウザ(Vue)は 安全面や仕組み上の理由で、**データベースに直接アクセスできません。**

そのため、Vue とデータベースの間に「API(サーバー側のプログラム)」を置く必要があります。

API が必要な理由

API は、フロントエンド(ブラウザ画面)とデータベースの間に次の役割を担います。

1. ブラウザからのリクエストを受け取る
2. データベースに問い合わせを行う
3. 結果を JSON 形式で Vue に返す

Vue だけでは「画面操作」はできても、「**データの読み書き**」はできません。

そのため、API が必要になります。

API を作る方法

API はサーバー側で動くプログラムのため、

Java、Python、Node.js などの サーバーサイド言語で作られます。

どの言語を使っても「APIをつくる」という目的は同じです。

本書では、次の理由から **Node.js + Express** を採用します。

- 画面(Vue)と同じ JavaScript で書ける
- セットアップが簡単で、コード量も少なく済む

業務では Java(Spring)、Python(Django/Flask)、Go、PHP などよく使われますが、最初に学ぶ構成としては Node.js がもっとも扱いやすい選択です。

REST API とは

API の中でも、現代では **REST API(レスト API)** という形式が一般的に使われています。

REST API は、URL と HTTP メソッドを組み合わせてデータを操作するための設計方法です。

1. URL が「データの住所」を表す

例えば、次のような URL でデータの位置を表します。

- /api/products → 商品一覧
- /api/products/3 → ID=3 の商品データ

2. HTTP メソッドで「操作内容」を表す

HTML の基礎では「GET」と「POST」の 2 種類を学びましたが、REST API では、PUT と DELETE も登場します。

従来の API なら、1 つの機能ごとに異なる URL を設定するべきです。URL が重複するとエラーになります。

しかし、REST API は同じ URL でも **HTTP メソッドが異なると、重複と判定しない**ので、**/api/products/** という URL だけで、**検索・登録・更新・削除の API の作成ができる**わけです。

メソッド	意味	操作の例	URL とメソッドの組み合わせ
GET	取得	商品一覧を取得する	GET /api/products
GET	取得	ID=3 の商品詳細を取得する	GET /api/products/3
POST	登録	新しい商品を追加する	POST /api/products
PUT	更新	ID=3 の商品を上書きする	PUT /api/products/3
DELETE	削除	ID=3 の商品を削除する	DELETE /api/products/3

POST なら、登録処理、PUT なら、更新処理、DELETE なら、削除処理として使われます。

よって、「URL がシンプルになる」、「API の役割を直感的に理解できる」という利点があります。

3. データの受け渡しは JSON 形式

```
{ "id": 3, "name": "ノートパソコン", "price": 120000 }
```

上記は、JSON 形式の例です。JavaScript と相性が良いため、どの言語・どのフレームワーク (Java / Python / Node.js など) でも扱えます。この特性から、REST API ではデータの受け渡しに JSON が広く利用されています。

本研修で採用している REST API とデータベース製品の種類

■API の土台 : Node.js

JavaScript をブラウザだけでなく「サーバー側でも動かせる」仕組みです。

Vue と同じ言語で API を書けるため、学習効率が高くなります。

■REST API のフレームワーク : Express

Node.js 上で動く Web アプリケーション用のフレームワークです。

Java の Spring Boot や Python の Django、PHP の Laravel を代替できます。

- どの URL にアクセスされたら
- どの処理を行って
- 何を返すのか

をシンプルに記述できるため、REST API 開発に適しています。

Vue(非同期通信) → API(Node.js + Express) → データベース → 結果を返す

という流れを実現できます。

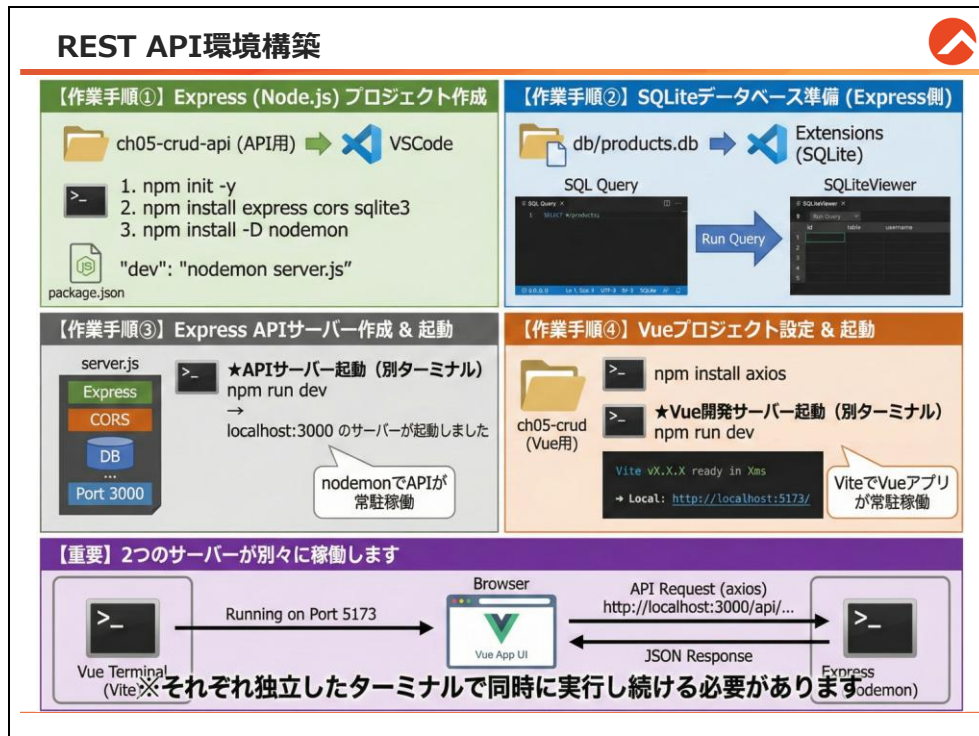
■データベース製品 : SQLite

今回利用する **SQLite** は、
扱いやすさに特化した「**軽量データベース**」です。

- 専用サーバーが不要(.db ファイル 1 つで動く)
- インストールが不要
- ログイン不要/パスワード不要/データベース選択不要
- USE コマンド不要で、1 つの db ファイルが 1 つのデータベースとなる
- 小規模なシステムや学習用途に最適

一般的なサーバー型データベース(Oracle / MySQL / PostgreSQL)より
準備が簡単で、学習の妨げになる要素が少ないため採用しています。

5.2. REST API 環境構築

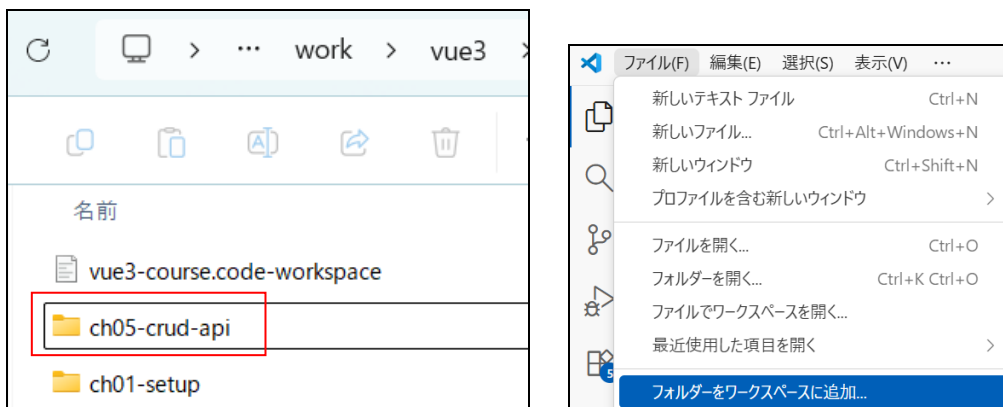


※研修では、既に環境構築は完了していますので、ゼロから構築する際に参考

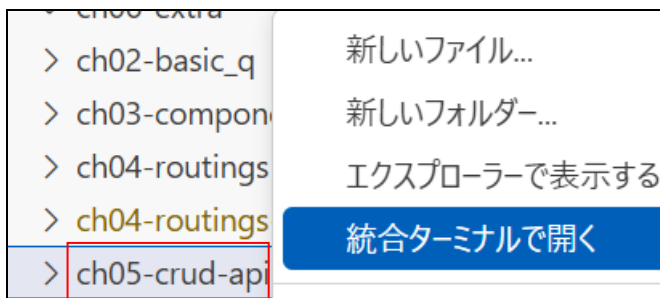
【作業手順①】Node.js プロジェクトの作成

REST API を構築するため、新しい Node.js プロジェクトを作成する必要があります。Vue のフォルダとは構成がかなり異なるので、同じフォルダは避けたほうがよいです。

1. エクスプローラーで C:\work\vue3 の中に、api 用の新しいフォルダを追加します。
その後、VSCode 上で「ファイル」→「フォルダをワークスペースに追加」します。



2. 追加されたフォルダを右クリックし、総合ターミナルを開きます。



3. 以下のコマンドで初期設定を行います。

```
npm init -y
```

4. 続いてコマンドで Express と cors と sqlite のモジュールを導入します。

```
npm install express cors sqlite3
```

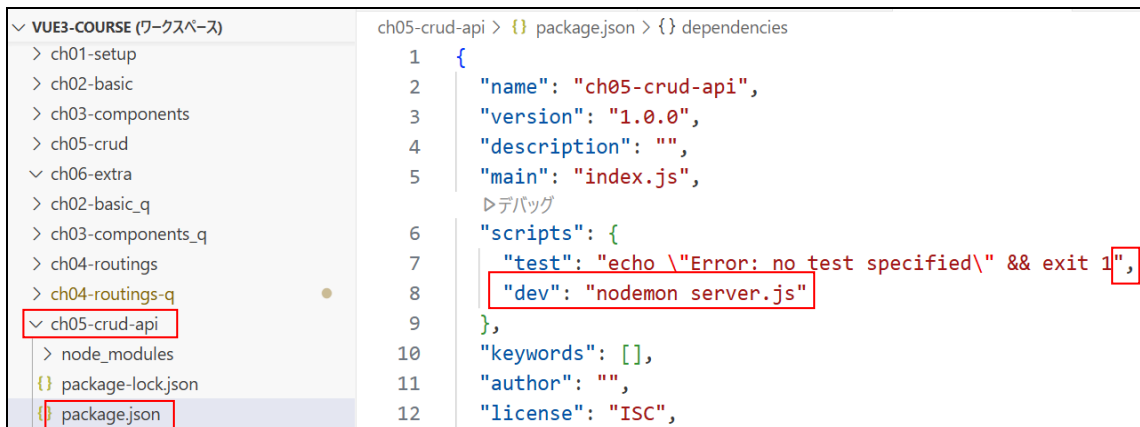
5. Node.js の便利な開発環境、**nodemon** を導入します。Vue3 の Vite のようなものです。

```
npm install -D nodemon
```

6. package.json の scripts に開発用コマンドを追加します。

※ 既存のコードの後尾にカンマ(,)も入れます。

```
"dev": "nodemon server.js"
```



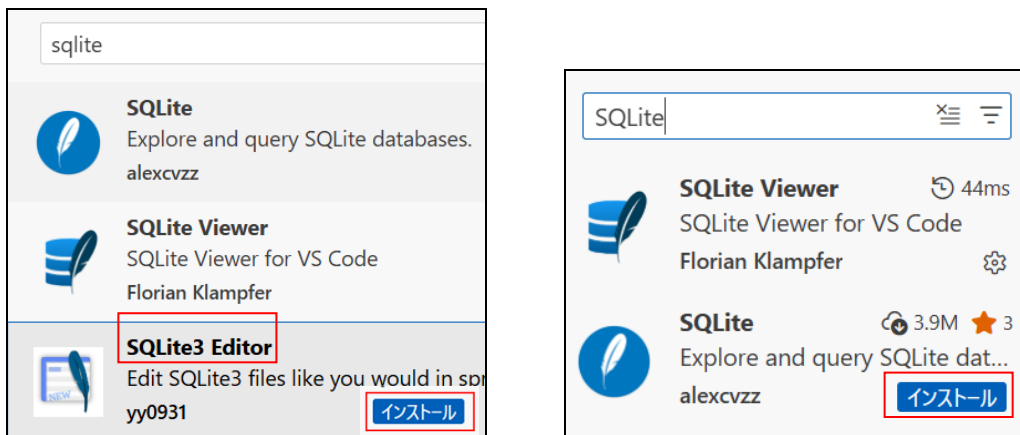
【作業手順②】SQLite によるサンプルテーブル作成

1. ch05-crud-api に「db」フォルダを作成し、中に「products.db」ファイルを作成します。



2. VSCode の拡張機能から、「SQLite」と「SQLiteEditor」をインストールします。

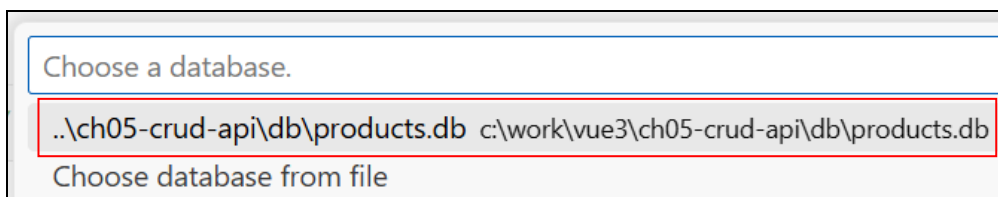
※「SQLite Viewer」も DB を GUI で確認できるので、どちらかをインストールしてください。



3. コマンドパレットを開き、「sqlite」と入力します。Open Database をクリックします。



4. 手順1で作成した products.db を選択すると、使用するデータベースを選択できます。



5. 再度コマンドパレットを開き、「SQLite: New Query」をクリックします。

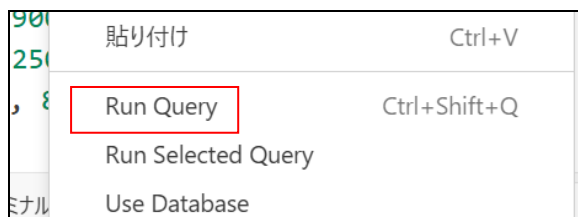
以下の CREATE TABLE 文と INSERT 文を入力します。

```
-- SQLite は軽量な組み込み型データベース管理システムです
-- ログインもパスワードも不要で、db ファイルを作成するだけで使用できます
-- SQLite では、整数型:INTEGER、文字列型:TEXT が使われます
-- 自動採番には AUTOINCREMENT を使用します

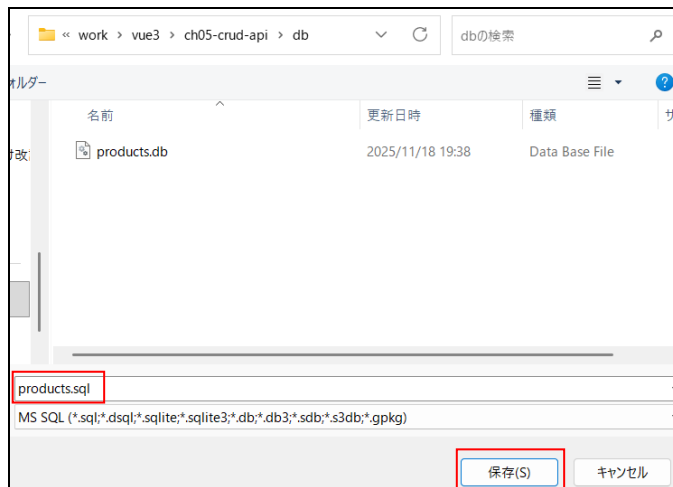
CREATE TABLE products (
  id      INTEGER PRIMARY KEY AUTOINCREMENT, --
  name    TEXT    NOT NULL,
  price   INTEGER NOT NULL,
  category TEXT    NOT NULL
);

INSERT INTO products (name, price, category) VALUES
('ノートパソコン', 120000, '電子機器'),
('スマートフォン', 90000, '電子機器'),
('オフィスチェア', 25000, '家具'),
('ボールペンセット', 800, '文房具');
```

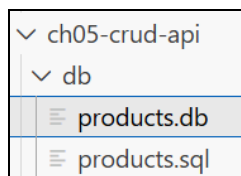
6. 任意の場所を右クリックします。「Run Query」をクリックすると、記載した全 SQL 文を実行できます。(選択式で実行したい時は、「Run Selected Query」をクリックします。)



7. 作成した SQL ファイルは、db フォルダ内に db ファイル名と同じ名前で保存します。



8. products テーブルが作成されると、products.db ファイルを開くことで、
テーブルの構成を GUI で確認することができます。これは SQLite Editor の機能の一つで
す。



ch05-crud-api > db > ≡ products.db

SELECT * FROM products

スキーマ

SQLエディター

自動リロード

検索

その他

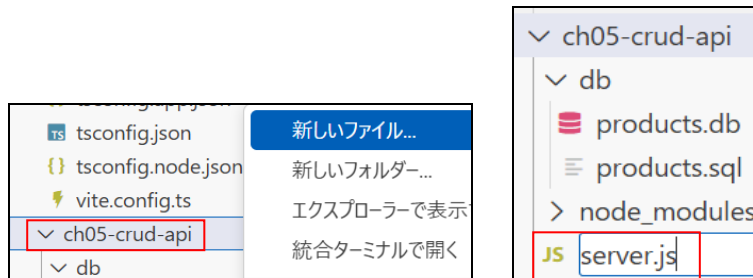
	id	INTEGER PRIMARY KEY AUTOINCREMENT	name	TEXT NOT NULL	price	INTEGER NOT NULL	category	TEXT NOT NULL	+
1	1		ノートパソコン		120000		電子機器		
2	2		スマートフォン		90000		電子機器		
3	3		オフィスチェア修正		9999		その他		
4	4		ボールペンセット		800		文房具		
+									

列をクリックすると、修正も簡単にできますし、「+」ボタンをクリックすると、行の挿入もできます。

【作業手順③】Express+SQLite を利用した REST API サーバー作成

Vue がデータベースへアクセスできるようにするため、
Node.js+Express で API の土台となるプログラムを作成します。

1. ch05-crud-api フォルダに、server.js ファイルを作成します。APIの入り口になるファイルです。



2. server.js に以下のようにコードを記載し、保存します。

ch05-crud-api\server.js

```

1:// 必要なモジュールの読み込み
2:const express = require('express')
3:const cors = require('cors')
4:const sqlite3 = require('sqlite3').verbose()
5:
6:// Express アプリの作成
7:const app = express()
8:
9:// JSON データを扱うためのミドルウェア
10:app.use(express.json())
11:
12:// CORS (他のサーバーとの通信を許可する仕組み)
13:app.use(cors())
14:
15:// SQLite データベースへの接続
16:const db = new sqlite3.Database('./db/products.db')
  
```

7 行では、API の役割をするモジュールの、Express を読み込み、10 行では、JSON 形式でデータを受け取るためにしました。

13 行は、異なるサーバー(Vue の Vite サーバーは 5173 番ポート、API サーバーは 3000 番ポート)同士の通信を許可するための重要なコードです。

16 行は、データベース(/db/products.db)と接続させるためのコードです。

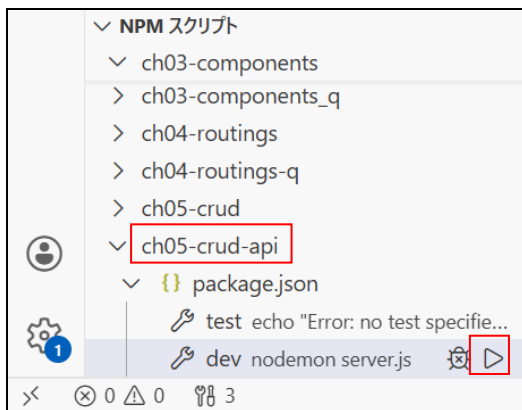
3. 動作確認のために、テスト用コードを追記します。

ch05-crud-api\server.js

```
.....
18:// 動作確認用の API (GET /api/test)
19:app.get('/api/test', (req, res) => {
20:  res.json({ status: 'ok' });
21:})
22:
23:// ポート番号 3000 でサーバーを起動
24:app.listen(3000, () => {
25:  console.log('localhost:3000 のサーバーが起動しました')
26:})
```

4. API サーバーの起動確認を行います。

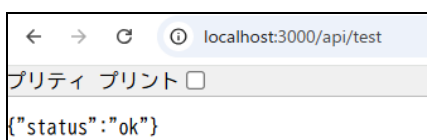
NPM スクリプト欄の「ch05-crud-api」の dev の ▶ ボタンをクリックし、API サーバーを起動します。



コンソールに以下のように出力されるか確認します。

```
[nodemon] starting `node server.js`
localhost:3000 でサーバーが起動しました
```

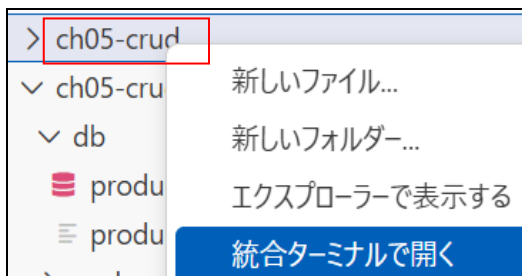
5. ブラウザに、「localhost:3000/api/test」と入力し、以下の画面が表示されたら、URL による API 接続も正常に動作していることが確認できます。



【作業手順④】 Vue プロジェクトに axios の導入

Vue 側から API を呼び出すために、「fetch」関数を使う方法もありますが、REST API 向けだと、外部モジュールである「**axios**」がより使いやすいので導入します。

1. api 側ではなく、Vue プロジェクトである「ch05-crud」プロジェクトを右クリックし、ターミナルを開きます。



2. 以下のコマンドを入力し、axios を導入します。

```
npm install axios
```

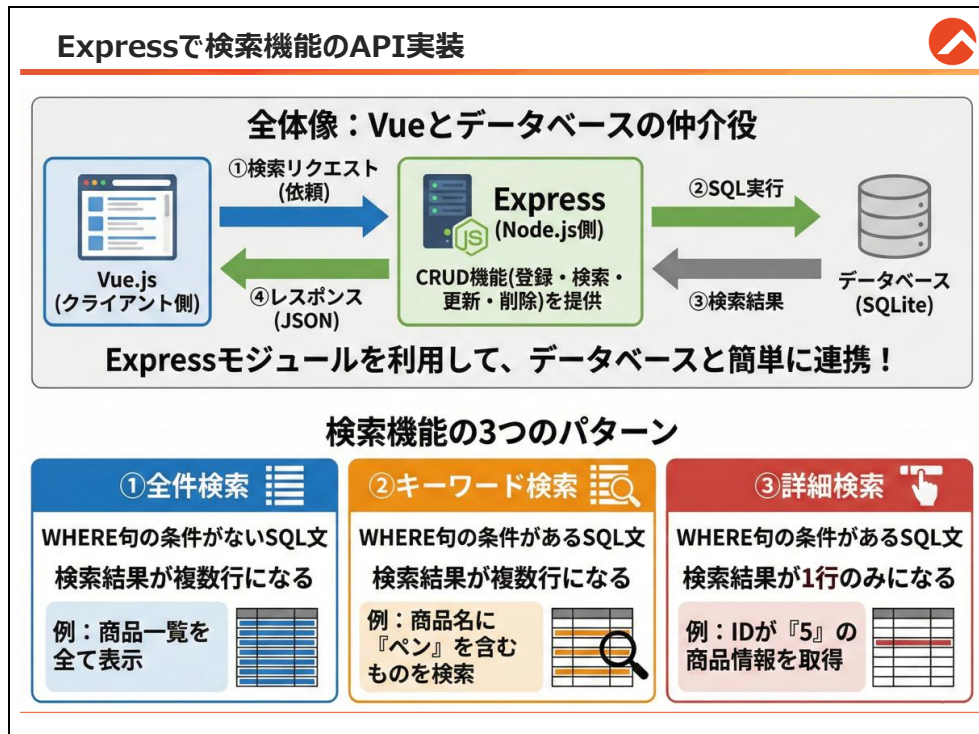
```
PS C:\work\vue3\ch05-crud> npm install axios

added 23 packages, and audited 123 packages in 2s

16 packages are looking for funding
  run `npm fund` for details

found 0 vulnerabilities
npm notice
npm notice New minor version of npm available! 11.5.1 -> 11.6.2
npm notice Changelog: https://github.com/npm/cli/releases/tag/v11.6.2
npm notice To update run: npm install -g npm@11.6.2
npm notice
PS C:\work\vue3\ch05-crud>
```

5.3. Express で検索機能の API 実装



Express で検索機能の REST API を作成

Node.js 側 (ch05-crud-api プロジェクト内) の `server.js` にソースコードの追記することで、Vue からの依頼により、データベースとの CRUD(登録・検索・更新・削除)を行う機能を実装できます。

その際、データベースと簡単に連携ができる、Express というモジュールを利用します。

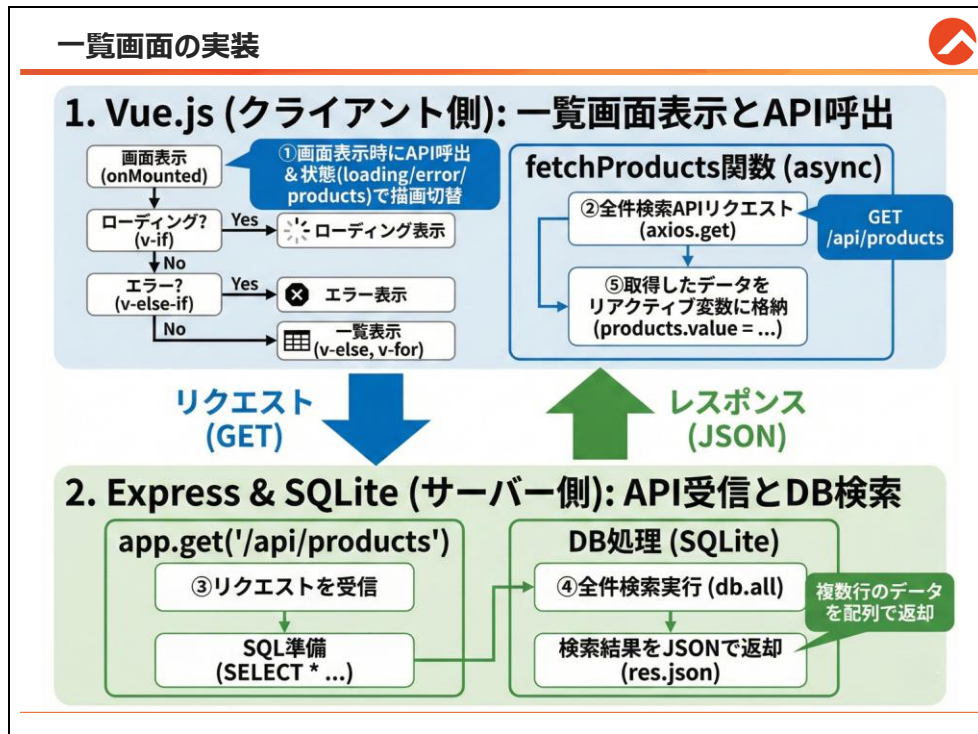
まずは、一番頻度が高い、検索機能の REST API の実装方法について説明します。

データベースに検索の依頼をする際は、主に以下の 3 パターンのどれかです。

- ① WHERE 句の条件が**ない** SQL 文で、検索結果が**複数行**になる (例：全件検索)
- ② WHERE 句の条件が**ある** SQL 文で、検索結果が**複数行**になる (例：キーワード検索)
- ③ WHERE 句の条件が**ある** SQL 文で、検索結果が**1 行のみ**になる (例：詳細検索)

それぞれ、REST API の実装内容と、Vue 側のコードが変わりますので、一つずつ説明します。

5.3.1. 一覧画面の実装(REST API+Vue)



全件検索をする REST API の実装

まずは、全件検索をする API の書き方を説明します。

なお、API の実装は全て server.js 内に追記をしていく形式となります。

ch05-crud-api/server.js

```

6:// Express アプリの作成
7:const app = express()
.....
15:// SQLite データベースへを読み込む
16:const db = new sqlite3.Database('./db/products.db')
.....
28:// 【5.3.1 全件検索】商品一覧取得 API (GET /api/products)
29:app.get('/api/products', (req, res) => {
30:  const sql = 'SELECT * FROM products'
31:
32:  // db.all は、複数行の結果を配列で取得するメソッド
33:  db.all(sql, [], (err, rows) => {
34:    if (err) {
35:      console.error(err)

```

```

36:    // サーバー側でエラーが発生した場合は、500 番エラーを返す
37:    res.status(500).json({ error: 'Database error' })
38:    return
39:  }
40:  // 正常に取得できた場合は、JSON 配列として返す
41:  res.json(rows)
42: })
43: })

```

一見難しそうに見えますが、29 行～43 行までは、ほぼ定型文ですので、コピー&ペーストでのご利用を推奨します。

```

7:const app = express()
.....
29:app.get('/api/products', (req, res) => {

```

7 行で express の機能を利用するため「app」で初期化をしています。
29 行の意味は、express の機能を使い、HTTP の **GET メソッド**で「/api/products」にアクセスされたとき、この中の関数を実行するという意味です。

```

30: const sql = 'SELECT * FROM products'

```

関数内では、**実行したい SQL 文を宣言**しておきます。今回は全件検索をするための、WHERE 句がない SQL 文を設定しています。

```

15:// SQLite データベースへの接続
16:const db = new sqlite3.Database('./db/products.db')
.....
33: db.all(sql, [], (err, rows) => {

```

16 行で SQLite のデータベースファイル(products.db)を開いているため、
33 行でそのデータベースに対して SQL 文を実行させ、結果を受け取っています。

db.all は、「複数行の結果を配列で受け取れる」sqlite3 の専用メソッドです。
sqlite3 以外のデータベース製品(Oracle/MySQL/PostgreSQL など)と連携する際は、それぞれ異なるメソッドを呼び出すので、注意してください。本テキストでは、sqlite3 専用メソッドのみで説明します。

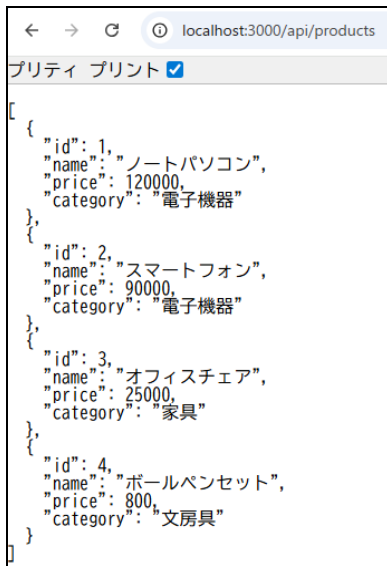
ここで、第 3 引数の部分ですが、(err, rows) の部分は **callback 関数**と呼ばれ、処理後の戻り値を受け取っている部分です。1 行に、実行前の引数(sql, [])と、実行後の戻り値(err, rows)が並んでいると理解してください。

rows には、検索結果が[{ id:1, name:..., ... }, { id:2, ... }, ...] と配列の形で格納されます。**err** には、エラーが発生した際のエラー情報が格納されます。34-39 行の if 文は、エラー処理のためのコードです。

41: `res.json(rows)`

エラー処理の if 文に引っかからなかった場合は、41 行で正常の動作を行います。

rows に入っている検索結果を呼び出し元(Vue 側)に JSON 形式で返す重要なコードです。



```
[
  {
    "id": 1,
    "name": "ノートパソコン",
    "price": 120000,
    "category": "電子機器"
  },
  {
    "id": 2,
    "name": "スマートフォン",
    "price": 90000,
    "category": "電子機器"
  },
  {
    "id": 3,
    "name": "オフィスチェア",
    "price": 25000,
    "category": "家具"
  },
  {
    "id": 4,
    "name": "ボールペンセット",
    "price": 800,
    "category": "文房具"
  }
]
```

ブラウザで「localhost:3000/api/products」にアクセスすると、テーブルの中身が JSON 形式で現れますので、API が正しく動いているか確認できます。

ここまでの API 側の実装で、この API を Vue 側で呼び出す方法を説明します。

◆ 【補足】 3 章・4 章で学習したデータの受け渡し構文について

4 章までは、画面で表示するデータは App.vue 内に固定値として持ち、子コンポーネントでは props や provide/inject を使って受け取っていました。これらは Vue のコンポーネント同士の連携方法として非常に重要で、今後も使う技術です。

しかし、5 章からは 画面ごとに API からデータを取得するため、props/provide/inject の利用は減ります。データベースと連携しない親・子コンポーネント同士では使うことがありますので、(商品一覧画面内に、複数の商品カードを配置する時など)その点を踏まえて、次に進みましょう。

一覧画面から API を呼び出す方法

ここからは、ch05-crud プロジェクトに戻って、4 章まで作成をしていた一覧画面 (views/ProductListView.vue) にソースコードを追記していきます。

説明は<script>部分から始めます。

ch05-crud¥src¥views¥ProductListView.vue

```
1:<script setup lang="ts">
2:import { ref, onMounted } from 'vue'
3:import axios from 'axios'
4:import { RouterLink } from 'vue-router'
5:
6:// 商品配列の型定義
7:type Product = {
8:  id: number;
9:  name: string;
10: price: number;
11: category: string
12:}
13:
14:// 商品一覧データ、ローディング状態、エラー状態を保持する変数
15:const products = ref<Product[]>([])
16:const loading = ref(true)
17:const error = ref<string | null>(null)
18:
19:
20:// 商品一覧を API から取得する関数
21:const fetchProducts = async () => {
22:  loading.value = true
23:  error.value = null
24:
25:  try {
26:    // データ取得（ここが API との通信部分）
27:    const res =
28:      await axios.get<Product[]>('http://localhost:3000/api/products')
29:    products.value = res.data
30:    console.log('商品データを取得しました:', products.value)
```

```
31: // ローディング検証用コード（ローディング表示を維持したい場合）
32: const sleep = (ms: number) =>
      new Promise(resolve => setTimeout(resolve, ms))
33: await sleep(3000) // 3 秒待機
34:
35: } catch (err) {
36:   // エラーが起きた場合
37:   console.error(err)
38:   error.value = '商品情報の取得に失敗しました。'
39:
40: } finally {
41:   // 成功・失敗にかかわらず、ロード中フラグを終了
42:   loading.value = false
43: }
44:}
45:// この画面が表示された瞬間に API を呼び出す
46:onMounted(() => {
47:  fetchProducts()
48:})
49:</script>
```

💡 構文の解説

onMounted と axios の import

```
1:<script setup lang="ts">
2:import { ref, onMounted } from 'vue'
3:import axios from 'axios'
```

一覧ページは「画面が表示された瞬間」に API を呼び出す必要があります。

Vue には「ライフサイクル」という概念があり、その中に **Mounted** というタイミングがあります。

Mounted とは、「Vue ファイルの<template>に書かれた HTML が画面に描画された直前」の**タイミング**を意味します。

つまり、**テーブルの枠が画面に表示されたタイミングで、テーブルの中身を入れるために、2 行目の onMounted モジュールが必要なわけ**です。

3 行の **axios** は、REST API を簡単に呼び出せるモジュールなので import しておきます。

onMounted による自動実行

```
45:// この画面が表示された瞬間に API を呼び出す
46:onMounted() => {
47:  fetchProducts()
48:})
```

この画面が表示される瞬間、onMounted()に登録した処理が自動で実行されます。そのため、API を呼び出す fetchProducts()が自然に動き始めます。

ref によるリアクティブ変数

```
14:// 商品一覧データ、ローディング状態、エラー状態を保持する変数
15:const products = ref<Product[]>([])
16:const loading = ref(true)
17:const error = ref<string | null>(null)
```

15 行目: products:

API から取得する「商品一覧データ(JSON 配列)」を入れておく箱です。

16 行目: loading:

API 通信中にローディング画面(ぐるぐるアニメーション)を表示するための変数です。

17 行目: error

API 処理で問題があったとき、エラーメッセージを入れるための変数です。

fetchProducts 関数のしくみ

```

20: // 商品一覧を API から取得する関数
21: const fetchProducts = async () => {
22:   loading.value = true
23:   error.value = null
24:
25:   try {
26:     // データ取得（ここが API との通信部分）
27:     const res = await
        axios.get<Product[]>('http://localhost:3000/api/products')
        .....

```

ch05-crud-api¥server.js

```

40:   // 正常に取得できた場合は、JSON 配列として返す
41:   res.json(rows)

```

21 行から本格的な API を呼び出すための関数のコードが始まります。

async()関数は、この関数の中で `await` が使えます」という宣言です。

27 行の `await` は「通信が終わるまで待ってから次へ進む」という意味です。

そして `axios` が使われますが、HTTP の GET メソッドで、指定した URL の **REST API を呼び出す**、一番重要な部分です。この関数が実行された時、**REST API との通信が開始**されます。通信が成功的に終了した場合は、先ほど API 内の 41 行で記載したように、`res` のデータが返ってくるので、**全件検索のデータが 27 行の `res` に代入される**わけです。

通信後の分岐の説明

```

28:   products.value = res.data
29:   console.log('商品データを取得しました:', products.value)
35: } catch (err) {
36:   // エラーが起きた場合
37:   console.error(err)
38:   error.value = '商品情報の取得に失敗しました。'
40: } finally {
41:   // 成功・失敗にかかわらず、ロード中フラグを終了
42:   loading.value = false
43: }

```

API から取得できた `res` オブジェクトは、「**`res.data`**」で **JSON データを取得**できますので、`products.value` で**データを更新**しています。後ほど説明しますが、Vue の HTML 側と `products` は繋がっているため、Vue が自動で画面を再描画し、一覧が表示されます。

エラーが起きた際は、catchの中に入りますので、38行目でerrorのリアクティブ変数を更新します。

40行目の**finally**は、**正常でもエラーでも、最後に必ず実行されるブロック**です。

処理後は loading の値を false に変えています。

最初は true になっていたため、ローディング中と描画されますが、この処理が実行されると、**ローディングが終了したと判定させることができます。**

ローディング検証コード(本番では不要)

```
31: // ローディング検証用コード (ローディング表示を維持したい場合)
32: const sleep = (ms: number) =>
      new Promise(resolve => setTimeout(resolve, ms))
33: await sleep(3000) // 3 秒待機
```

31 行目～33 行目は、本番稼働の時は削除すべきコードです。

通常だと、ローディング画面の表示が一瞬で終わるため、あえて3秒の間を入れてローディング画面の検証をするためのデモ用コードです。

API により更新されたデータの HTML 描画方法

続いて、HTML 側を解説します。

API を呼び出す場合、v-if～v-else-if～else を利用し、条件による表示の切り替えを行います。

ch05-crud¥src¥views¥ProductListView.vue

```

55: <h2 class="mb-3">商品一覧</h2>
56:   <!-- ローディング中 -->
57:   <div v-if="loading" class="d-flex justify-content-center my-4">
58:     <div class="spinner-border text-primary" role="status"></div>
59:   </div>
60:   <!-- エラー発生時 -->
61:   <div v-else-if="error" class="text-danger">
62:     {{ error }}
63:   </div>
64:
65:
66:   <!-- 正常に取得できた場合の一覧表示 -->
67:   <table v-else class="table table-striped table-hover align-middle">
68:     <thead class="table-dark">
69:       <tr>
70:         <th>商品番号</th>
71:         <th>商品名</th>
72:         <th>価格</th>
73:         <th>カテゴリ</th>
74:       </tr>
75:     </thead>
76:     <tbody>
77:       <tr v-for="p in products" :key="p.id">
78:         <td>{{ p.id }}</td>
79:         <td>
80:           <RouterLink :to="`/products/${p.id}`">
81:             {{ p.name }}
82:           </RouterLink>
83:         </td>
84:         <td>{{ p.price }} 円</td>
85:         <td>{{ p.category }}</td>
86:       </tr>
87:     </tbody>
88:   </table>

```

💡 構文の解説

元々は、テーブルを描画する HTML のみでしたが、API と連携する場合は

「loading」「error」の値の変化によって、異なる HTML を描画するようにしています。

- ① loading が true ならローディング画面
- ② loading=false かつ error があればエラー画面
- ③ どちらでもなければ一覧テーブル

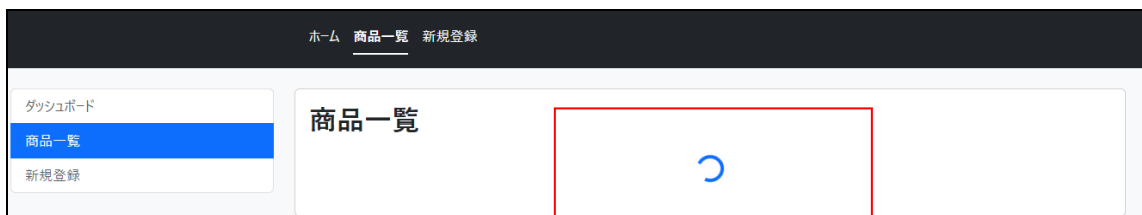
```

56: <!-- ローディング中 -->
57: <div v-if="loading" class="d-flex justify-content-center my-4">
58:   <div class="spinner-border text-primary" role="status"></div>
59: </div>

```

まず、57 行目で、リアクティブ変数 loading の値の判定を行います。API 通信が終了する前は、値が true ですので、このローディング中の画面が表示されます。

通常、ローディング中は、アニメーションを表示することが多いので、今回は **Bootstrap の spinner クラス**を使い、青い円がぐるぐる回転するアニメーションを描画します。



API 通信が終了されると、値が false に変わるので、そのタイミングで v-if の判定も false となり、アニメーションも終了され、次の v-else-if 判定にいきます。

続いて、エラー時の HTML の説明です。

```
61: <!-- エラー発生時 -->
62: <div v-else-if="error" class="text-danger">
63:   {{ error }}
64: </div>
```

リアクティブ変数 `error` の値は通常 `false` ですので、この HTML は無視されますが、API 通信が失敗したら `error` が「商品情報の取得に失敗しました。」の値に更新されるので、文字列が描画されます。



次は、ローディングも終わり、エラーも起きなかった正常動作の場合の HTML です。

```
66: <!-- 正常に取得できた場合の一覧表示 -->
67: <table v-else class="table table-striped table-hover align-middle">
.....
76:   <tbody>
77:     <tr v-for="p in products" :key="p.id">
78:       <td>{{ p.id }}</td>
79:       <td>
80:         <RouterLink :to="`/products/${p.id}`">
81:           {{ p.name }}
82:         </RouterLink>
83:       </td>
84:       <td>{{ p.price }} 円</td>
85:       <td>{{ p.category }}</td>
86:     </tr>
```

67 行目で `v-else` を付与しているので、ローディングも終了し、エラーも起きなかった際、67 行目以降の HTML が描画されます。

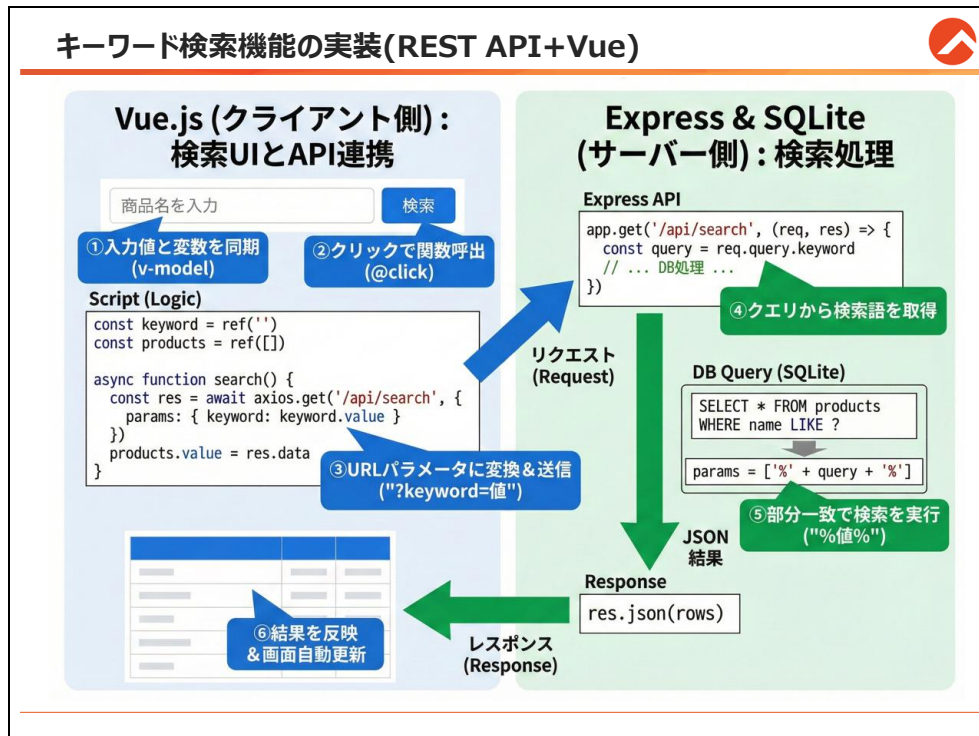
77 行目で `v-for` の対象にしているのは、API からの一覧結果を格納したリアクティブ変数「`products`」です。



これで、一覧画面の描画が完了しました。

複数のファイルを記載しながら 1 つの画面が完成されますので、
ソースコードを最初から最後まで熟読して次のページの学習に入りましょう。

5.3.2. キーワード検索機能の実装(REST API+Vue)



検索機能について

商品が増えてくると、一覧画面だけでは目的の商品を探すのが難しくなります。そこで多くの業務システムでは、

- 「商品名で検索する」
- 「カテゴリで絞り込む」
- 「金額で範囲検索」

といった 検索機能(絞り込み機能) が用意されています。

この節では、その中でも最も基本となる「商品名で検索」機能を追加し、Vue から REST API に WHERE 句付きの検索を依頼する流れを説明します。

検索フォームは一覧画面内に配置

検索画面だけ別ページにする方法もありますが、以下の理由から一覧画面にフォームを置く方法が、実務では一般的です。

- 一覧と検索を行ったり来たりする必要がない
- 検索結果も表形式で表示したい

- レイアウトが一貫し、ユーザの迷いが減る
- 一覧画面のソースコードを流用できる

そのため、この節では一覧画面に検索フォームを追加する方法を採用します。

ただ、全件検索のサンプルにそのまま追記すると、既存のサンプルが混雑になりますので、今回は前の節までのコードをそのままコピーした「ProductListView2.vue」という vue ファイルに追記します。

実際は、ファイルを分けるのではなく、一覧画面に追記する形となります。



ch05-crud¥src¥router¥index.ts

```
import ProductListView2 from '../views/ProductListView2.vue'
.....
{ path: '/products2', name: 'products2', component: ProductListView2 },
```

ch05-crud¥src¥components¥SideMenu.vue

```
<RouterLink to="/products2" class="list-group-item list-group-item-action"
exact-active-class="active">商品一覧(検索機能付き)</RouterLink>
```

index.ts と、SideMenu.vue に上記の追記をすることで、「商品一覧(検索機能付き)」ボタンをクリックすると、ProductListView2.vue が表示されるようにしました。

ch05-crud¥src¥views¥ProductListView2.vue

```
81:<h2 class="mb-3">商品一覧</h2>
82:<!-- 【5.3.2】 検索フォーム -->
83:<div class="mb-2 input-group w-50 gap-1">
84:  <input type="text" class="form-control" placeholder="商品名で検索">
85:  <button v-model="keyword" class="btn btn-primary">
86:    検索
```

```
87: </button>  
88:</div>
```

ProductListView2.vue には一覧の上に、ボタンと横並びの検索フォームを追記しました。

REST API 側での実装(Express による WHERE 句つき検索)

キーワード検索機能を実装するために、次は API サーバー側で検索リクエストを受け付け、WHERE 句を使ってデータを抽出する機能を追加します。その後、Vue 側で検索フォームと連動させ、API を呼び出す機能を実装します。

全件検索(5.3.1 節)では、すべての商品データを取得しました。キーワード検索では、Vue 側から送られてきた商品名の一部(キーワード)を受け取り、それを SQL の WHERE 句に適用してデータベースを検索します。

この検索リクエストは、GET メソッドとクエリパラメータを用いて行われます。

ペン	検索
商品番号	商品名
1	ノートパソコン
2	スマートフォン
3	オフィスチェア
4	ボールペンセット

通常の HTTP リクエストの場合、上のような検索フォームで検索をすると、以下のような URL が生成されます。

例: GET /api/products2?keyword=ペン

ここで、?keyword=値 の部分を、「クエリパラメータ」と呼びますので、覚えておきましょう。

ch05-crud-api\$server.js

```

.....
45:// 【5.3.2 キーワード検索】商品名で部分一致検索する API (
46:app.get( '/api/products2' , (req, res) => {
47:  // 1. URL パラメータではなく、クエリパラメータからキーワードを取得
48:  const keyword = req.query.keyword
49:
50:  // 2. WHERE 句で部分一致検索 (name にキーワードを含む商品を検索)
51:  // LIKE ? を使用し、SQL インジェクションを防ぐためパラメータを分離します
52:  const sql = `SELECT * FROM products WHERE name LIKE ?`
53:  const params = [`%${keyword}%`] // 前後にワイルドカード(%)を付けることで
    部分一致検索となる
54:

```

```

55: // 3. SQLite に対して SQL を実行
56: db.all(sql, params, (err, rows) => { // db.all は複数行検索に使用される
57:   if (err) {
58:     console.error(err)
59:     res.status(500).json({ error: 'Database error' })
60:     return
61:   }
62:
63:   // 4. 結果を JSON 形式で返却
64:   res.json(rows) // 検索結果が配列として返る
65: })
66: })
67:
68: // ... ポート番号 3000 でサーバーを起動部分は省略 ...

```

💡 構文の解説

エンドポイントの定義とクエリパラメータの取得 (46-48 行目)

```

46: app.get( '/api/products2' , (req, res) => {
48:   const keyword = req.query.keyword

```

クライアント (Vue 側) からは、以下のような URL でアクセスが届く予定です。

GET http://localhost:3000/api/products2?keyword=ペン

ここで、?keyword=ペンの部分 (クエリパラメータ) を取得できるのが、48 行です。

Express では、URL の ? 以降にある「キー=値」のペアは **req.query** オブジェクトに自動的に格納されます。

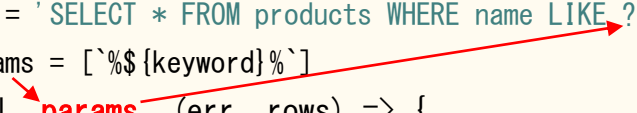
よって、**req.query.keyword** で、?keyword=ペン の「ペン」を取得できます。

SQL の構築とデータベース実行 (52-65 行目)

```

52: const sql = 'SELECT * FROM products WHERE name LIKE ?'
53: const params = [`%${keyword}%`]
55: db.all(sql, params, (err, rows) => {
..... (以降は全件検索のコードと同じ)

```



52 行では、SQL の LIKE 演算子を使用しています。この SQL 文でキーワード検索ができます。

53 行では、`%${keyword}%` とすることで、「キーワードの前後に何がついていてもヒットする」という部分一致の状態を作り出しています。

例: キーワードが「ペン」の場合、SQL 上では '%ペン%' として扱われ、「ボールペン」「ペンシル」などがヒットします。

params は、55 行の配列を代入する場所になるので、**[] で囲って配列にする**必要があります。

55 行以降は、全件検索の API とほぼ同じく、**db.all を実行**します。複数行を検索する可能性があるからです。ただ、55 行の「params」の部分だけが異なります。

全件検索の時は空欄だった、**db.all の第 2 引数は、実行する SQL 文に、「？」があった場合に、順番に params を代入する役割**があります。

今回は、「？」に「%keyword%」が代入されますので、キーワード検索を実行するわけです。

Vue 側で REST API を呼び出し、表示する

ch05-crud¥src¥views¥ProductListView2.vue

```

.....
19:// 【5.3.2】 検索キーワードを保持する変数
20:const keyword = ref<string>('')
21:
22:// 【5.3.2】 商品検索を行う関数
23:const searchProducts = async () => {
24:  loading.value = true
25:  error.value = null
26:
27:  try {
28:    const res = await axios.get(
29:      'http://localhost:3000/api/products2', {
30:        params: {
31:          keyword: keyword.value
32:        }
33:      })
34:    products.value = res.data
35:    console.log('商品データを取得しました:', products.value)
36:
37:  } catch (err) {
38:    error.value = '該当する商品がありません。'
39:    console.error(err)
40:
41:  } finally {
42:    loading.value = false
43:  }
44:}
.....
81:<h2 class="mb-3">商品一覧</h2>
82:<!-- 【5.3.2】 検索フォーム -->
83:<div class="mb-2 input-group w-50 gap-1">
84:  <input v-model="keyword" type="text" class="form-control" placeholder="
商品名で検索">
85:  <button class="btn btn-primary" @click="searchProducts">

```

```

86:   検索
87: </button>
88:</div>
.....

```

💡 構文の解説

キーワード検索から、関数を呼び出す

```

20:const keyword = ref<string>('')
.....
23:const searchProducts = async () => {
.....
84: <input v-model="keyword" type="text" class="form-control" placeholder="
商品名で検索">
85: <button class="btn btn-primary" @click="searchProducts">
86:   検索
87: </button>
.....

```

全件検索で REST API を呼び出していた部分と同じ部分が多いので、差分のみ説明します。

20 行では、ユーザが検索ボックスに入力した文字列を保持します。

84 行の `v-model` で入力フォームと紐付けることで、入力値が自動的にこの変数に入ります。

85 行のボタンには、23 行の関数を呼び出すクリックイベントを設定しています。

クエリパラメータを利用し、API へキーワードを渡した後、結果を取得

```

27: try {
28:   const res = await axios.get(
29:     'http://localhost:3000/api/products2', {
30:     params: {
31:       keyword: keyword.value
32:     }
33:   })
34:   products.value = res.data

```

29 行-32 行は、先ほど実装した API 側の `req.query.keyword` にデータを送るための書き方です。axios は、この `params` オプションの中身を自動的に URL の後ろに付与してくれます。

- 変換イメージ:

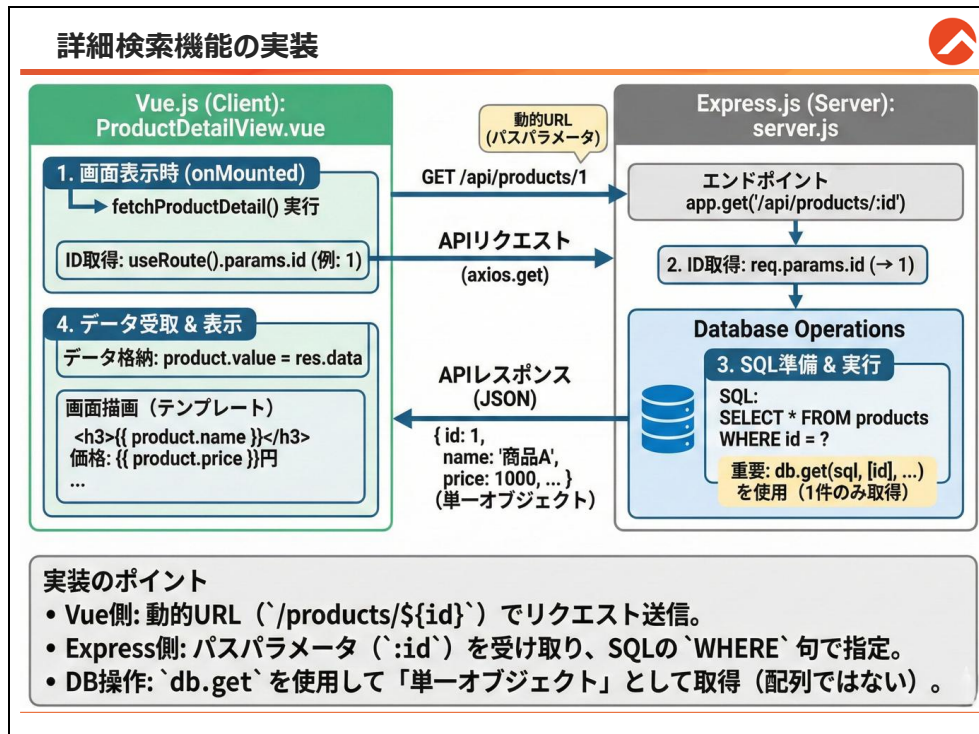
- keyword.value が「ペン」の場合
- 送信される URL: `http://localhost:3000/api/products2?keyword=ペン`

手動で `'?keyword=' + keyword.value` と文字列を結合する必要がないため、記述ミスが減り、特殊な文字(スペースや記号など)のエンコードも自動でやってくれるので安全です。

他のコードは、全件検索と全く一緒です。

以上、キーワード検索の実装方法の説明でした。

5.3.3. 詳細検索機能の実装



この節で実装する部分

4.6 節「パラメータ付きルート」では、以下までの実装方法を説明していました。

1. 一覧画面で商品名をクリックすると、URL が `/products/1` のように切り替わる。
2. 詳細画面 (ProductDetailView.vue) で、URL から ID (1 など) を取得する。
3. 画面には「商品 ID: 1 の詳細ページです」と表示されるのみ。

本節では、この「ID」を使ってサーバー (API) に問い合わせを行い、**取得した商品詳細データ (価格やカテゴリなど) を表示できるようにアップグレード**します。

Express 側での詳細検索 API 実装

まずは、ID を指定して商品を 1 件だけ取得する API を作成します。

既存のコードに以下のエンドポイントを追加します。

ch05-crud-api¥server.js

```

68:// :id の部分はパスパラメータとして動的に受け取ります
69:app.get('/api/products/:id', (req, res) => {
70:  // 1. URL パラメータから id を取得
71:  const id = req.params.id;
72:
73:  // 2. SQL の準備 (1 件だけ取得するので WHERE id = ?)
74:  const sql = 'SELECT * FROM products WHERE id = ?';
75:
76:  // 3. SQL 実行
77:  // db.get は「1 行だけ」取得するメソッドです (検索結果が 1 件の場合に使用)
78:  db.get(sql, [id], (err, row) => {
79:    if (err) {
80:      console.error(err);
81:      res.status(500).json({ error: 'Database error' });
82:      return;
83:    }
84:    // 4. 結果を JSON で返却 (配列ではなく、単一のオブジェクトが返ります)
85:    res.json(row);
86:  });
87:});

```

詳細検索の URL について

前節のキーワード検索では、URL の末尾に ?keyword=ペン を付け、req.query の中に格納される「クエリパラメータ」を使用しました。

しかし、詳細検索は商品名などに動的 URL を生成させ、URL にデータが付いてくるパターンです。

```

<RouterLink :to="/products/${p.id}">
  {{ p.name }}
</RouterLink>

```

このような動的 URL で呼び出される場合は、クエリパラメータとは異なる「パスパラメータ (Route Parameters)」という手法を使用します。

💡 構文の解説

パスパラメータの取得

```

68:// :id の部分はパスパラメータとして動的に受け取ります
69:app.get('/api/products/:id', (req, res) => {
70:  // 1. URL パラメータから id を取得
71:  const id = req.params.id;

```

Express.js では、URL のパス内にコロン : を付けることで、その部分を動的 URL のパラメータとして定義できます。

69 行のように、パスの中に :id と書くと、Express は「products/ の後ろに来る値を、id という名前の変数として扱う」と解釈します。そして値は、req.params オブジェクトの中に格納されます。

71 行のように、req.params.id と記述すると、動的 URL を取得できます。

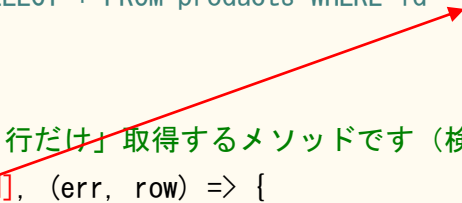
ID を取得できたら、次はデータベースから該当する商品を検索します。

SQL の構築と単一データの取得

```

.....
74: const sql = 'SELECT * FROM products WHERE id = ?';
75:
76: // 3. SQL 実行
77: // db.get は「1 行だけ」取得するメソッドです（検索結果が 1 件の場合に使用）
78: db.get(sql, [id], (err, row) => {
79:   if (err) {
.....

```



74 行の SQL 文ですが、今回は「特定の商品 1 つ」が欲しいので、WHERE 句で id を指定します。

78 行ですが、一覧検索（全件取得）とは異なるメソッドを使用している点に注目してください。

一覧取得のときは db.all を使いましたが、今回は db.get を使用しています。

ID(主キー)による検索は、データベースの仕組み上、**結果が「1 件」か「0 件(見つからない)」のどちらかしかありません**。db.get を使うと、配列ではなく、**単一オブジェクトとして取得**できますので、扱いしやすくなります。

db.get の第 2 引数には、db.all と同様に「?」に代入するデータを入れます。71 行で id を取得できているので、id を入れます。

その他のソースコードは、他の検索 API と同じです。

Vue 側の実装(ProductDetailView.vue の拡張)

4.6 節で作ったモック(ダミー)の状態から、API 通信を行う本格的なコンポーネントへ書き換えます。

変更の概要:

- Before (4.6 節): useRoute で ID を取って、そのまま表示していた。
- After (今回): useRoute で取った ID を使い、axios.get でデータを取得。取得したデータをリアクティブ変数 product に格納して表示する。

ch05-crud¥src¥views¥ProductDetailView.vue

```

14:// ルート情報の取得 (4.6 節と同じ)
15:const route = useRoute()
16:// URL パラメータから ID を取得 (文字列なので数値に変換)
17:const id = Number(route.params.id)
18:
20:// 初期値は null にしておき、データ取得後にオブジェクトが入るようにします
21:const product = ref<Product | null>(null) //配列ではなく単一オブジェクト
22:const loading = ref(true)
23:const error = ref<string | null>(null)
24:
25:// 詳細データ取得関数 (★追加)
26:const fetchProductDetail = async () => {
27:  try {
28:    // API リクエスト: 末尾に ID を付与して送信
29:    const res = await axios.get(`http://localhost:3000/api/products/${id}`)
30:    product.value = res.data
31:  } catch (err) {
32:    console.error(err)
33:    error.value = '商品データの取得に失敗しました。'
34:  } finally {
35:    loading.value = false
36:  }
37:}
39:// 画面表示時に実行
40:onMounted(() => {
41:  fetchProductDetail()
42:})
43:</script>

```

💡 構文の解説

自動実行の設定

```

26:const fetchProductDetail = async () => {
.....
39:// 画面表示時に実行
40:onMounted(() => {
41:  fetchProductDetail()
42:})

```

4.6 節では ID を表示するだけだったので即時表示できましたが、今回は API 通信が必要です。

40 行の onMounted を利用し、**ProductDetailView の画面の枠組みが表示された直後に 26 行の fetchProductDetail() を自動実行**するよう設定しています。

4.6 のコードの振り返り

```

15:const route = useRoute()
16:// URL パラメータから ID を取得（文字列なので数値に変換）
17:const id = Number(route.params.id)

```

この部分は、4.6 でも説明した通り、useRoute メソッドを利用して、RouterLink で送信された動的 URL の id を取得するコードです。

```

<RouterLink :to="/products/${p.id}">
  {{ p.name }}
</RouterLink>

```

リアクティブ変数を宣言

```

21:const product = ref<Product | null>(null) //配列ではなく単一オブジェクト
22:const loading = ref(true)
23:const error = ref<string | null>(null)

```

21-23 行目では、スクリプト内で使われる変数の初期化を行っていますが、商品データの product 変数に注目してください。

今までの検索画面では、複数の商品データを扱っていたため、「products」という名前で配列として宣言していました。しかし、今回は**単一オブジェクトを扱うため、「s」を付けないで「product」と命名**しました。

また、型定義も ref<Product[]>(配列)から ref<Product | null>(単一または null)に変わっている点も重要です。初期状態はデータがないため null を許容するようにしています。

API リクエスト

```

26:const fetchProductDetail = async () => {
27:  try {
28:    // API リクエスト : 末尾に ID を付与して送信
29:    const res = await axios.get(`http://localhost:3000/api/products/${id}`)
30:    product.value = res.data
    ~~以降、エラー処理やローディング関連処理~~

```

一覧画面では固定の URL でしたが、詳細画面では「どの商品か」を指定する ID が必要です。バッククォート「`」を使い、`${id}` の形式で変数を埋め込むことで、`http://localhost:3000/api/products/5` のようなリクエスト URL を動的に生成し、API へ送信できます。

```
app.get('/api/products/:id', (req, res) => {
```

送信したパスパラメータは、先ほど実装した、詳細検索の API で「/:id」で取得できます。

API からの通信が完了したら、30 行目以降のコードは、他の検索機能と同様ですので、割愛します。

取得したデータを画面に描画

```

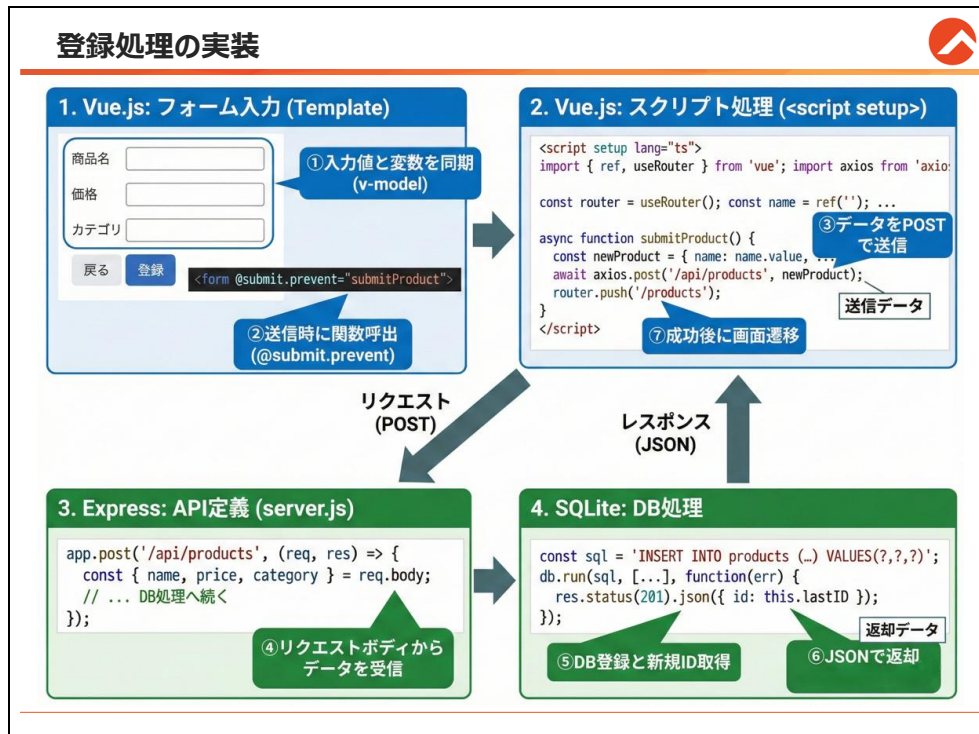
<!-- 上記は、ローディング画面やエラーメッセージの分岐 -->
58:<div v-else-if="product">
59:  <h2 class="mb-3">商品詳細</h2>
60:
61:  <div class="card">
62:    <div class="card-body">
63:      <h3 class="card-title">{{ product.name }}</h3>
64:      <hr>
65:      <div class="row mb-3">
66:        <div class="col-md-3 fw-bold">商品 ID</div>
67:        <div class="col-md-9">{{ product.id }}</div>
68:      </div>
69:      <div class="row mb-3">
70:        <div class="col-md-3 fw-bold">価格</div>
71:        <div class="col-md-9">{{ product.price }} 円</div>
72:      </div>
73:      <div class="row mb-3">
74:        <div class="col-md-3 fw-bold">カテゴリ</div>

```

```
75:      <div class="col-md-9">
76:        <span class="badge bg-info text-dark">{{ product.category }}</span>
77:      </div>
.....
```

REST API との通信で取得した 1 件の商品データは、product の中に格納されていますので、マスタッシュ構文 {{ }} で描画できます。配列ではないので、v-for を使う必要もなく簡単に描画ができます。

5.4. 登録処理の実装



登録フォームの作成

これまでは、検索機能を実装しました。ここからは、登録処理の実装方法を説明します。

まず、4章で空欄にしていた NewProductView.vue に、登録フォームを配置しておきます。

商品管理システム

ホーム 商品一覧 新規登録 ようこそ、〇〇さん

ダッシュボード

商品一覧

新規登録

商品一覧(検索機能付き)

新規商品登録

商品名 *

例: マウス

価格(円) *

0

カテゴリ

電子機器

戻る 登録

© 2025 TrainoVue Corporation

ch05-crud¥src¥views¥NewProductView.vue

```
.....
62:<form class="card p-4 shadow-sm" @submit.prevent="submitProduct">
63:
64:  <!-- 商品名 -->
65:  <div class="mb-3">
66:    <label class="form-label">商品名
        <span class="text-danger">*</span></label>
67:    <input type="text" class="form-control" v-model="name"
        placeholder="例: マウス" required>
68:  </div>
69:
70:  <!-- 価格 -->
71:  <div class="mb-3">
72:    <label class="form-label">価格(円)
        <span class="text-danger">*</span></label>
73:    <div class="input-group">
74:      <input type="number" class="form-control"
        v-model="price" placeholder="0" min="1" required>
75:    </div>
76:  </div>
77:
78:  <!-- カテゴリ（セレクトボックス） -->
79:  <div class="mb-3">
80:    <label class="form-label">カテゴリ</label>
81:    <select class="form-select" v-model="category">
82:      <option value="電子機器">電子機器</option>
83:      <option value="家具">家具</option>
84:      <option value="文房具">文房具</option>
85:      <option value="書籍">書籍</option>
86:      <option value="その他">その他</option>
87:    </select>
88:  </div>
89:
90:<!-- ボタンエリア(d-flex と justify-content-betweenはボタンを横並びにする)-->
91:  <div class="d-flex justify-content-between mt-4">
92:    <!-- 戻るボタン（履歴を一つ戻る） -->
```

```

93:    <button type="button" class="btn btn-secondary" @click="router.back()">
94:        戻る
95:    </button>
96:    <!-- 送信ボタン -->
97:    <button type="submit" class="btn btn-primary">
98:        登録
99:    </button>
100: </div>
101:</form>

```

💡 構文の解説

注目するポイントは、各入力項目に「v-model」が付与されている部分です。ユーザからの入力があれば、リアルタイムでスクリプト内に宣言した変数に代入されるはずです。

```

62:<form class="card p-4 shadow-sm col-md-6" @submit.prevent="submitProduct">
100:    <button type="submit" class="btn btn-primary">
101:        登録
102:    </button>

```

登録データの入力が完了したら、ユーザは 100 行目の「登録」の submit ボタンをクリックしますが、その時は、62 行目のように、**form 要素に「@submit.prevent=関数名」と付与し submit 発生時のイベント発火をさせます**。登録時は、**通常のクリックイベントではなく、この方法を使うのが公式でも推奨**されています。

※94 行目の router.back()は、後ほど説明します

新規登録 API の実装 (POST /api/products)

引き続き、サーバーサイド (Express) に、データベースへ新しい商品を登録するための API を追加します。ここでは、HTTP メソッドとして POST を使用し、登録後の ID をクライアントに返す処理を実装します。

ch05-crud-api¥server.js

```
89:// 【5.4】新規商品登録 API (POST /api/products)
90:app.post('/api/products', (req, res) => {
91:  // 1. リクエストボディからデータを取り出す
92:  const name = req.body.name;
93:  const price = req.body.price;
94:  const category = req.body.category;
95:
96:  // 2. サーバー側でのバリデーション (必須チェック)
97:  if (!name || !price || !category) {
98:    // 400 Bad Request: クライアント側のリクエストミス
99:    res.status(400).json({ error: 'データに誤りがあります' });
100:    return;
101:  }
102:
103:  // 3. SQL の準備
104:  const sql = 'INSERT INTO products (name, price, category) VALUES (?, ?, ?)';
105:  const params = [name, price, category];
106:
107:  // 4. SQL 実行
108:// ここではアロー関数 (err) => {} ではなく function(err) {} を使います
109:  db.run(sql, params, function (err) {
110:    if (err) {
111:      console.error('登録エラー:', err);
112:      res.status(500).json({ error: 'Database error' });
113:      return;
114:    }
115:    // 5. 登録成功レスポンス (新しく発番された ID を返却)
116:    res.status(201).json({ id: this.lastID });
117:  });
118:});
```

💡 構文の解説

エンドポイントの定義とデータの受け取り

```

90: app.post('/api/products', (req, res) => {
91:   // 1. リクエストボディからデータを取り出す
92:   const name = req.body.name;
93:   const price = req.body.price;
94:   const category = req.body.category;

```

90 行目ですが、データの登録には、HTTP メソッドの POST を使用します。

検索処理用の GET メソッド(取得)とは異なり、サーバー上のデータを変更・追加する際に用いられます。

Vue 側から送られる新規登録の入力データは、「req.body オブジェクト」の中に格納されます。

92-94 行目では、name, price, category の各プロパティを個別に変数に取り出しています。

```

104: const sql = 'INSERT INTO products (name, price, category) VALUES (?, ?, ?)';
105: const params = [name, price, category];
.....
108: // ここではアロー関数 (err) => {} ではなく function(err) {} を使います
109: db.run(sql, params, function (err) {

```

104 行目は、新規データを追加するための SQL 構文です。ここでもプレースホルダー ? を使用して、SQL インジェクション対策を行っています。

109 行目のように、データの更新系処理 (INSERT, UPDATE, DELETE) には db.run メソッドを使用します。ここでは、コールバック関数にアロー関数 (err) => {} ではなく、従来の function(err) {} を使用しています。

※ SQLite3 ライブラリの仕様上、実行結果(自動採番された ID など)は this コンテキストに含まれます。アロー関数では this が正しく参照できないため、従来の関数構文を使う必要があります。

```

116: res.status(201).json({id: this.lastID });

```

登録が成功した場合、「新しいリソースが作成された」ことを意味するステータスコード 201 を返します。同時に、自動採番で生成された ID を JSON で返すことで、クライアント側ですぐにそのデータを活用(詳細画面への遷移など)できるようになります。

登録処理の API の呼び出し

ユーザがフォームに入力した内容を受け取り、API へ送信して登録を行い、完了後に画面を遷移させる一連の流れを説明します。

ch05-crud¥src¥views¥NewProductView.vue

```
4: import { useRouter } from 'vue-router'
5:
6: // ルーターインスタンスの取得
7: const router = useRouter()
8:
9: // 入力データを管理するリアクティブ変数
10: const name = ref('')
11: const price = ref<number | null>(null) // 初期値は空にしたいので null 許容
12: const category = ref(' 電子機器') // セレクトボックスの初期値
13:
14: // エラーメッセージ用
15: const error = ref<string | null>(null)
16:
17: // フォーム送信処理関数
18: const submitProduct = async () => {
19:   // 1. 簡易バリデーション（入力チェック）
20:   if (!name.value || !price.value || price.value <= 0) {
21:     error.value = '商品名と正しい価格を入力してください。'
22:     return
23:   }
24:
25:   try {
26:     // 2. 送信データの作成
27:     const newProduct = {
28:       name: name.value,
29:       price: price.value,
30:       category: category.value
31:     }
32:
33:     // 3. POST リクエスト送信
34:     // 第 2 引数に送信したいデータオブジェクトを渡します
```

```

35:   const res = await axios.post(
      'http://localhost:3000/api/products', newProduct)
37:   // 4. 成功時の処理
38:   const newId = res.data.id
39:   alert(`商品が登録されました。ID: ${newId}`)
40:
41:   // 一覧画面へ戻る (SPA 的な画面遷移)
42:   router.push('/products')
43:   //router.push(`/products/${newId}`) //登録直後に詳細画面へ遷移する場合
.....

```

💡 構文の解説

useRouter の活用

```

4: import { useRouter } from 'vue-router'
5:
6: // ルーターインスタンスの取得
7: const router = useRouter()
.....
42:   router.push('/products')
.....
93:   <!-- 戻るボタン (履歴を一つ戻る) -->
94:   <button type="button" class="btn btn-secondary" @click="router.back()">

```

4 行目で import している useRouter は、Vue Router の機能 (ページ遷移など) を JavaScript コード内から利用するために必要です。

ページ遷移をさせたい時、42 行目の router.push('/URL') のようにメソッドを呼び出すことで、ユーザを別の画面へプログラマ的に移動させることができます。これは SPA (シングルページアプリケーション) における画面遷移の基本テクニックです。

94 行目の router.back() は、ブラウザの『戻る』ボタンをクリックした時と同じ動作をします。

送信処理開始と、入力チェック

```

18: const submitProduct = async () => {
19:   // 1. 簡易バリデーション（入力チェック）
20:   if (!name.value || !price.value || price.value <= 0) {
21:     error.value = '商品名と正しい価格を入力してください。'
22:     return
23:   }

```

submit イベントが発生すると、送信処理が開始されます。

20-23 行目では、サーバーに送る前に、最低限のチェックを行います。無駄な通信を防ぎ、ユーザーに即座にフィードバックを返すために重要です。

データの整形とデータ送信

```

27:   const newProduct = {
28:     name: name.value,
29:     price: price.value,
30:     category: category.value
31:   }
35:   const res = await axios.post(
                                'http://localhost:3000/api/products', newProduct)

```

```

app.post('/api/products', (req, res) => {
  // 1. リクエストボディからデータを取り出す
  const name = req.body.name;
  const price = req.body.price;
  const category = req.body.category;

```

登録処理の送信をする前に、API が期待する形に入力値をまとめておく必要があります。

そのためのコードが、27-31 行目のコードです。

35 行目のように、データの登録には POST メソッドを使います。第二引数に送りたいデータ (newProduct) を渡します。

そうすることで、API 側の req の中にデータが送信され、req.body でデータを扱うことができるようになります。

通信成功後のデータの取得

```

38:   const newId = res.data.id
39:   alert(`商品が登録されました。ID: ${newId}`)

```

API から返ってきた res.data には、サーバー側で生成された id が含まれています。これを取得してアラートに表示しています。

登録完了後の画面遷移

```
42: router.push('/products')
43: //router.push(`/products/${newId}`) //登録直後に詳細画面へ遷移する場合
```

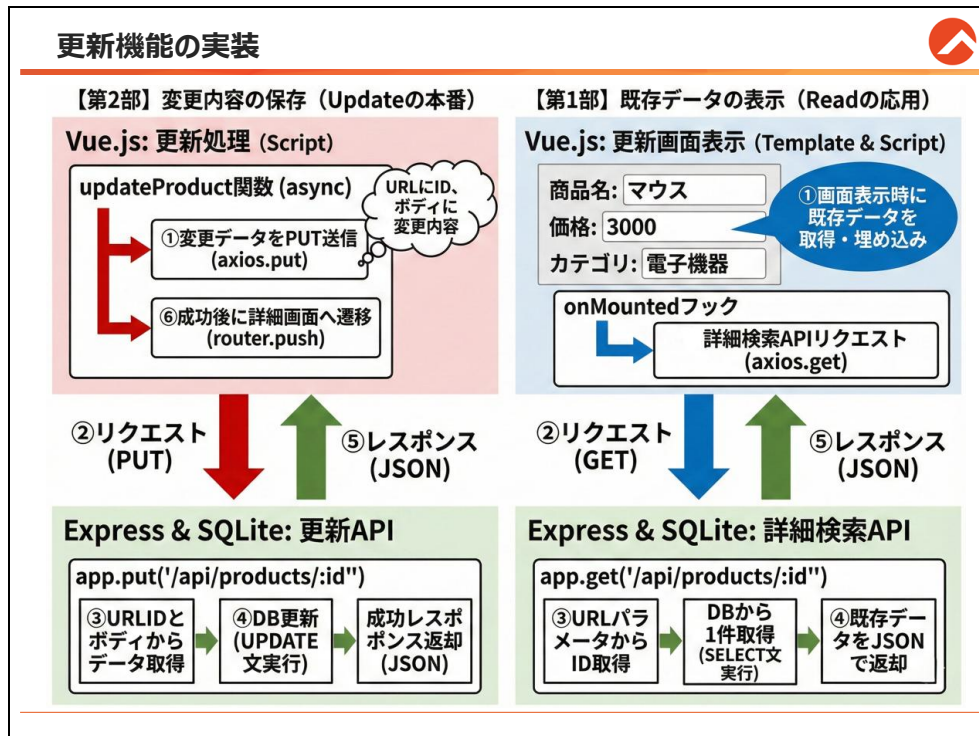
処理が成功したら、`router.push('/products')` を実行して一覧画面へ移動させます。これで、ユーザは登録したデータをすぐに確認できます。

コメントアウトされている `router.push(`/products/${newId}`)` を使うと、登録直後にその商品の詳細画面へ直接遷移させることも可能です。

The image shows two parts of the application. On the left is the '新規商品登録' (New Product Registration) form. It has three input fields: '商品名 *' (Product Name) with the value '子供ベット' (Children's Bed), '価格(円) *' (Price in Yen) with the value '60000', and a 'カテゴリ' (Category) dropdown menu with '家具' (Furniture) selected. There are '戻る' (Back) and '登録' (Register) buttons. On the right is a confirmation modal titled 'localhost:5173 の内容' (Content of localhost:5173). It says '商品が登録されました。ID: 5' (Product registered successfully. ID: 5). There is an 'OK' button. A red arrow points from the '登録' button in the form to the modal.

商品一覧			
商品番号	商品名	価格	カテゴリ
1	ノートパソコン	120000 円	電子機器
2	スマートフォン	90000 円	電子機器
3	オフィスチェア	25000 円	家具
4	ボールペンセット	800 円	文房具
5	子供ベット	60000 円	家具

5.5. 更新機能の実装



更新機能の全体像

CRUD の「U (Update)」にあたる、既存データの更新機能を実装します。これまで学習した「参照 (Read)」や「登録 (Create)」と異なり、**更新機能は「2つの異なるアクション」の組み合わせで成り立っている**点が重要です。ここを理解せずに実装を始めると、データの流れて混乱してしまいます。

更新処理は、ユーザ体験 (UX) の観点から以下の2段階で構成されます。

【第1部】既存データの表示 (Read の応用)

いきなり空のフォームを表示して修正を促しても、ユーザは困ってしまいます。編集画面を開いた瞬間、「**現在データベースに保存されている情報**」がフォームにあらかじめ入力されている状態を作る必要があります。

すでに作成済みの 詳細取得 API (GET /api/products/:id) を再利用します。

【第2部】変更内容の保存 (Update の本番)

ユーザがフォームの内容を書き換えて「保存」ボタンを押した時、変更後のデータをサーバーに送り、データベースを上書き保存します。

ここで、更新用 API (PUT /api/products/:id) が登場します。

本節では、まずバックエンド(API)側で更新を受け入れる準備をし、その後にフロントエンド(Vue)側で画面遷移とデータ連携を作り込んでいきます。

Express 側での更新 API 実装

既存のデータを上書きするためのAPIを作成します。server.js に以下のコードを追記します。

ch05-crud-api/server.js

```
121:// :id で更新対象を指定し、body で変更内容を受け取ります
122:app.put('/api/products/:id', (req, res) => {
123:  // 1. URL パラメータから id を取得（詳細検索と同じ）
124:  // これで「どの商品を更新するか」を特定します
125:  const id = req.params.id;
127:  // 2. リクエストボディから変更後のデータを取り出す（新規登録と同じ）
128:  // これが「新しい商品の情報」になります
129:  const name = req.body.name;
130:  const price = req.body.price;
131:  const category = req.body.category;
132:
133:  // 3. サーバー側でのバリデーション
134:  if (!name || !price || !category) {
135:    res.status(400).json({ error: 'データに誤りがあります' });
136:    return;
137:  }
139:  // 4. SQL の準備
140:  // UPDATE 文を使用し、WHERE 句で id を指定して特定の行だけを書き換えます
141:  const sql =
    'UPDATE products SET name = ?, price = ?, category = ? WHERE id = ?';
142:  const params = [name, price, category, id];
143:
144:  // 5. SQL 実行
145:  //更新処理なので db.run を使用します(function(err)構文で実行結果を取得)
146:  db.run(sql, params, function (err) {
147:    if (err) {
148:      console.error('更新エラー:', err);
149:      res.status(500).json({ error: 'Database error' });
150:      return;
151:    }
```

```

152:    // 6. 成功レスポンス
153:    res.json({id: id}); // 更新した商品の ID を返す
154:  });

```

💡 構文の解説

この API は、これまでに作った 2 つの機能のような構造をしています。

1. 「どの商品をも?」: URL の ID で指定します。これは **詳細検索 API** と同じ仕組み(パスパラメータ)です。
2. 「どう書き換える?」: ユーザからのデータを受け取ります。これは **新規登録 API** と同じ仕組み(リクエストボディ)です。

パラメータの受け取り

```

122: app.put('/api/products/:id', (req, res) => {
123:   // 1. URL パラメータから id を取得 (詳細検索と同じ)
124:   // これで「どの商品を更新するか」を特定します
125:   const id = req.params.id;
127:   // 2. リクエストボディから変更後のデータを取り出す (新規登録と同じ)
128:   // これが「新しい商品の情報」になります
129:   const name = req.body.name;
130:   const price = req.body.price;
131:   const category = req.body.category;

```

122 行のように、データの「更新」には、HTTP メソッドの **PUT** を使用するのが一般的です。

そして、必ず**更新対象を指定する必要がある**ため、URL から **ID を取得**する必要がありますので、125 行目で**パスパラメータ**から取得をしています。この部分は**詳細検索 API**と同じです。

129-131 行目は、Vue から送られてきた「変更後のデータ」を受け取るために、**リクエストボディからデータを取得**しています。この部分は、**新規登録 API**と同じです。

UPDATE 文の定義

```

141: const sql =
      'UPDATE products SET name = ?, price = ?, category = ? WHERE id = ?';
142: const params = [name, price, category, id];

```

141 行目の UPDATE 文は、WHERE id = ? を忘れると、テーブル内の全商品が同じ名前に書き換わってしまいますので、注意しましょう。

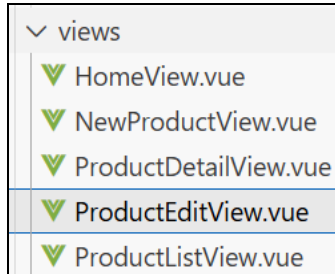
142 行目の **params 配列の順番も、SQL の ? の順番 (name, price, category, id) と厳密に合わせる**必要があります。

```
146: db.run(sql, params, function (err) {  
.....
```

146 行以降の解説は、登録処理の API と全く一緒ですので、割愛します。

更新画面コンポーネントの作成とルーティング

src/views フォルダに ProductEditView.vue を新規作成します。



ch05-crud¥src¥router¥index.ts

```
.....
10:import ProductEditView from '../views/ProductEditView.vue'
.....
19:{ path: '/products/edit/:id', name: 'product-edit', component:
ProductEditView}
```

更新画面へアクセスできるようにルーター設定を追加します。URL は `/products/edit/:id` とし、詳細画面と同じく ID をパスパラメータで受け取れるようにします。

詳細画面への「編集」ボタン追加

更新画面に遷移させるボタンをどこに配置するかは、いくつかのパターンがありますが、今回は一般的なパターンである、**詳細画面からのボタンで更新する例**で説明します。



```
87:<RouterLink:to="\`/products/edit/${product.id}`" class="btn btn-primary ms-2">
88:      編集する
89:</RouterLink>
```

詳細画面には、商品 ID のデータが既にあるので、それを利用して、遷移先 URL に付与しています。

更新画面コンポーネントの実装【第 1 部 既存データの表示】

ProductEditView.vue について、まずは、既存データの表示の部分を説明します。

「初期表示時にデータを取得して埋め込む」という処理が必要ですので、詳細検索のコードとほぼ同じです。

ch05-crud¥src¥views¥ProductEditView.vue

```
1:const router = useRouter ()
2:const route = useRoute ()
3:
4:// URL から更新対象の ID を取得
5:const id = route.params.id
6:
7:// 入力データ（初期値は空だが、onMounted でデータが入る）
8:const name = ref('')
9:const price = ref<number | null>(null)
10:const category = ref(' 電子機器')
11:
12:// エラーメッセージ用
13:const error = ref<string | null>(null)
14:
15:/**
16: * 【第 1 部】既存データの取得（初期表示用）
17: * 詳細画面と同じ API を使って、現在の情報を取得します
18: */
19:const getProduct = async () => {
20:  try {
21:    const res = await
      axios.get(`http://localhost:3000/api/products/${id}`)
22:    const product = res.data
23:
24:    //取得したデータをフォームの変数にセット（これで画面に表示されます）
25:    name.value = product.name
26:    price.value = product.price
27:    category.value = product.category
28:
29:  } catch (err) {
30:    console.error(err)
31:    error.value = '商品データの取得に失敗しました。'
```

```

32:  }
33:}
.....
67:// 画面が表示されたら、まず既存データを取得する
68:mounted() => {
69:  getProduct()
70:})
71:</script>
.....
90:<!-- 商品名 -->
91: <div class="mb-3">
92:   <label class="form-label">商品名
       <span class="text-danger">*</span></label>
93:   <input type="text" class="form-control" v-model="name" required>
94: </div>
.....

```

💡 構文の解説

詳細画面から、「編集」ボタンをクリックすることで、更新画面へ遷移します。

その後、68 行目の `mounted()` によって、詳細検索の API を呼び出す関数が実行されます。

編集対象の商品データを取得し、HTML に埋め込む

```

21:  const res = await
       axios.get(`http://localhost:3000/api/products/${id}`)
22:  const product = res.data
24: //取得したデータをフォームの変数にセット（これで画面に表示されます）
25:  name.value = product.name
26:  price.value = product.price
27:  category.value = product.category
.....
92: <label class="form-label">商品名
       <span class="text-danger">*</span></label>
93: <input type="text" class="form-control" v-model="name" required>

```

21 行で詳細検索の API を呼び出し、結果は 22 行の `product` 変数に入ります。

その後、リアクティブ変数の `name/price/category` にデータを更新します。

すると、HTML の各フォームは、93 行目のように v-model で双方向データバインディングをしているため、自動的に以下のようにフォーム内にデータが埋め込まれます。HTML の value 属性を使う必要はありません。

商品情報の編集

商品名 *

オフィスチェア

価格(円) *

25000

カテゴリ

家具

戻る

保存

更新画面コンポーネントの実装【第 2 部 変更内容の保存】

ch05-crud¥src¥views¥ProductEditView.vue

```
: * 登録処理と似ていますが、メソッドが PUT になっています
44: const updateProduct = async () => {
45:   // 簡易バリデーション
46:   if (!name.value || !price.value || price.value <= 0) {
47:     error.value = '商品名と正しい価格を入力してください。'
48:     return
49:   }
50:
51:   try {
52:     const updatedProduct = {
53:       name: name.value,
54:       price: price.value,
55:       category: category.value
56:     }
57:
58:     // PUT リクエスト送信 (URL に ID を含めるのを忘れずに)
```

```

59:   await axios.put(
      `http://localhost:3000/api/products/${id}`, updatedProduct)
60:
61:   alert('商品を更新しました。')
62:
63:   // 更新後は、その商品の「詳細画面」に戻るのが一般的です
64:   router.push(`/products/${id}`)
65:
66: } catch (err) {
67:   console.error(err)
68:   error.value = '更新に失敗しました。'
69: }
70:}
.....
87:<!-- フォーム（送信時に updateProduct を実行） -->
88:<form class="card p-4 shadow-sm col-md-6" @submit.prevent="updateProduct">
.....
122: <!-- 送信ボタン -->
123:   <button type="submit" class="btn btn-primary">
124:     保存
125:   </button>
.....

```

💡 構文の解説

第2部は、新規登録のスクリプトと同じ部分が多いので、重要な部分だけ解説します。

ユーザがデータを編集した後、123行目の「保存」ボタンをクリックすると、88行目の submit イベントより、44行目の更新処理を行う関数が呼ばれます。

編集済みのデータを更新後、API 呼び出し

```

52:   const updatedProduct = {
53:     name: name.value,
54:     price: price.value,
55:     category: category.value
56:   }
.....
59:   await axios.put(
      `http://localhost:3000/api/products/${id}`, updatedProduct)

```

その後、52-56 行目で**編集済みのデータの更新**を行います。

v-model で**各変数が連動されている**ので、可能なことです。

59 行目で更新処理の API が呼び出す際は、**axios.put** を利用します。

更新処理の際は、必ず ID を渡さないといけないので、URL に id を付けて送信しました。

第 2 引数には、編集済みのデータをまとめたオブジェクトを入れます。

更新完了後の画面遷移

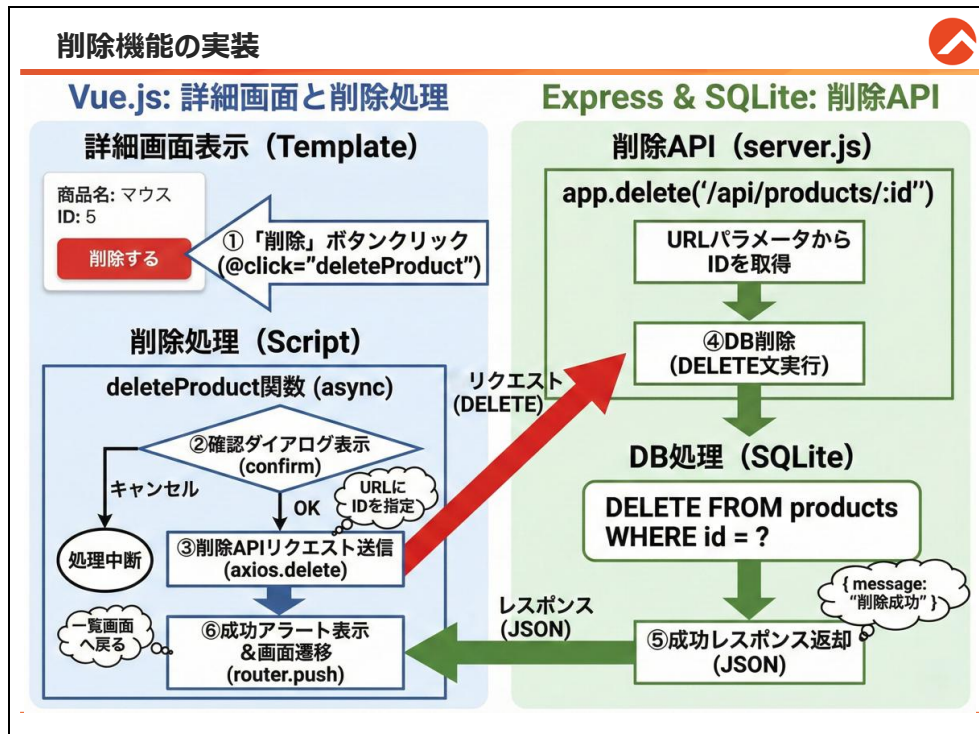
```
63: // 更新後は、その商品の「詳細画面」に戻るのが一般的です
64: router.push(`/products/${id}`)
```

処理が完了したら、更新後の詳細画面を表示する流れが自然ですので、router.pushを使って画面遷移させます。

The image displays two side-by-side screenshots of a web application interface. The left screenshot, titled '商品情報の編集' (Edit Product Information), shows a form with three input fields: '商品名 *' (Product Name) containing 'オフィスチェア修正', '価格(円) *' (Price) containing '9999', and 'カテゴリ' (Category) with a dropdown menu showing 'その他'. Below the form are two buttons: '戻る' (Back) and '保存' (Save). The right screenshot, titled '商品詳細' (Product Details), shows the details for 'オフィスチェア修正'. It lists '商品ID' (Product ID) as 3, '価格' (Price) as 9999 円, and 'カテゴリ' (Category) as 'その他'. At the bottom are two buttons: '一覧に戻る' (Return to List) and '編集する' (Edit). A red arrow points from the '保存' button in the left form to the '商品詳細' page in the right screenshot.

以上、更新処理の実装でした。

5.6. 削除機能の実装



削除機能の全体像

CRUD の最後、データの削除機能 (Delete) を実装します。削除機能は、データそのものを消去してしまう危険な操作です。そのため、**実装においては「誤操作の防止」を考慮**する必要があります。

今回は以下のフローで実装します。

1. 詳細画面に「削除」ボタンを配置する。
2. ボタンが押されたら、JavaScript の確認ダイアログ (confirm) を出して削除確認。
3. OK が押されたら、API へ削除リクエスト (DELETE) を送信する。
4. 削除が完了したら、そのページはもう存在しないため、商品一覧画面へ戻る。

Express 側での削除 API 実装

ID を指定してデータを削除するための API を作成します。server.js に以下のコードを追記します。

ch05-crud-api/server.js

```

158: app.delete('/api/products/:id', (req, res) => {
159:   // 1. URL パラメータから id を取得
160:   const id = req.params.id;
161:
162:   // 2. SQL の準備
163:   // 特定の ID の行を物理削除します
164:   const sql = 'DELETE FROM products WHERE id = ?';
165:
166:   // 3. SQL 実行
167:   db.run(sql, [id], function (err) {
168:     if (err) {
169:       console.error('削除エラー:', err);
170:       res.status(500).json({ error: 'Database error' });
171:       return;
172:     }
173:     // 4. 成功レスポンス
174:     res.json({ message: '削除成功' });
175:   });
176: });

```

💡 構文の解説

```

158: app.delete('/api/products/:id', (req, res) => {
160:   const id = req.params.id;

```

リソースの削除には、HTTP メソッドの DELETE を使用します。
UPDATE と同様に、削除処理も ID を使って対象を決めて削除をしますので、パスパラメータから ID を取得しています。

```

164: const sql = 'DELETE FROM products WHERE id = ?';
167: db.run(sql, [id], function (err) {

```

削除の際は、ID を条件に入れて、db.run で SQL 文を実行させます。その後は割愛します。

Vue 側での削除機能の実装

詳細画面 (ProductDetailView.vue) に削除ボタンと、クリック時の処理を追加します。

ch05-crud¥src¥views¥ProductDetailView.vue

```

2: import { useRoute, RouterLink, useRouter } from 'vue-router'
.....
13: // 【5.6】 削除後の画面遷移用
14: const router = useRouter()
.....
    //商品削除処理
    //削除ボタンがクリックされたときに実行されます
46: const deleteProduct = async () => {
47:   // 1. 削除確認ダイアログの表示
48:   // 誤操作を防ぐため、必ずユーザーに確認を求めます
49:   const isConfirmed = confirm('本当にこの商品を削除してもよろしいですか?')
50:
51:   // キャンセルが押された場合は、処理を中断して終了します
52:   if (!isConfirmed) {
53:     return
54:   }
55:
56:   try {
57:     // 2. DELETE リクエスト送信
58:     // 削除対象の ID を URL に含めます
59:     await axios.delete(`http://localhost:3000/api/products/${id}`)
60:
61:     alert('商品を削除しました。')
62:
63:     // 3. 画面遷移
64:     // 削除された商品の詳細画面には留まらないため、一覧画面へ戻します
65:     router.push('/products')
.....
    <!--削除ボタン-->
123: <button class="btn btn-danger ms-2" @click="deleteProduct">
124:   削除する
125: </button>
.....

```

💡 構文の解説

削除ボタンの配置

```

123: <button class="btn btn-danger ms-2" @click="deleteProduct">
124:   削除する
125: </button>

```

「編集する」ボタンの横に、「削除する」ボタンを追加します。削除は画面の切り替えが不要で、詳細画面の中で削除 API を呼び出せば良いので、クリックイベントを付与しています。

削除確認ダイアログ表示

```

46: const deleteProduct = async () => {
48:   // 誤操作を防ぐため、必ずユーザーに確認を求めます
49:   const isConfirmed = confirm('本当にこの商品を削除してもよろしいですか?')
50:
51:   // キャンセルが押された場合は、処理を中断して終了します
52:   if (!isConfirmed) {
53:     return
54:   }

```

クリックイベントにより、46 行目から削除処理が開始されます。ただ、削除処理は誤操作が致命的になるので、JavaScript の confirm メソッドを使い、確認画面を表示させます。

49 行目の confirm メソッドの戻り値は、「OK」クリックで true、「キャンセル」クリックで false になるので、52-54 行まで簡単な分岐を作り、誤操作を防止します。

削除 API の呼び出しと、一覧画面へ遷移

```
59:   await axios.delete(`http://localhost:3000/api/products/${id}`)  
61:   alert('商品を削除しました。')  
64:   // 削除された商品の詳細画面には留まれないため、一覧画面へ戻します  
65:   router.push('/products')
```

削除処理は、HTTP の DELETE メソッドで呼び出します。削除処理のためには、ID が必要ですので、ID を URL に付けて送信します。

削除処理では、削除されたデータそのものを返す必要はありません。61 行目のように、「成功した」という事実が伝われば十分です。

削除後は、開いている商品の詳細画面も開けなくなるため、65 行目のように、一覧ページへ遷移するコードが必須です。

商品一覧

商品番号	商品名	価格	カテゴリ
1	ノートパソコン	120000 円	電子機器
2	スマートフォン	90000 円	電子機器
3	オフィスチェア修正	9999 円	その他
4	ボールペンセット	800 円	文房具

以上、削除機能の実装方法でした。

章のまとめ

章のまとめ



- REST APIの役割と開発環境の全体像
- REST API環境構築
- Expressで検索機能のAPI実装
 - 一覧画面の実装
 - キーワード検索機能の実装
 - 詳細検索機能の実装
- 登録処理の実装
- 更新処理の実装
- 削除処理の実装

この章では、REST API と連携した SPA を構築する方法についてご説明しました。

Vue.js 3 によるフロントエンド開発入門
～ 作って理解する SPA (TypeScript 対応)～

無断複写・複製の禁止

本書は著作権法上の保護を受けています。
本書の全部または一部を無断で複写複製することは
禁じられています。

2025 年 11 月 29 日	第 1 版	発行
2026 年 3 月 15 日	第 2 版	改訂

著 者
発行者

トレノケート株式会社
〒 163-6020
東京都新宿区西新宿 6 丁目 8 番 1 号
住友不動産新宿オークタワー 20 階
電話 03-3347-9686 FAX 03-3347-9699

<http://www.trainocate.co.jp/>

落丁本、乱丁本はお取り替えいたします。

© 2026 Trainocate Japan, Ltd.

WSC0134G-T1- 02